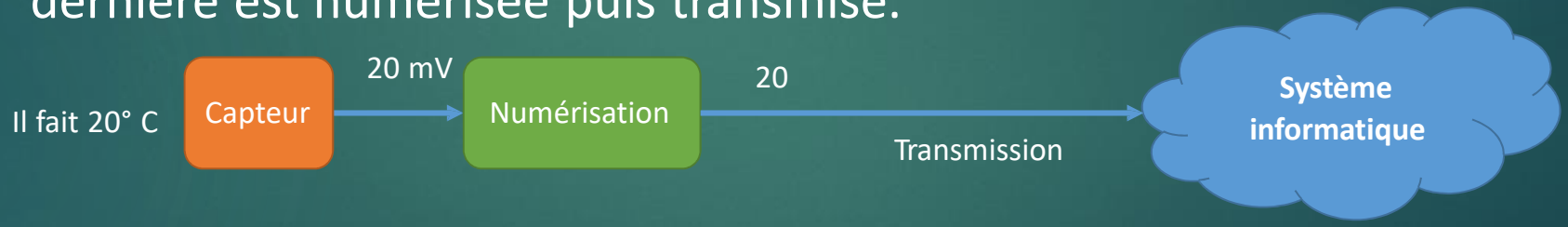
Concepts des entrées sorties

Chap Capteurs

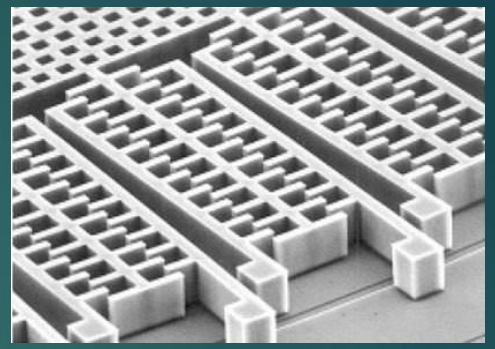
Actionneurs et contrôleurs de périphériques

1 Les capteurs

- Ils permettent de **traduire une grandeur physique en un signal électrique**. Ce dernier est ensuite **numérisé** pour être **transmis**
- au système informatique.
  Par exemple : un capteur de température permet de traduire l'amplitude de la température en une tension électrique. Cette dernière est numérisée puis transmise.



1 Capteurs



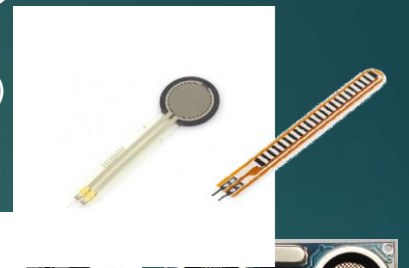
- Principes
  - Transforme une grandeur physique en :
    - Une grandeur électrique : tension ou courant

    ou
    - Un temps : durée ou fréquence
  - La transmet sous forme numérique (binaire) ou analogique (tension)

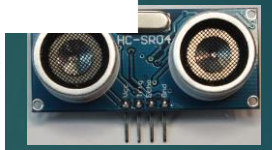
- Moyens :
  - Mécanique (contact, résistance ou capacité variable)
  - Piézoélectrique (tension induite par la déformation d'un cristal)
  - Effet Hall (tension induite par un champ magnétique)
  - Capacitif (tactile ou chimique)
  - Rayonnement (radio, infrarouge, laser)
  - Ultra sons (puissance, temps de propagation)
  - Lumière (phototransistor ou photorésistance)
  - Chimique (électrodes)

2 Types de capteurs

- Déplacement
  - Interrupteur, potentiomètre, joystick (contact ou résistance mécanique)
  - Tactile (contact ou décharge capacitive par contact)
  - Pression, Force, Flexion (résistance variable ou effet piézoélectrique)
- Présence
  - Passage, mouvement, vibration (laser, IR ou radar)
  - Proximité, distance (laser, IR ou ultra sons par mesure de temps)
- Position
  - Roue codeuse (contact)
  - GPS (réception de satellites)
  - Accélération (capacitif mécanique ou effet piézoélectrique)
  - Inclinaison (capacitif mécanique)
- Rayonnement
  - Magnétisme (effet Hall)
  - Ondes (radio, IR)
  - Lumière, couleur et UV (phototransistor, photorésistance)
- Environnement
  - Température (résistance variable)
  - Humidité (capacitif chimique)
  - Anémomètre (mécanique + magnétique)
  - Caméra (phototransistors)
  - Son, US (capacitif mécanique ou effet piézoélectrique)
  - Chimique (électrodes pour gaz ou liquides)



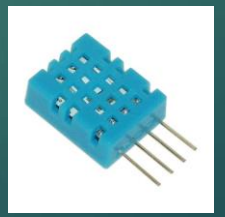






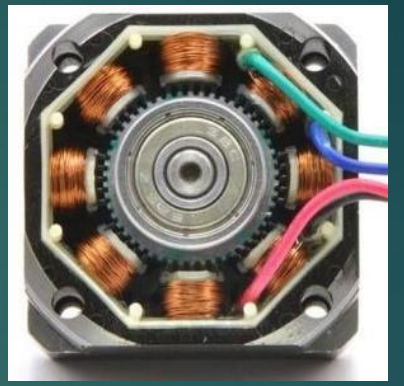








3 Actionneurs



- Principe
  Transforme une information reçue sous forme :
    - numérique (binaire)

    ou
    - analogique (tension)

    ou
    - temporelle (durée et fréquence)

  en :
    - Un mouvement (translation, rotation)
    - Un affichage (écran, témoin lumineux ou sonore)
    - Une émission (ondes, lumière, IR, chaleur)

- Moyens
  - Mécanique et magnétique (moteur, électroaimant)
  - Piézoélectrique (vibreur)
  - Emission de rayonnements (radio, infrarouge, laser)
  - Lumière (DEL, écran)

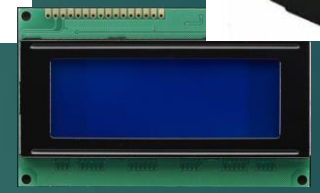
Types d'actionneurs

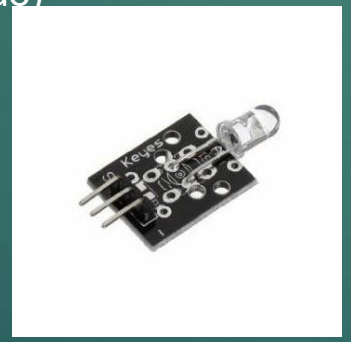


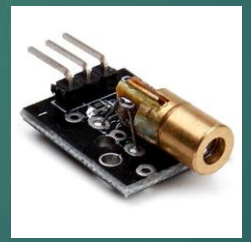
- Mouvement
  - Electroaimant, Electrovanne, Relais (magnétique)
  - Moteur à courant continu (contrôle de vitesse par courant ou temporelle)
  - Moteur linéaire (comme le précédent mais déplacement linéaire)
  - Moteur pas à pas (position définie par commande binaire)
  - Servomoteur (position asservie par commande temporelle)
- Indicateur
  - Haut parleur, vibreur (mécanique ou piézoélectrique)
  - Ecran (texte ou graphique), DEL ou afficheur (optoélectronique)
- Emetteur
  - Radio
  - Infrarouge
  - Ultra sons
  - Laser
  - Chaleur











4 Caractéristiques des capteurs/actionneurs

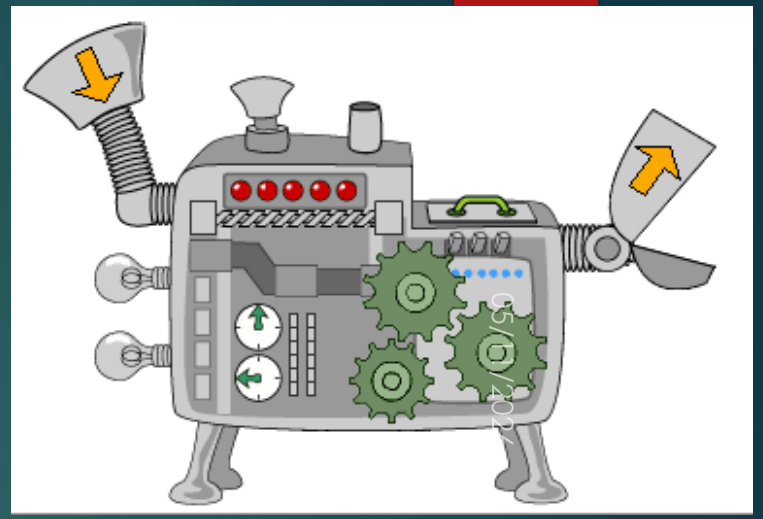
- Précision
  - Un capteur de température donne-t-il une mesure au ½°? au 1/10°?
  - Un servomoteur peut-il être placé au ½°près ? Au degré près ?
- Sensibilité
  - Un capteur de température donne-t-il une mesure différente si la température change de ½°? de 1/10°?
  - Un servomoteur bouge-t-il si on veut le déplacer de 1°? de ½°?
- Fiabilité
  - Un capteur de température qui mesure 14,1°pour 14°donnera-t-il toujours 14,1°une heure plus tard ? Le lendemain ? Dans un an ?
- Limites
  - Un capteur de température supporte-t-il une température de 100°? De 1000°?
  - Un servomoteur peut-il soulever une charge de 500 g ? De 10 Kg ?
- Prix
  - Très lié à la précision, la sensibilité, la fiabilité et les limites. Par exemple on trouve des servomoteurs à 2000 et d'autres à 1500 000 FCFA
- Type d'information envoyée/reçue
  - Numérique
  - Analogique
  - Temporelle
- Connexion
  - Directe
  - Par connexion spécialisé (SPI, I2C, I2S)





Contrôleurs de périphériques

Entrees et sorties



- Les périphériques d'E / S sont connectés à l'ordinateur
  - Qu'est-ce que ça veut dire?
  - Comment le processeur interagit-il avec les périphériques ?
  - Qu'est-ce que cela signifie pour le programmeur ?

Rappels architecture

- Système à microcontroleurs
  - µP+Coprocesseurs+Cache+Bus+Mémoires+Contrôleurs de périphériques
  - Co-processeurs : FPU, GPU,DUAL Core, quad-core
  - Unités Speciales : MMU (Mémoire Cache, Mémoire virtuelle), DEBUG, POWER, Clock, Watchdog, PIC (Programmable Interrupt controller)
  - Mémoires : SRAM, FLASH, ROM, EEPROM, DRAM
  - Controleurs de périphériques (Interface périphériques) : TIMER, GPIO, UART, I2C, SPI, ADC, DAC, PWM, RTC, …
  - BUS: DONNEES, ADRESSES, COMMANDES, parfois besoin d'un contrôleur de bus.

5 Types de contrôleurs de périphériques

Entrées (capteurs)

- Numériques
  - Lignes binaires en entrée (lecture 0 ou 1)
- Analogiques
  - Convertisseurs A/N : tension de 0 à V_1 → entier de 0 à N
  - Comparateurs de tensions
- Evénements
  - Détection de changement de ligne binaire : passe de 0 à 1 ou de 1 à 0

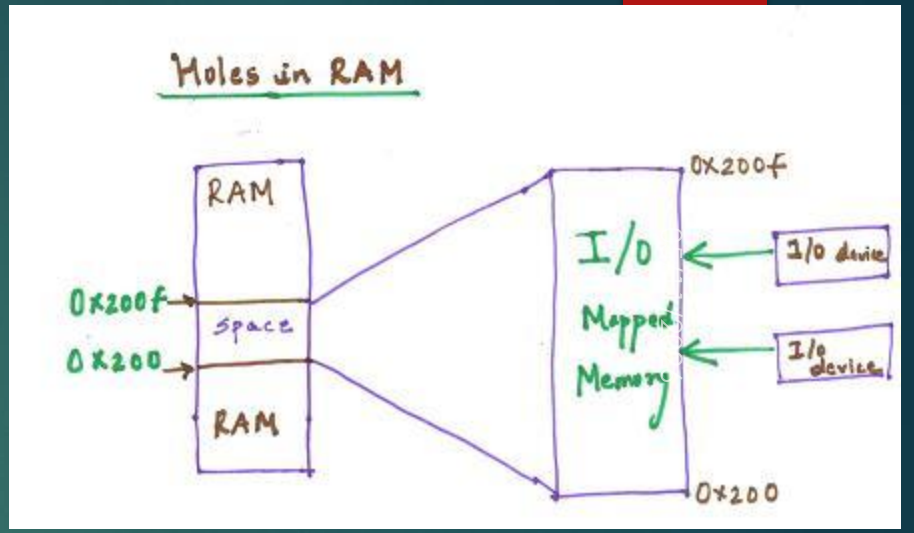
- Sorties (actionneurs)
  - Numériques
    - Lignes binaires en sortie (écriture 0 ou 1)
  - Analogiques
    - Convertisseurs N/A : entier de 0 à N → tension de V_1 à V_2
  - Temporelles
    - Lignes binaires en sortie avec choix de période et durée du 0 ou du 1

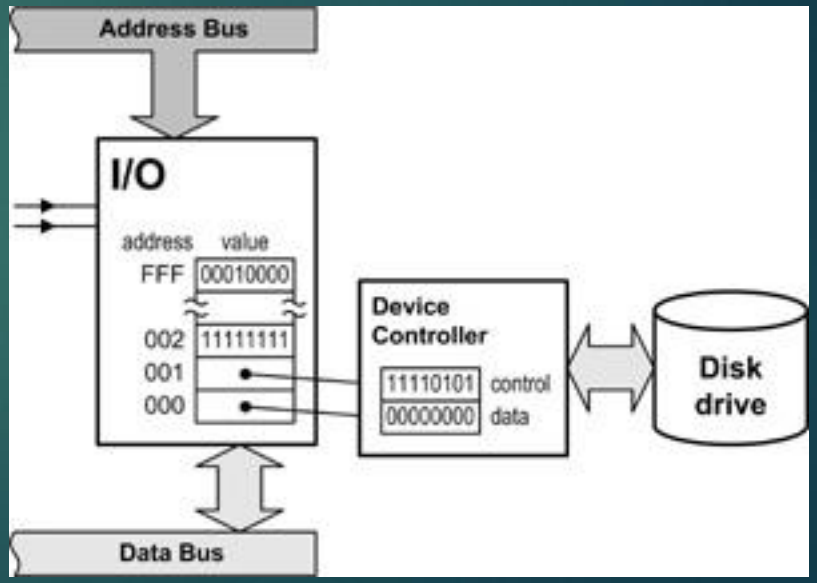
Types de Contrôleurs de périphériques

- Temps
  - Chronomètres (Timers)
    - Choix de la vitesse de comptage
    - Choix de la limite de comptage
- Communication
  - Avec des capteurs ou des actionneurs
    - Bus de données spécialisés (SPI, I2C, I2S)
  - Avec d'autres dispositifs numériques
    - Connexions réseau (RS232, Ethernet, CAN)
    - USB (USART)
    - Mémoire SD (SPI)
- Energie
  - Mise hors tension de périphériques
  - Détection de baisse d'alimentation

Accès aux périphériques Détails de bas niveau

- **Deux approches sont utiisées pour donner accès aux périphériques :**
- Approche 1 Allocation d'une partie de l'espace d'adressage aux E / S
  - Lire et écrire à ces adresses accède directement au périphérique.
  - Lire et écrire sur ces adresses donne accès à un contrôleur E / S
- Approche 2 Utilisation d'instructions spéciales
  - Pas de cache pour garantir l'achèvement de l'instruction
- Chaque fabriquant choisi une approche pour son processeur





Accès aux périphériques par le contrôleur de périphériques

- Chaque contrôleur de périphériques possède des registres pour gérer les périphériques;
  - **Registre de données** : pour envoyer/recevoir des données aux périphériques
  - **Registre de contrôle** : pour configurer les options du contrôleur de périphérique
  - **Registre d'état** : pour renseigner sur l'état du périphérique
- Le processeur accède aux différents registres par une opération de lecture / écriture quelconque comme pour un accès en RAM

Contrôleurs de périphériques sous Arduino

- Arduino Uno ATMEGA 328P
- Aduino MEGA ATMEA 2560

Comparaison entre Arduino Uno et MEGA 2560

Features	Arduino Uno	Arduino Mega (1280)	Arduino Mega 2560
Microcontroller (MCU)	ATmega328P (Datasheet)	ATmega1280	ATmega2560 (Datasheet)
Operating Voltage of the Microcontroller	5 V	5 V	5 V
Typical Supply Voltage for the board	7 V – 12V	7 V – 12 V	7 V – 12 V
Digital I/O Pins	14 (includes 6 PWM outputs)	54 (includes 15 PWM outputs)	54 (includes 15 PWM outputs)
PWM outputs	6	15	15
Analogue Input Pins	6	16	16
Max DC Current per I/O Pin	20 mA	20 mA	20 mA
Max DC Current for 3.3V Pin	50 mA	50 mA	50 mA
Flash memory of MCU	32 KB	128 KB	256 KB
Bootloader footprint	0.5 KB	4 KB	8 KB
SRAM of MCU	2 KB	8 KB	8 KB
EEPROM of MCU	1 KB	4 KB	4 KB
Clock Speed	16 MHz	16 MHz	16 MHz
Board Dimensions (Length)	68.6 mm	101.5 mm	101.5 mm
Board Dimensions (Width)	53.4 mm	53.4 mm	53.4 mm

6 Contrôleurs de périphériques du 2560

- Capteurs/Actionneurs
  - Numériques : 85 lignes d'E/S
  - Analogiques :
    - 1 Convertisseur A/N à 16 entrées + 2 entrées fixes (0V et 1,1V)
  - Temporelles : 16 lignes de signaux périodiques
- Temps
  - 2 Timers sur 8 bits (très limités)
  - 4 Timers sur 16 bits
- Energie
  - Possibilité d'allumer/éteindre chaque contrôleur de périphérique individuellement
- Communication
  - 4 USART (série asynchrone) ou SPI (Serial Peripheral Interface)
  - 1 I2C (Inter-Integrated Circuit)

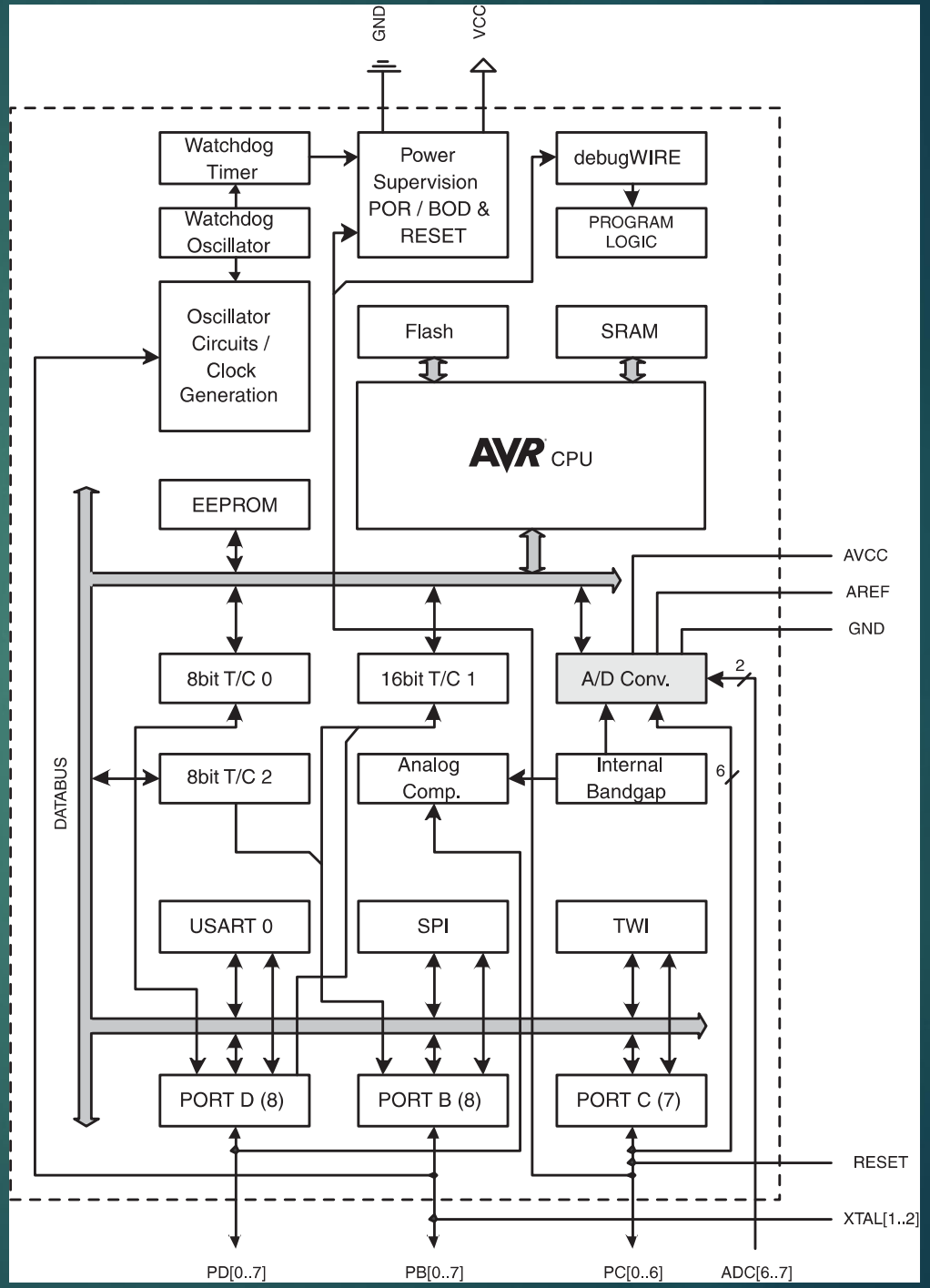
Contrôleurs de périphériques du 2560

- Architecture et utilisation :
  - Chaque contrôleur de périphériques contient des registres :
    - De commande pour définir le fonctionnement du contrôleur
    - D'état pour connaître l'état actuel du contrôleur
    - De données pour envoyer/recevoir des données depuis/vers le capteur ou l'actionneur piloté par le contrôleur
  - Les registres sont sur 8 ou 16 bits
  - Ils sont accessibles par le CPU comme des mots mémoire placés à des adresses fixes
  - Les registres sont découpés en champs de 1 ou plusieurs bits :
    - Chaque champ a un rôle ou un sens particulier
    - Le rôle ou le sens d'un champ peut dépendre du contenu d'un autre champ
  - La programmation d'un contrôleur de périphériques consiste à :
    - En définir le fonctionnement en remplissant les divers champs des registres de contrôle
    - En connaître l'état en consultant les divers champs des registres d'état
    - Echanger des données par les registres de données

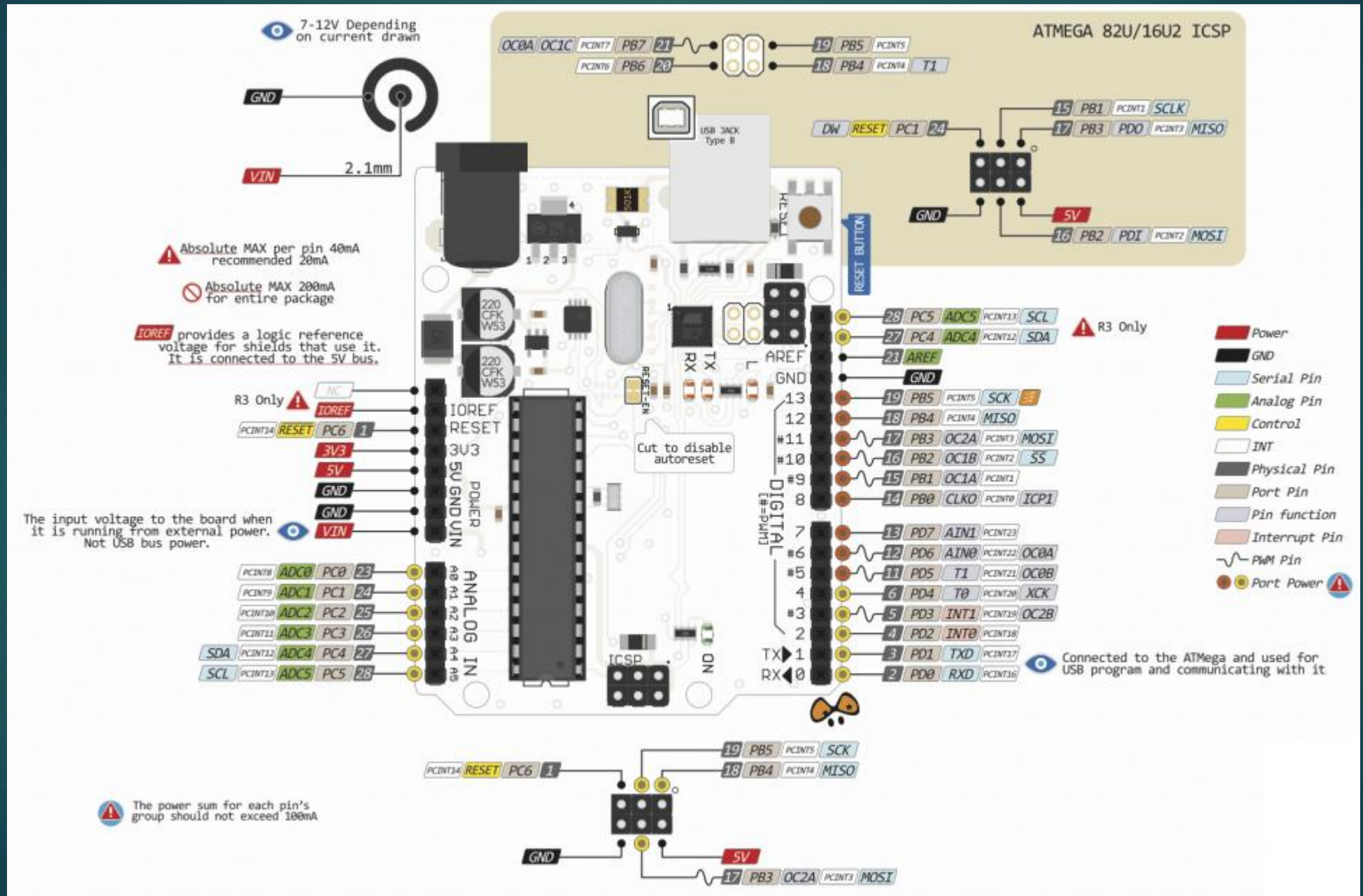
CPU du 2560

- Processeur RISC à 16 MHz
- Mémoire :
  - 256 Ko de flash
  - 4Ko EEPROM (sauvegardes)
  - 4Ko RAM extensible à 64Ko
- Interruptions :
  - Contrôleur d'interruptions (priorités fixes pas de préemption)
- Etats du processeur :
  - Actif
  - En sommeil (réveillé par interruption)
- Energie :
  - 1 Contrôleur d'alimentation (mise en/hors tension de chaque contrôleur de périphérique)
- Sécurité :
  - 1 Chien de garde (détection de pannes)
- Débogage :
  - Sonde JTAG
  - Moniteur de boot configurable par fusibles

Diagramme Bloc Atmega328P (Arduino Uno)

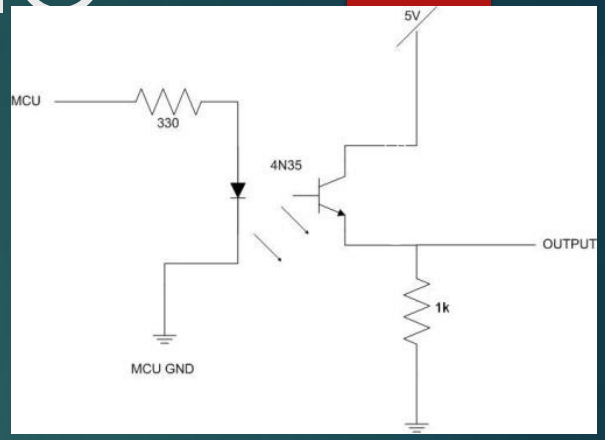


Brochage de l'ARDUINO UNO



Contrôleur de périphérique GPIO

- General-purpose input/output (GPIO)
- Fournit une ou plusieurs lignes d'E / S (parallèle)
- Mis en œuvre par un circuit appelé PIA (**Parrallel Interface Adapter**)
- Lignes d'entrée
  - Peut être connecté à l'entrée :
    - Directement
    - Par rappel à +V
    - Par déclencheur de Schmitt (basé sur le seuil)
    - Peut être mémorisé (latch)
  - Peut être indirect
    - Opto-isolé
  - Intégré dans l'Arduino (PORT B, PORT C, PORT D)



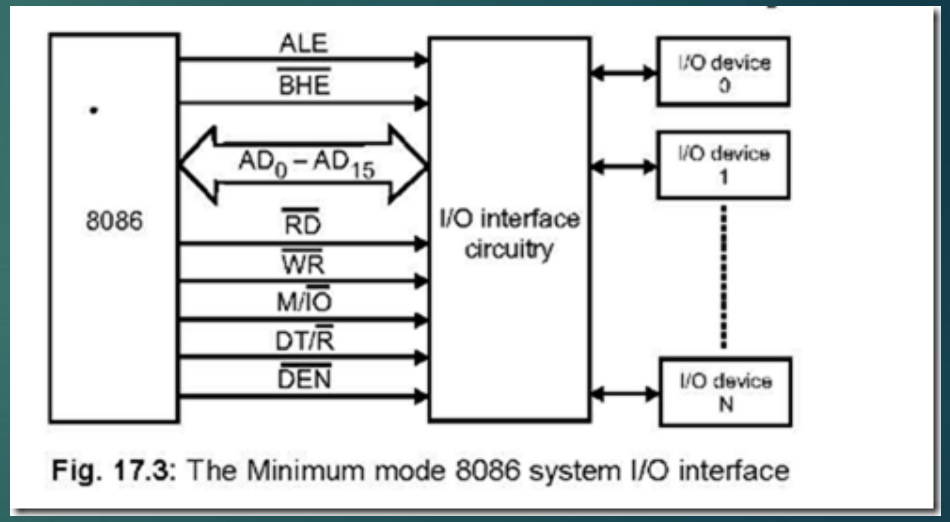
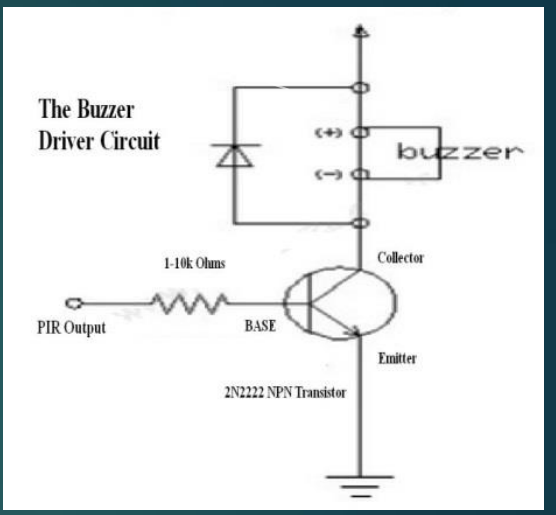
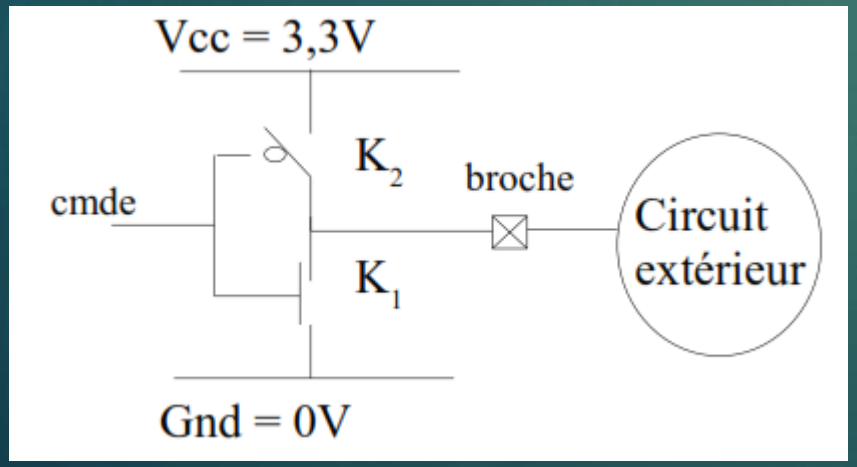


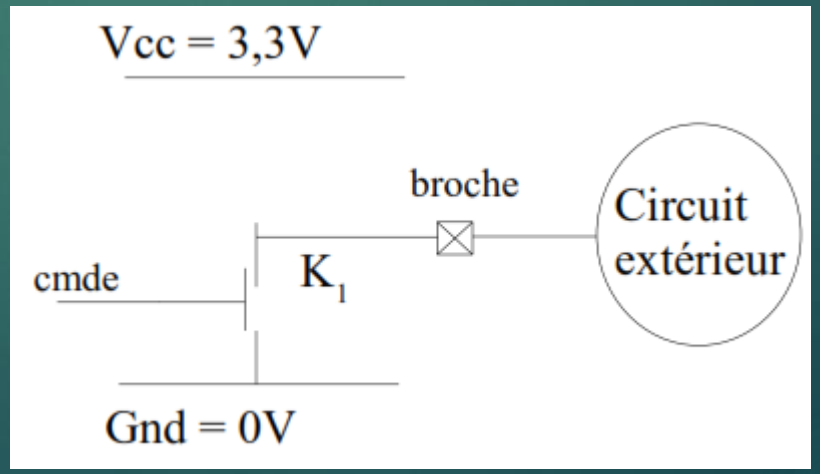
Fig. 17.3: The Minimum mode 8086 system I/O interface

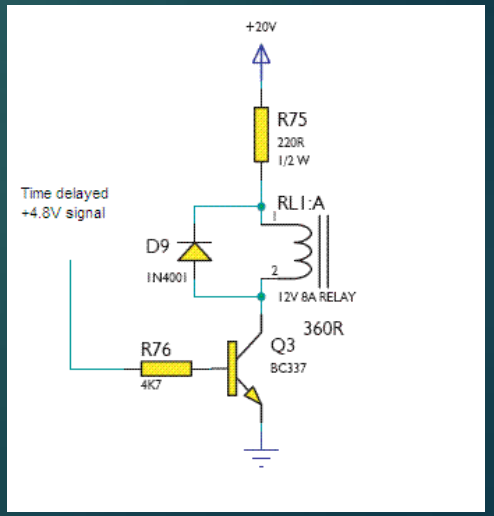
Contrôleur de périphérique

- Lignes de sortie
  - Broche connectée via un pilote pour la sortie
  - Fournit un courant de quelques mA (moyen)
  - Peut se connecter via un amplificateur pour plus de puissance
  - Peut être bufferisé (latch)
  - Sortie mode Push-pull et sortie Open-drain



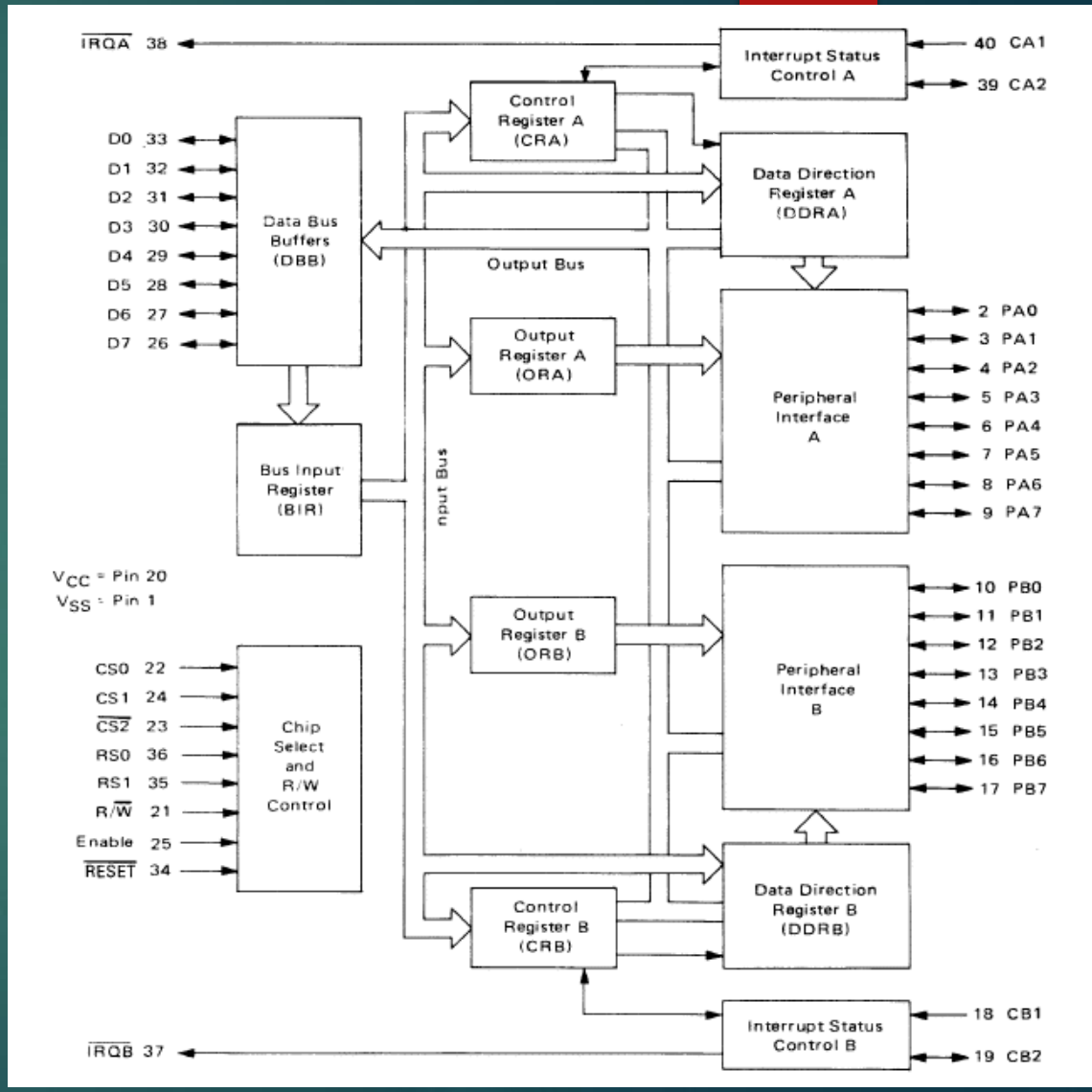






Matériel physique

- Peripheral Interface Adapter (PIA)
  - 16 lignes de données, 4 lignes de contrôle
  - Interface BUS vers le CPU
- Intégré dans l'Arduino
  - Fournit 14 lignes numériques
- Peut s'interfacer avec le bus mémoire / IO
- Lignes analogiques gérées par un matériel distinct (ADC)



Accès aux ports Arduino GPIO

- Fonctions standard
  - pinMode(NumPin,direction) pour le sens de la communication
  - digitalWrite(NumPin,valeur) pour écrire un bit
  - Valeur=digitalRead(NumPin) pour lire un bit
- Accès direct
  - Utiliser les variables DDRx et PORTx pour manipuler les ports
  - Rappels : broches A0-A5 →Port C
  - Broches 0-7 → Port D
  - Broches 8 à 13 →Port B

Accès direct aux GPIO

- Utile pour **lire/écrire** un **groupe de broches en même** temps
- Exemple configurer les broches 8 à 12 en sortie
- DDRB=0x1F; (0 entrée, 1 Sortie)
- Exemple écrire (10101) sur les broches 8 à 12
- PORTB=0x15;
- Exemple lire la valeur sur le groupe des broches numériques 4 à 7
- DDRD=0x0F; valeur=PORTD;
- valeur=valeur>>4; //pour ignorer les valeurs des broches 0 à 3
- PORTB=((PORTB & 0x80)>>7 | PORTB<<1); //rotation à gauche
- PORTB=((PORTB & 0x01)<<7 | PORTB>>1); //rotation à droite

Accès direct aux GPIO

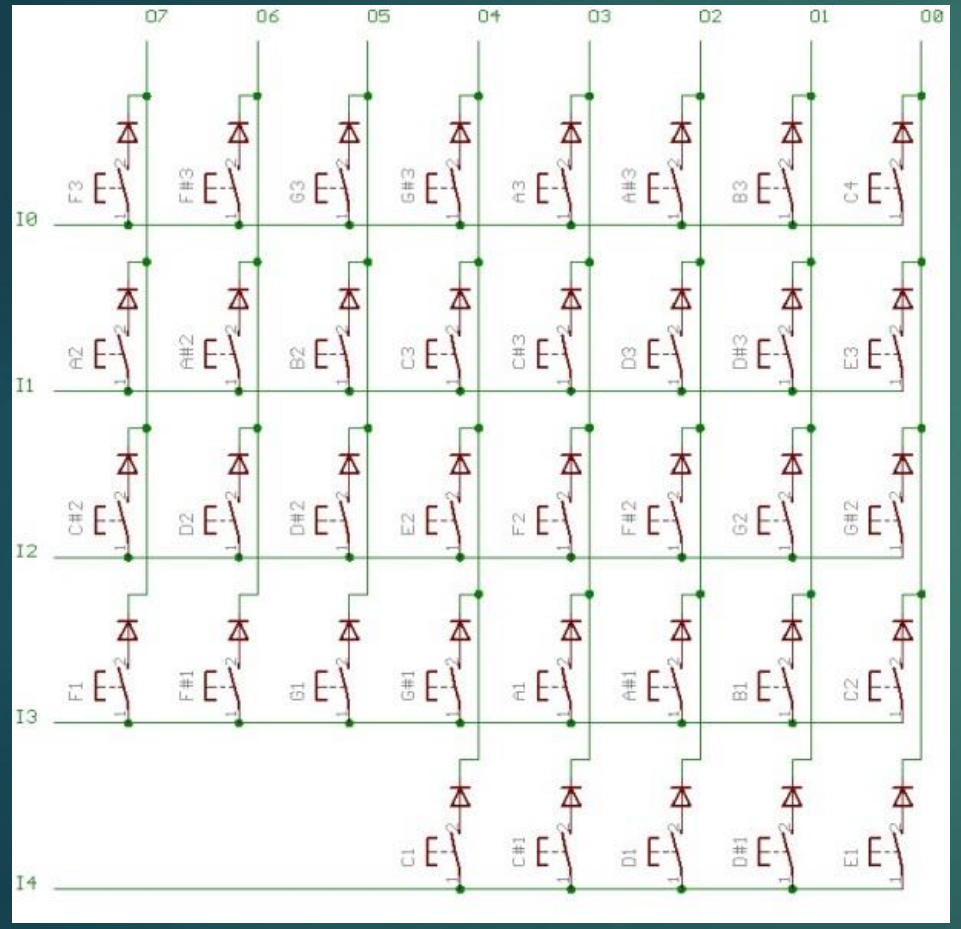
- **Manipulation des bits**
  - Un ensemble d'instructions **permet d'extraire des bits ou de changer la valeur des bits**
  - int x = 1; // binary: 0000000000000001
  - x <<= 3; // binary: 0000000000001000
  - x |= 3; // binary: 0000000000001011 – parce que 3 vaut 11 en binaire
  - x &= 1; // binary: 0000000000000001
  - x ^= 4; // binary: 0000000000000101 - inverse en utilisant le masque binaire 100
  - x ^= 4; // binary: 0000000000000001 - toggle with mask 100 again
  - int x = -16; // binary: 1111111111110000
  - int y = x >> 3; // binary: 1111111111111110 sign extension
  - int x = -16; // binary: 1111111111110000
  - int y = unsigned(x) >> 3; // binary: 0001111111111110

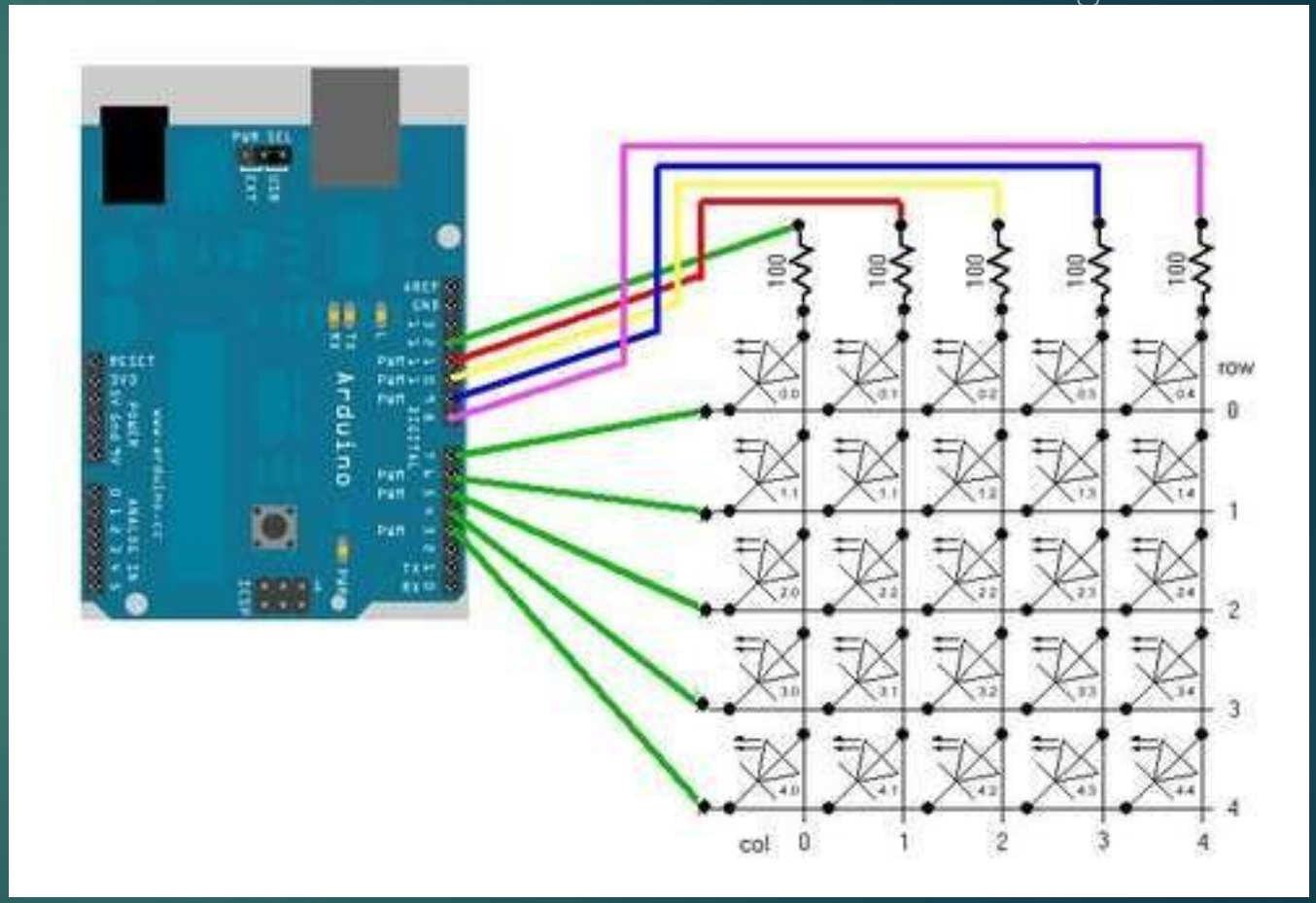
E/S d'un flipper (Pinball)

- Problème
  - 60 voyants lumineux à contrôler
  - 56 interrupteurs à détecter
  - Nombre limité de ports d'entrée / sortie
- Solution
  - Utiliser un tableau (matrice) de commandes



Matrice de commandes

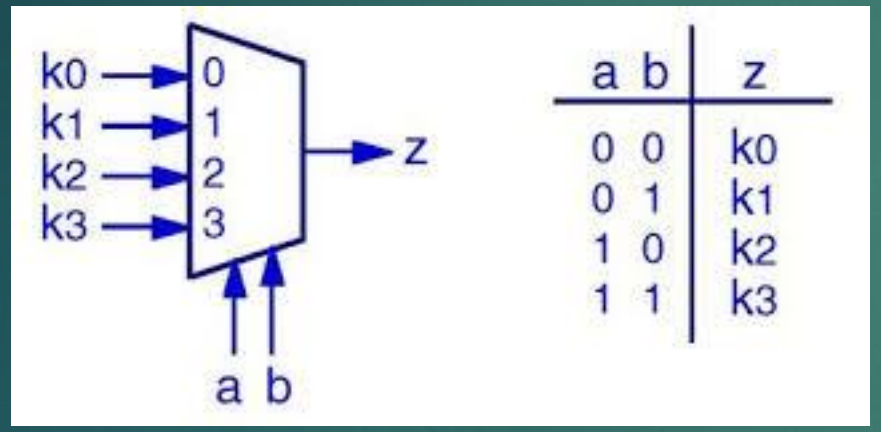




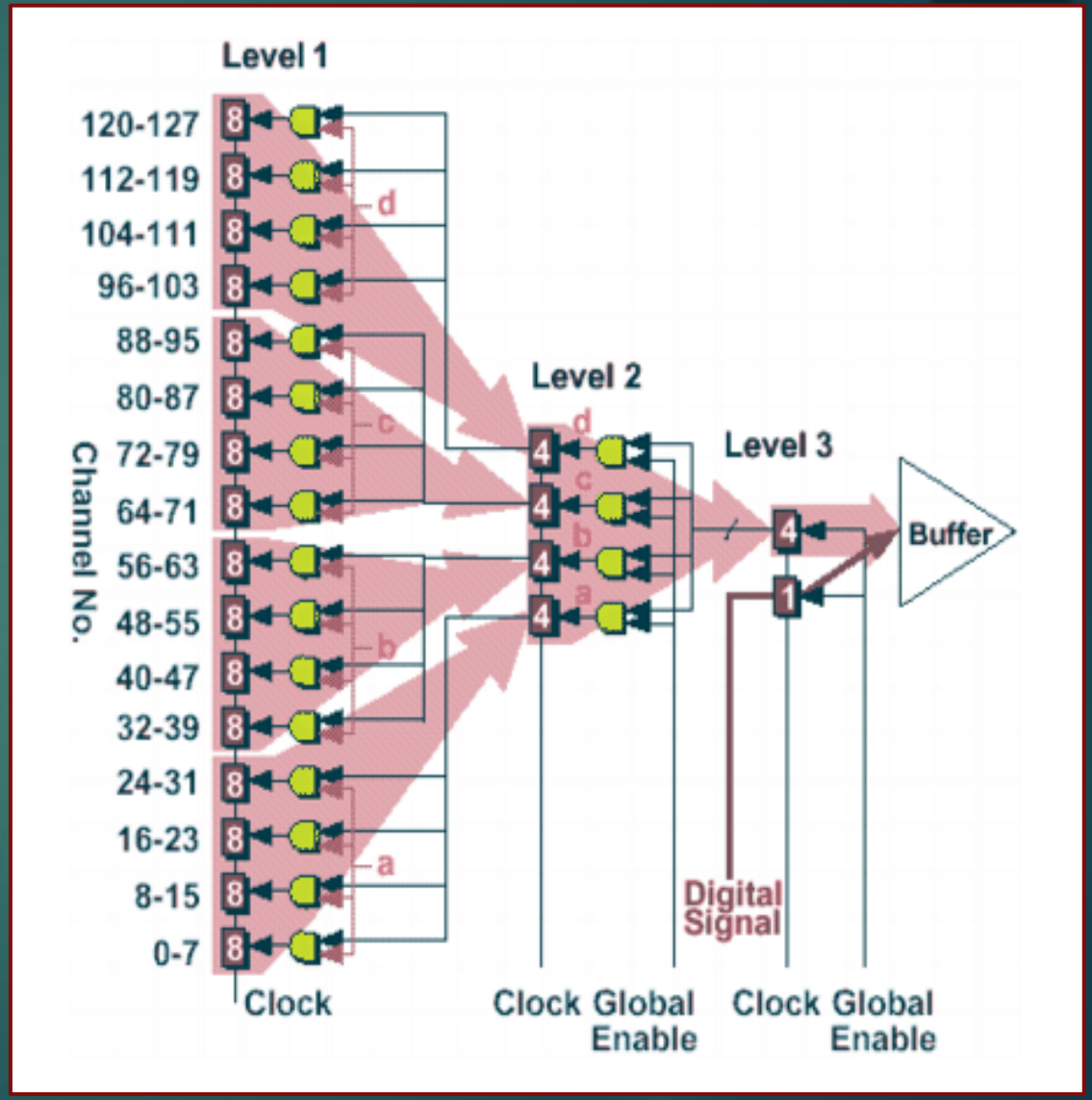
Solutions alternatives

- Utiliser un multiplexeur
- Utiliser un registre à décalage à verrouillage (latch)
- Utiliser une échelle de résistance

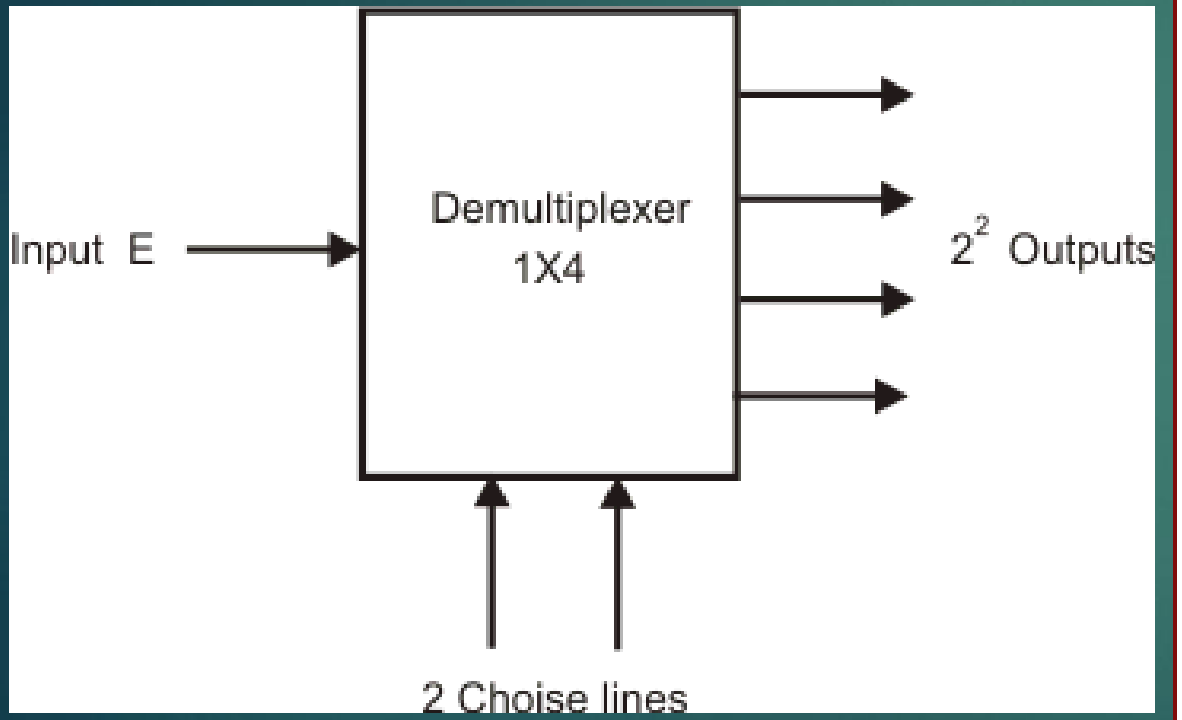
Multiplexeur

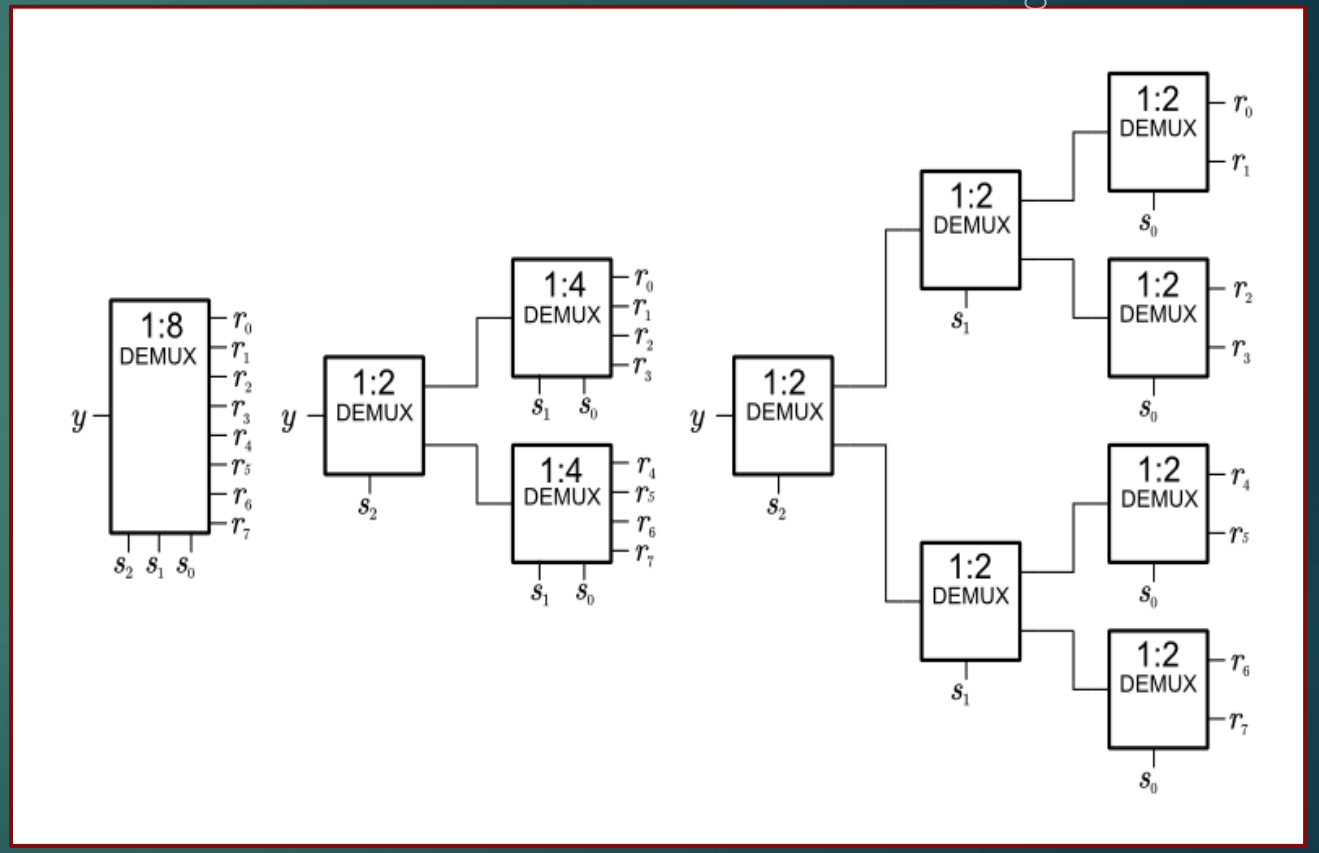


a	b	z
0	0	k0
0	1	k1
1	0	k2
1	1	k3



Démultiplexeur (Demux)



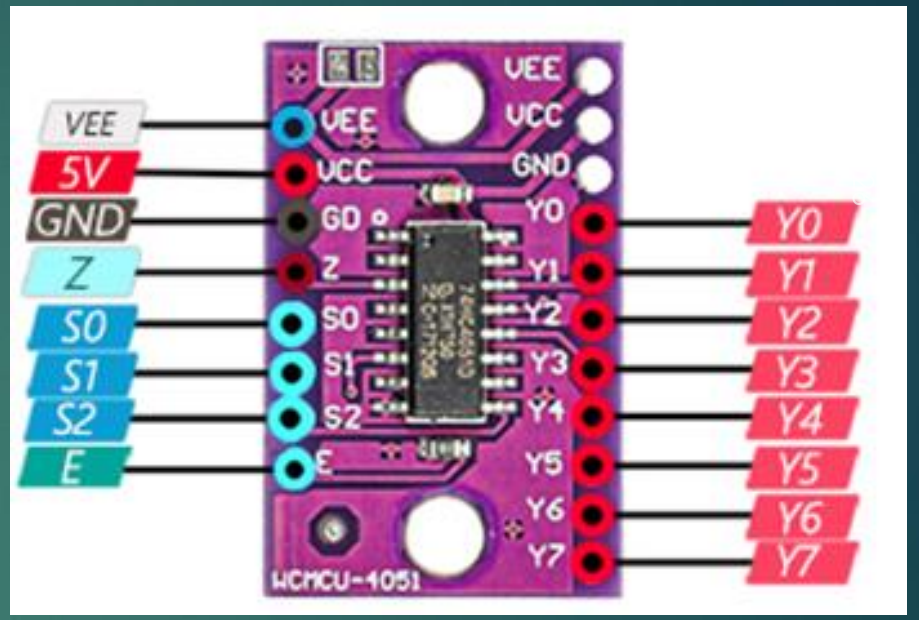


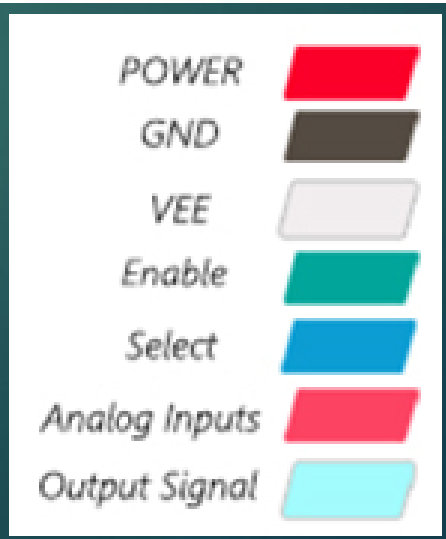
Interfaçage du multiplexeur à 8 canaux 74HC4051 avec Arduino

- Étape 1 : Circuit
  - Le circuit suivant montre comment vous devez connecter Arduino au module 74HC4051. Connectez les fils en conséquence.
- Etape 2 : Code
  - Téléchargez le code suivant sur Arduino.
  - Ensuite, ouvrez le moniteur série.
  - Dans le code ci-dessus, en donnant les valeurs appropriées aux broches S0 à s2 par seconde, nous sélectionnons les broches Y0-7 respectivement,
- et la tension de la broche Z, qui doit être égale à la broche sélectionnée, apparaît sur le moniteur série.

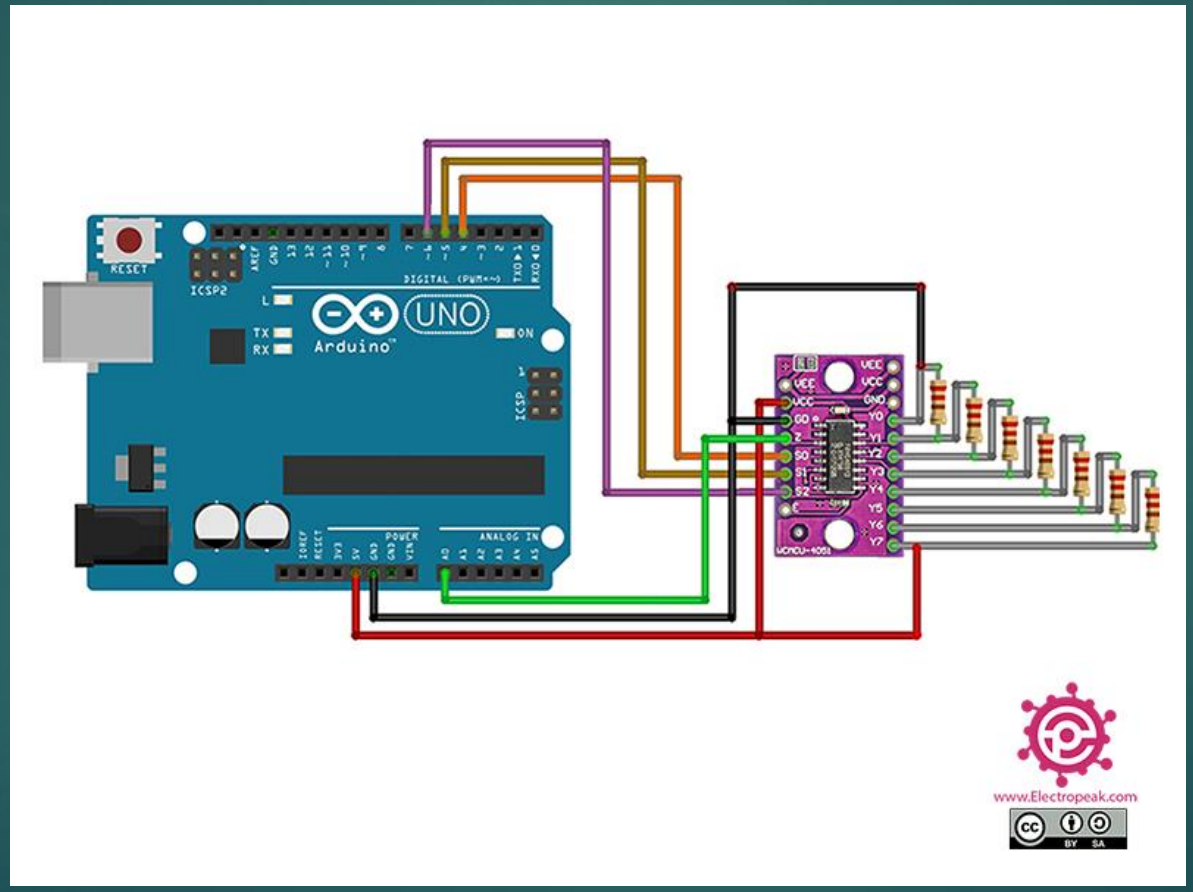
Brochage du 74HC4051

- **74HC4051 8-Channel Multiplexer Pinout**
- Ce module a 19 broches:
- **Côté gauche:**
- **VEE:** Negative power supply (Can be left unconnected)
- **VCC**: Module power supply – 5V
- **GND**: Ground
- **Z:** Output signal
- **S0-2:** Selecting one of the 8 analog input pins as the final output signal
- **E**: Enable pin (Active Low)
- **Côté droit :**
- **VEE:** Negative power supply (Can be left unconnected)
- **VCC**: Module power supply – 5V
- **GND**: Ground
- **Y0-7:** Analog inputs

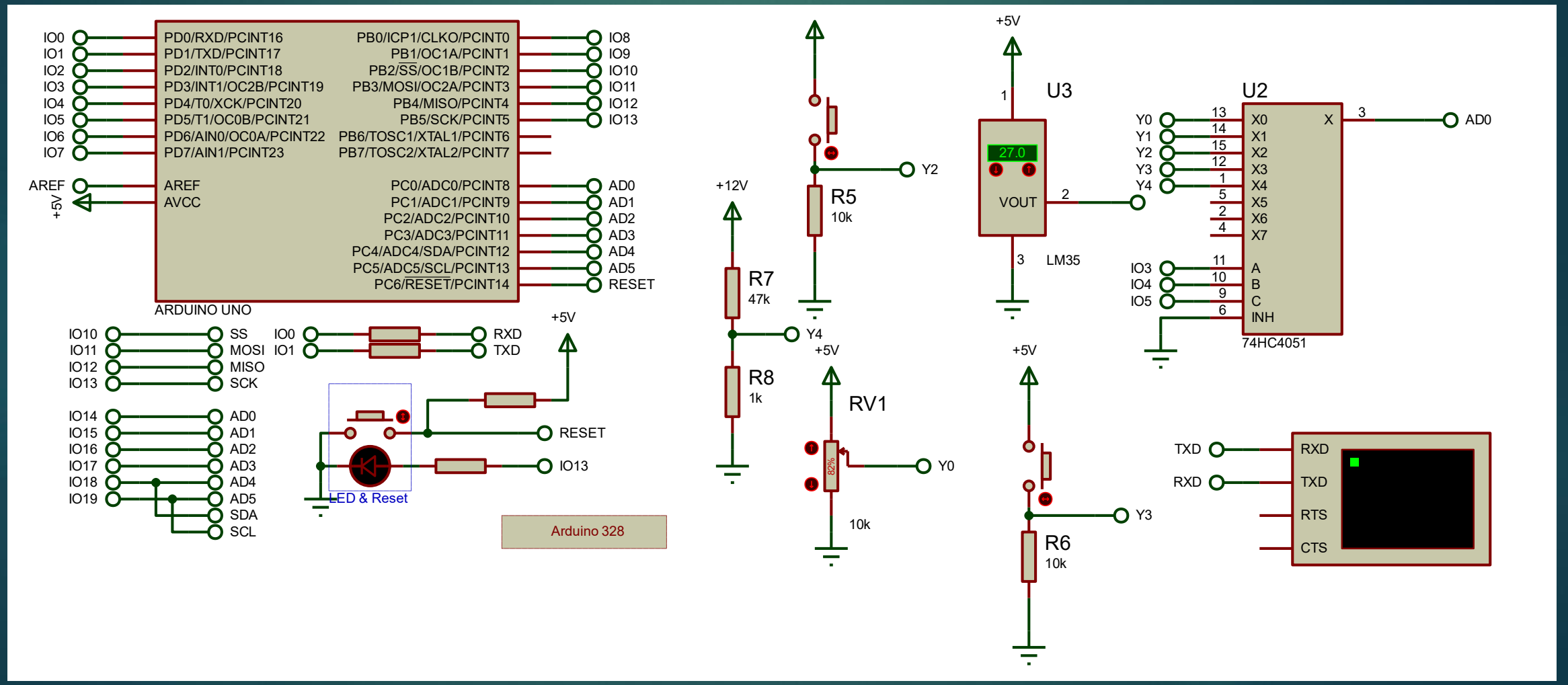




Interfaçage du multiplexeur à 8 canaux 74HC4051 avec Arduino



Interfaçage du multiplexeur à 8 canaux 74HC4051 avec Arduino



Interfaçage du multiplexeur à 8 canaux 74HC4051 avec Arduino (code)


```
/* Modified on Dec 1, 2020 Modified by
MehranMaleki from Arduino Examples Home */
//Mux control pins
int s0 = 4;
int s1 = 5;
int s2 = 6;
int Z_pin = 0; //Mux output "Z" pin on A0
void setup(){
pinMode(s0, OUTPUT);
pinMode(s1, OUTPUT);
pinMode(s2, OUTPUT);
digitalWrite(s0, LOW);
digitalWrite(s1, LOW);
digitalWrite(s2, LOW);
Serial.begin(9600); }
void loop(){
//Boucle pour lire les 8 entrées du MUX
for(int i = 0; i < 8; i ++){
Serial.print(« La valeur sur le canal ");
Serial.print(i); Serial.print(« est : ");
Serial.println(readMux(i));
delay(1000); } }
```

```
float readMux(int channel){
int controlPin[] = {s0, s1, s2};
int muxChannel[8][3]={
{0,0,0}, //channel 0
{1,0,0}, //channel 1
{0,1,0}, //channel 2
{1,1,0}, //channel 3
{0,0,1}, //channel 4
{1,0,1}, //channel 5
{0,1,1}, //channel 6
{1,1,1}, //channel 7
};
//positionner les trois bits de sélection
for(int i = 0; i < 3; i ++){
digitalWrite(controlPin[i],
muxChannel[channel][i]); }
 //lire la valeur de la broche Z du MUX
int val = analogRead(Z_pin);
//return the value
float voltage = (val * 5.0) / 1024.0;
return voltage; }
```

05/11/2024

Commande de 64 lampes

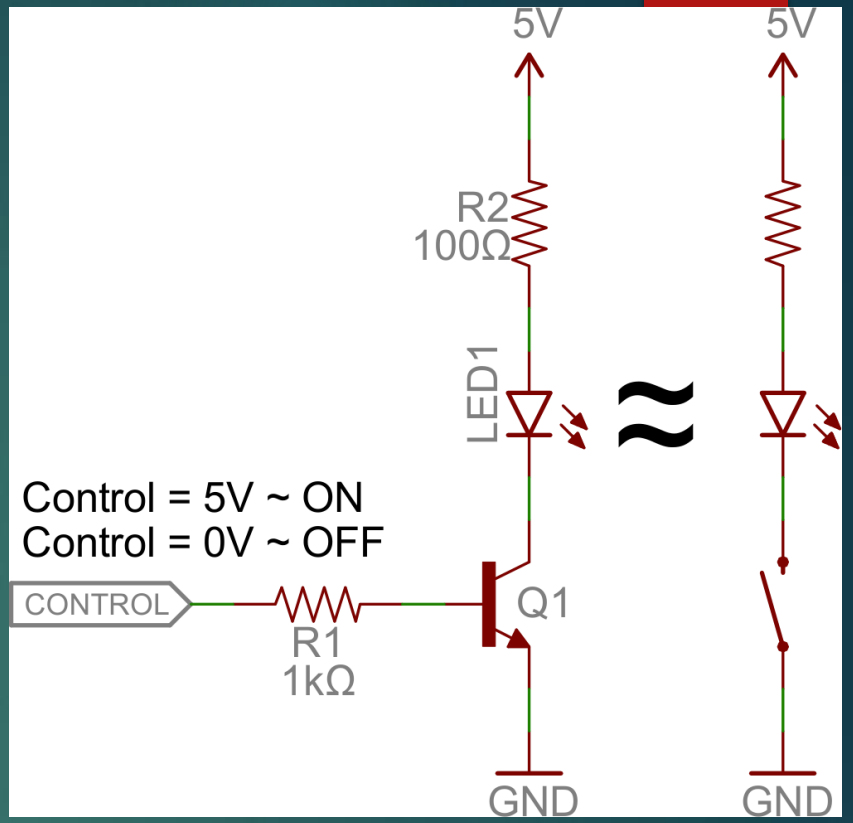
- Combien de lignes d'E / S sont nécessaires
- Problème
  - On ne peut commander qu'une partie à la fois
  - Les faire clignoter assez rapidement pour que ça marche
- Autre Problème
  - Il n'y a pas assez de courant pour alimenter toutes les lampes à la fois

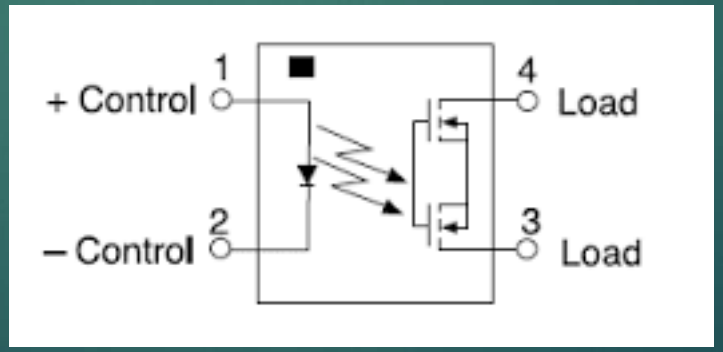
Commande des sorties

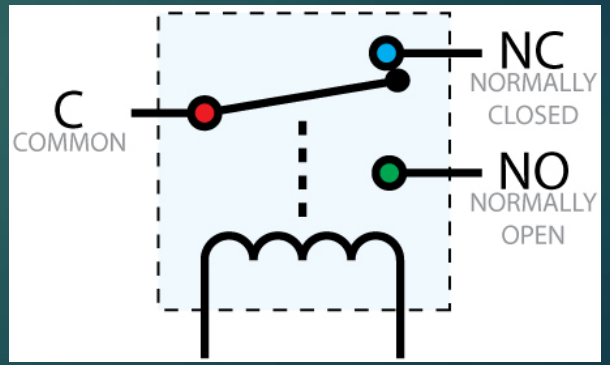
- Les broches de sortie Arduino Uno peuvent fournir au maximum **20mA**

Alternatives:

- Utiliser un circuit amplificateur (driver circuit)
- Uitilser un relais

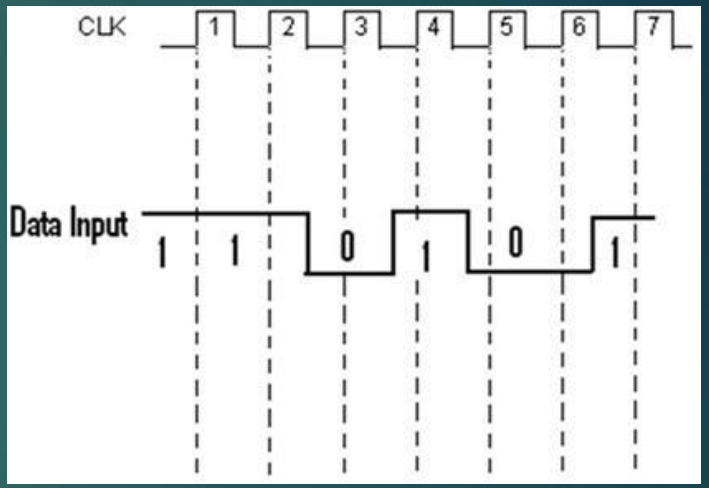






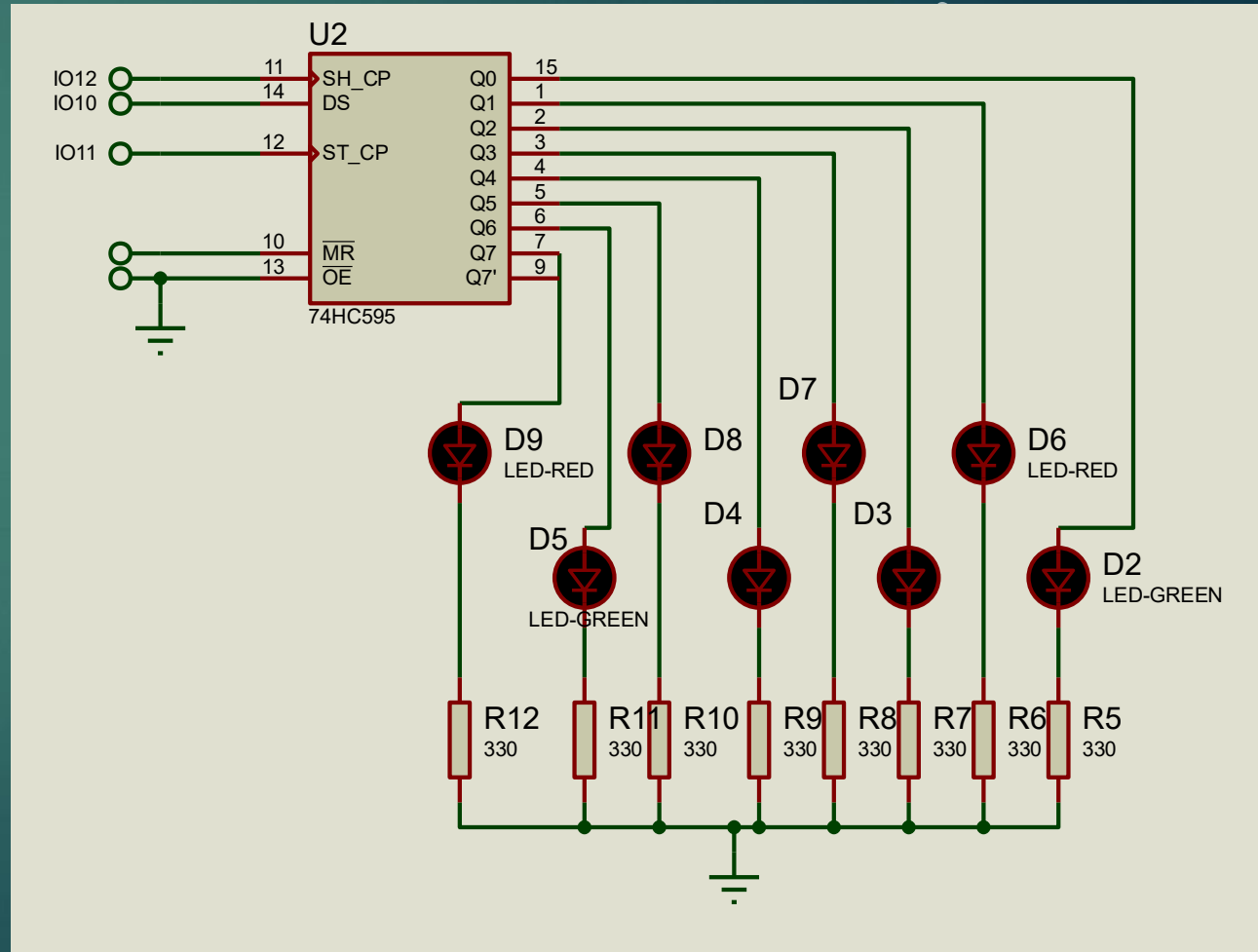
Interface série

- Le nombre des entrées/sorties parallèles est limitée
- Les E/S series : transmettent des données multi-bit en utilisant peu de broches
  - Principe : Envoyer des données sur une ligne à un débit fixe
  - Utilisé souvent pour la communication
  - Modes de cadencement
    - Asynchrone: doit s'accorder sur le timing
    - Synchrone: signal d'horloge partagée
  - Peut être utilisé avec des registres à décalage pour simuler des E / S parallèles



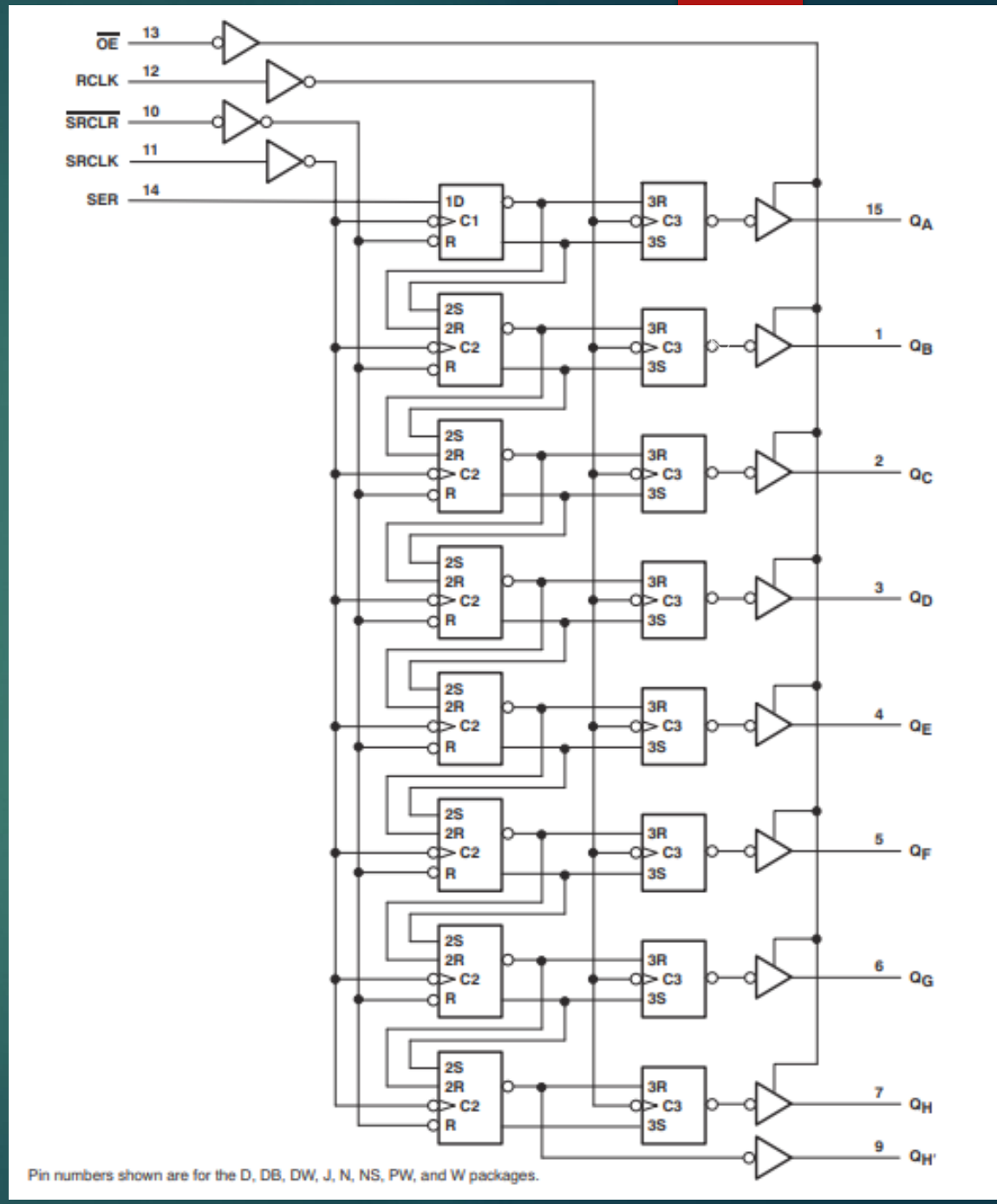
E/S série avec verrouillage

- 74HC595: registre à décallage 8 bits
  - Les données sont écrites sur la puce en série
  - Mémorisées dans le registre de stockage de la puce
  - Les valeurs mémorisées commandent ainsi les led



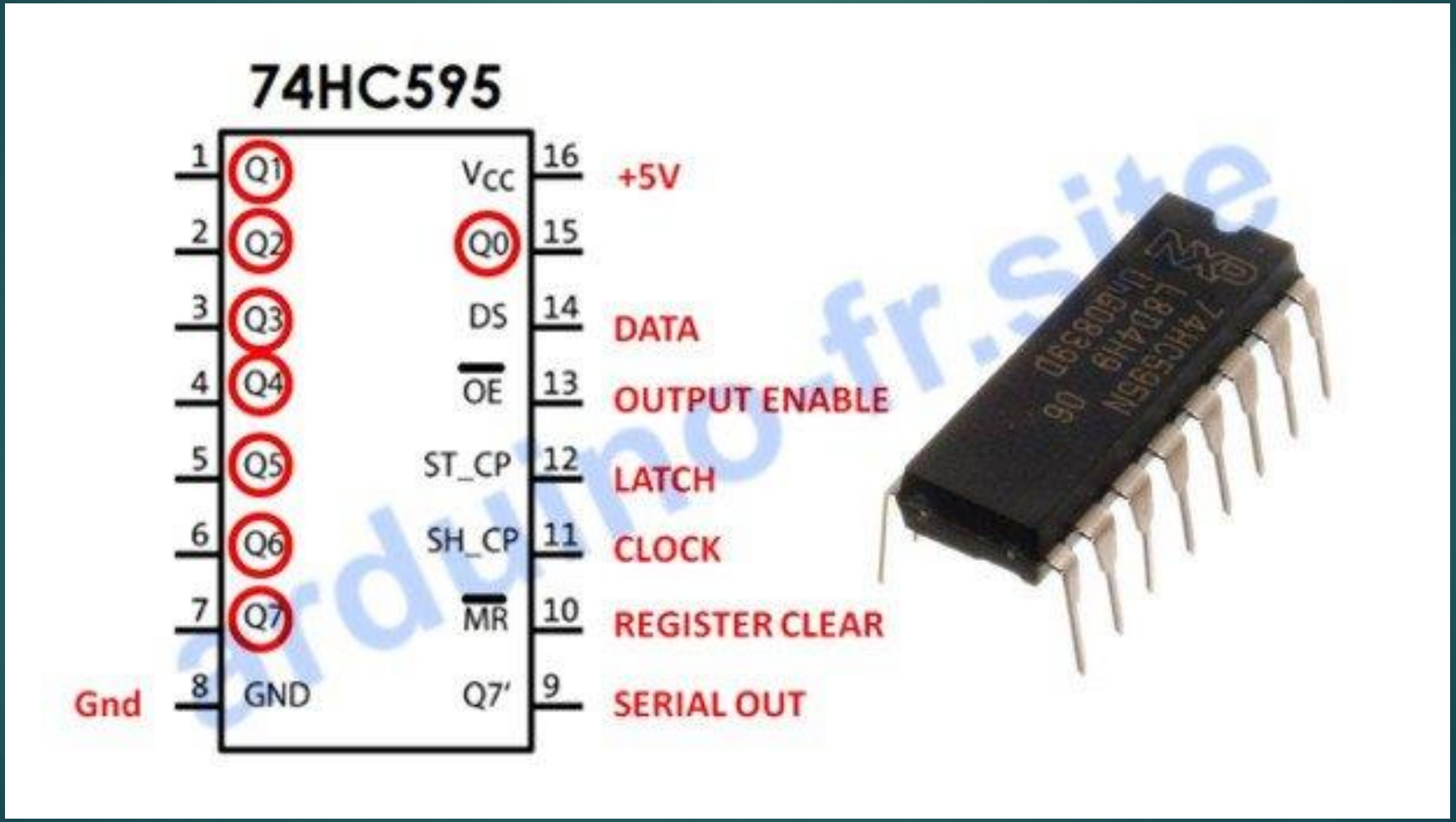
74HC595

- Diagramme fonctionnel
- Caractéristiques 74HC595
  - Entrée série de 8 bits sur 1 broche
  - Sortie série ou parrallèle des 8 bits sur 8 broches
  - Registre à décalage avec tampon de stockage en sortie, 3états
  - Câblage 2*8 broches
  - Décalages jusqu'à 100 MHz
  - Température de -40 à +85C°



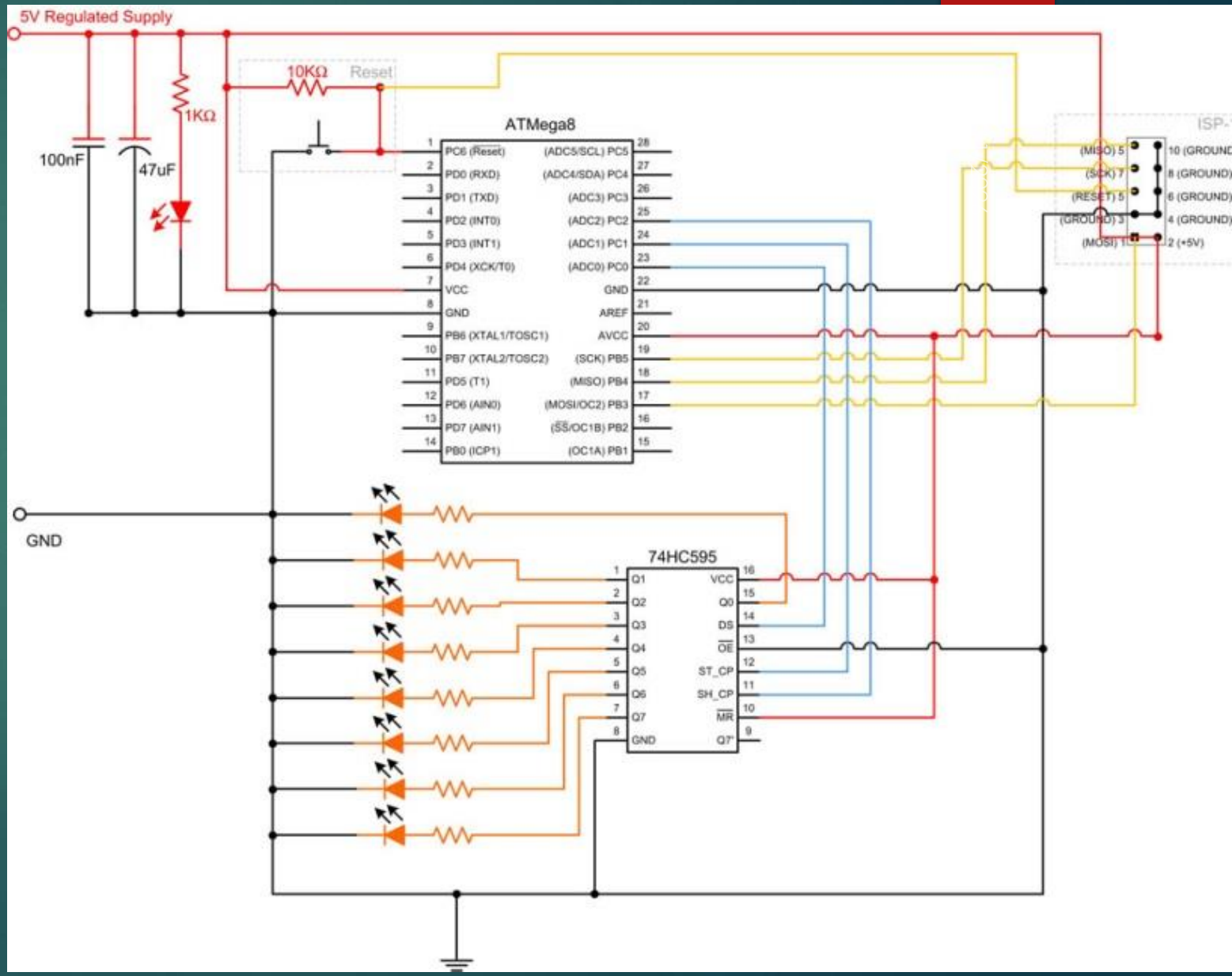
74HC595

- Brochage

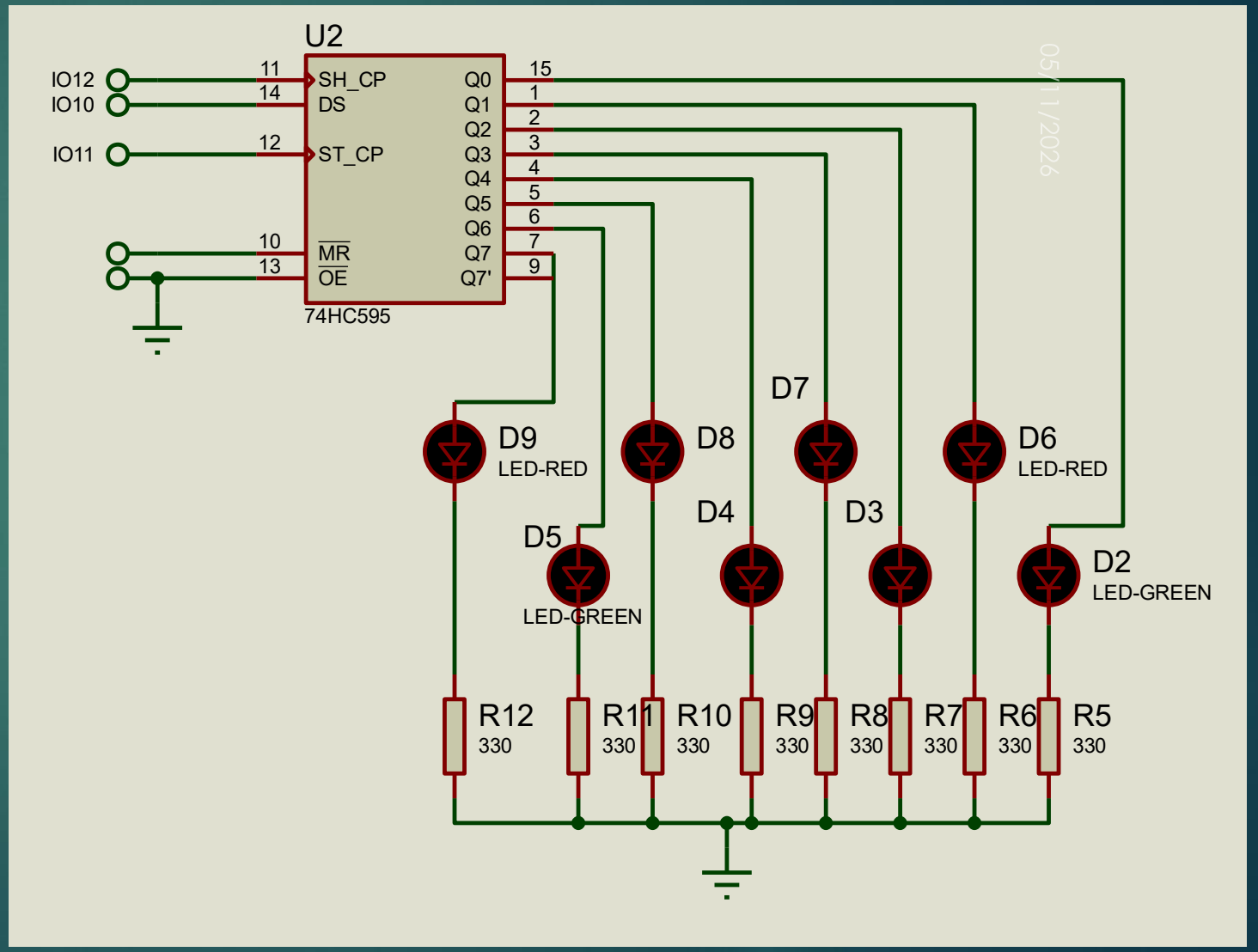


E/S série avec verrouillage

- 74HC595: registre à décallage 8 bits
- Le schéma ci-contre montre comment utiliser le circuit intégré 74HC595 pour allumer 8 leds.



Exemple d'utilisation du 74595



Code de démonstration


```
#define dataPin 10
#define latchPin 11
#define clockPin 12
byte byteToSend = 0;
 void setup() {
pinMode(latchPin, OUTPUT);
pinMode(clockPin, OUTPUT);
pinMode(dataPin, OUTPUT);
digitalWrite(latchPin, HIGH); }
void loop() {
for (byte bitPos = 0; bitPos < 8; bitPos++) { byteToSend = 0;
bitWrite(byteToSend, bitPos, HIGH);
digitalWrite(latchPin, LOW);
shiftOut(dataPin, clockPin, LSBFIRST, byteToSend);
 digitalWrite(latchPin, HIGH);
delay(100); } for (byte bitPos = 8; bitPos > 0; bitPos--) { byteToSend = 0;
bitWrite(byteToSend, bitPos, HIGH);
digitalWrite(latchPin, LOW);
shiftOut(dataPin, clockPin, MSBFIRST, byteToSend);
 digitalWrite(latchPin, HIGH);
delay(100); } }
```

Code de démonstration

- Explication du code pour composant 74HC595 et les LEDs:

1. dans la boucle for Arduino, la variable bitPos passe de 0 à 8.
2. La fonction **bitWrite(byteToSend, bitPos, HIGH);** modifie l'octet sous forme binaire et l'envoie au registre. Si le bitPos=0 on obtient B00000001, si le bitPos=1 on obtient B00000010, si le bitPos=2 on obtient B00000100, etc;
3. **MSBFIRST** et **LSBFIRST** change la direction du signal.

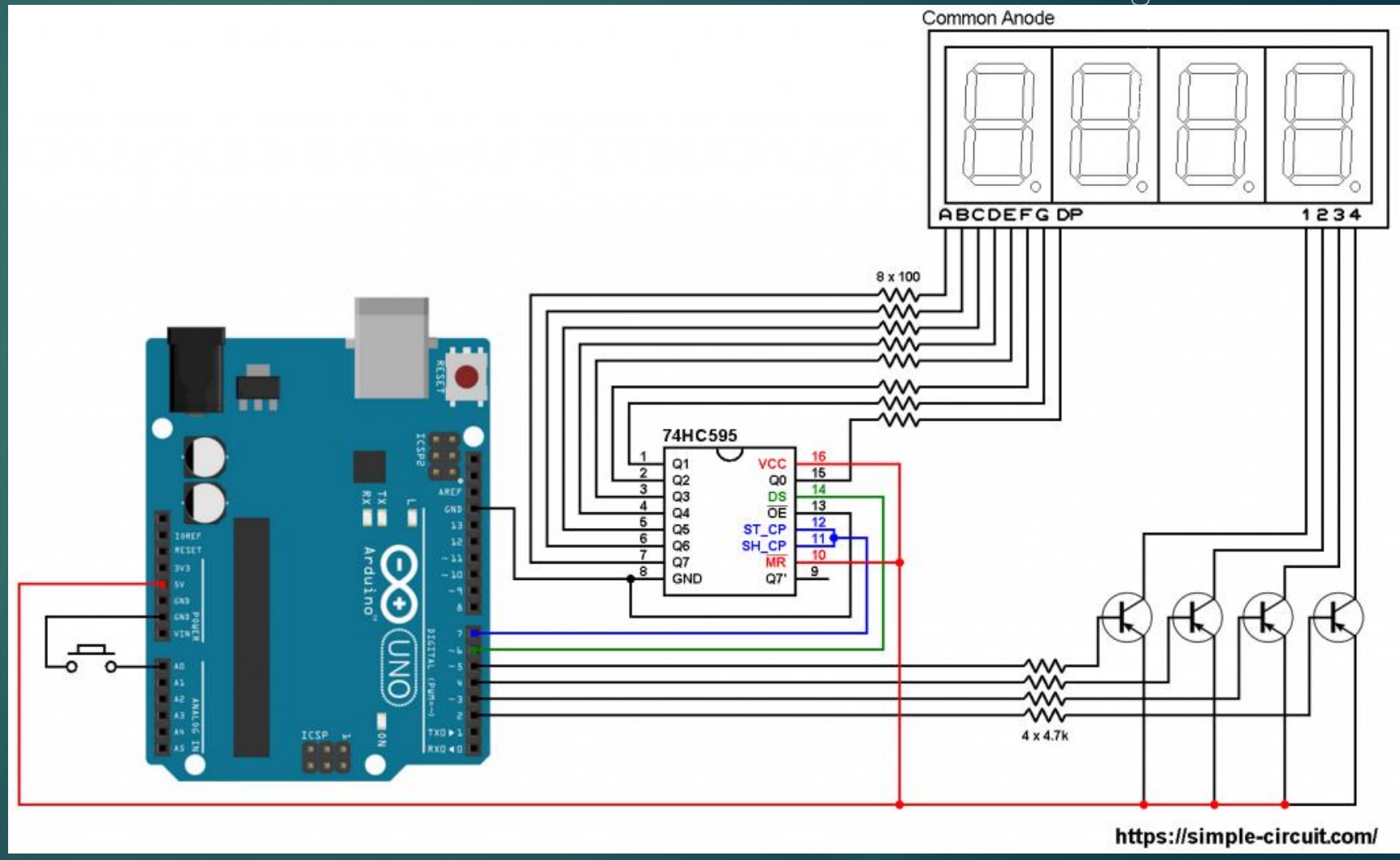
Afficheur 7 segments Arduino avec un circuit intégré de registre à décalage 74HC595

- l'ajout d'un registre à décalage entrée-serie-sortie-parallèle, tel que le populaire 74HC595, à un afficheur à 7 segments permet de réduire le nombre de broches nécessaires pour le piloter (l'afficheur).En fait, l'afficheur à 7 segments nécessite 9 broches : 8 broches de segment (A, B, C, D, E, F, G et DP) + une broche commune. En connectant toutes les broches de segment à un registre à décalage, le nombre de broches nécessaires n'est plus que de 3 : broche d'horloge et broche de données (pour le registre à décalage) + broche commune.
- Ainsi, pour un affichage à 4 chiffres et 7 segments, nous n'avons besoin que de 6 broches : horloge, données et 4 broches communes (chaque chiffre a sa propre broche commune).Comme le montre le schéma ci-dessous, toutes les broches des segments sont connectées aux broches de sortie du 74HC595, chacune par l'intermédiaire d'une résistance de 100 ohms :
- Les segments A ... G sont connectés aux broches Q7 ... Q1 du 74HC595 respectivement et le segment DP est connecté à la broche Q0.
- La broche de données du registre à décalage 74HC595 est appelée DS (#14) et est connectée à la broche 6 de l'Arduino. ST_CP (ou RCLK) et SH_CP (ou SRCLK) sont connectés ensemble et ensuite connectés à la broche 7 d'Arduino. C'est la broche d'horloge.
- Comme l'écran a 4 chiffres, il y a 4 broches communes : 1 (la plus à gauche), 2, 3 et 4. Chaque broche commune est connectée à la borne collectrice d'un transistor. Les bornes d'émission des 4 transistors sont connectées au +5V qui provient de la carte Arduino. Les bornes de base des quatre transistors sont connectées à l'Arduino par l'intermédiaire de résistances de 4,7k.Les 4 transistors sont du même type (PNP).

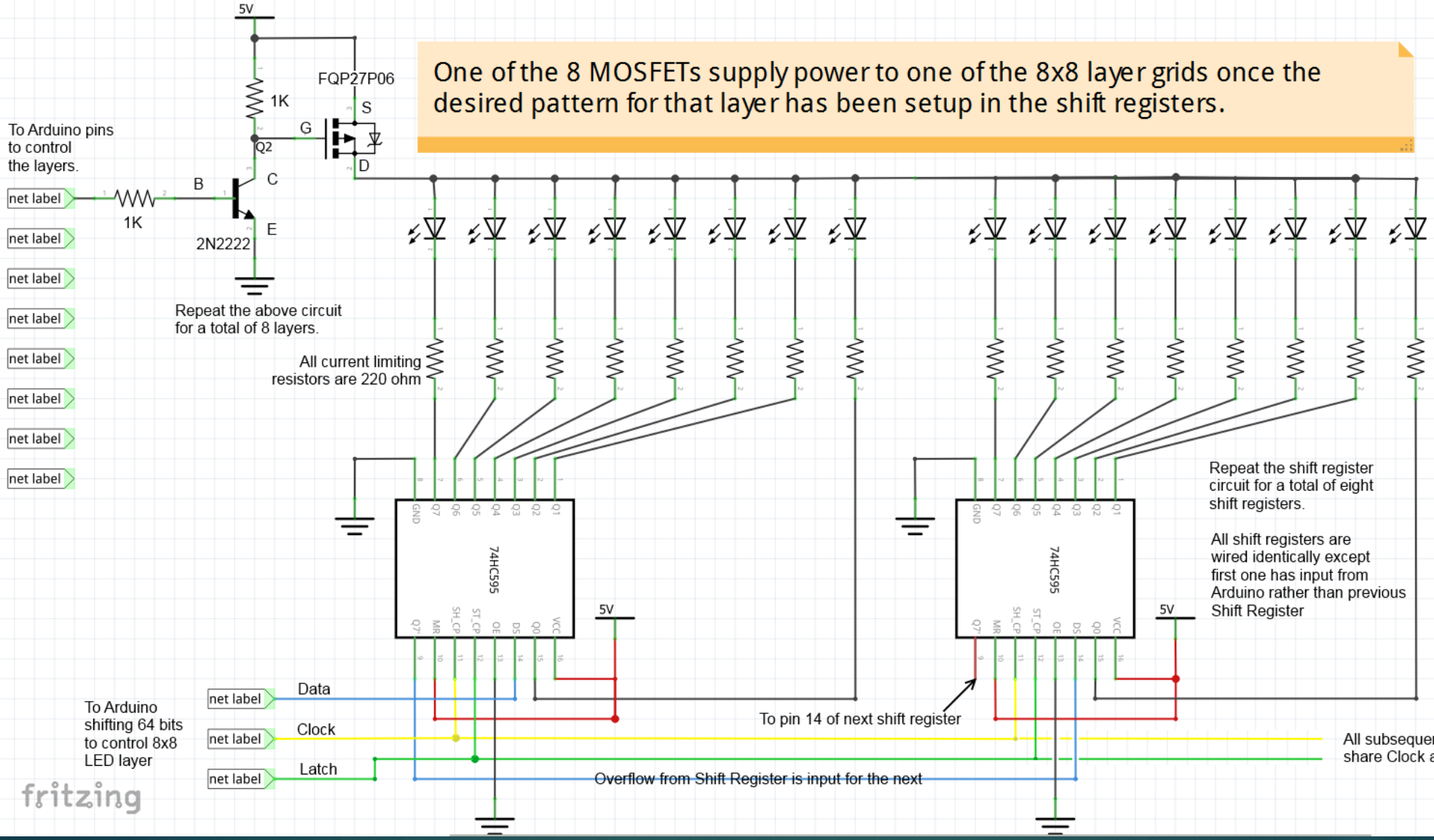
Afficheur 7 segments Arduino avec un circuit de registre à décalage 74HC595 avec Timer

Hardware Required:

- Arduino UNO board
- 4-Digit **common anode** 7-segment display
- 74HC595 shift register —-> datasheet
- 4 x PNP transistor (2SA1015, 2S9015, 2N3906 …)
- 8 x 100 ohm resistor
- 4 x 4.7k ohm resistor
- Push button
- Breadboard
- Jumper wires



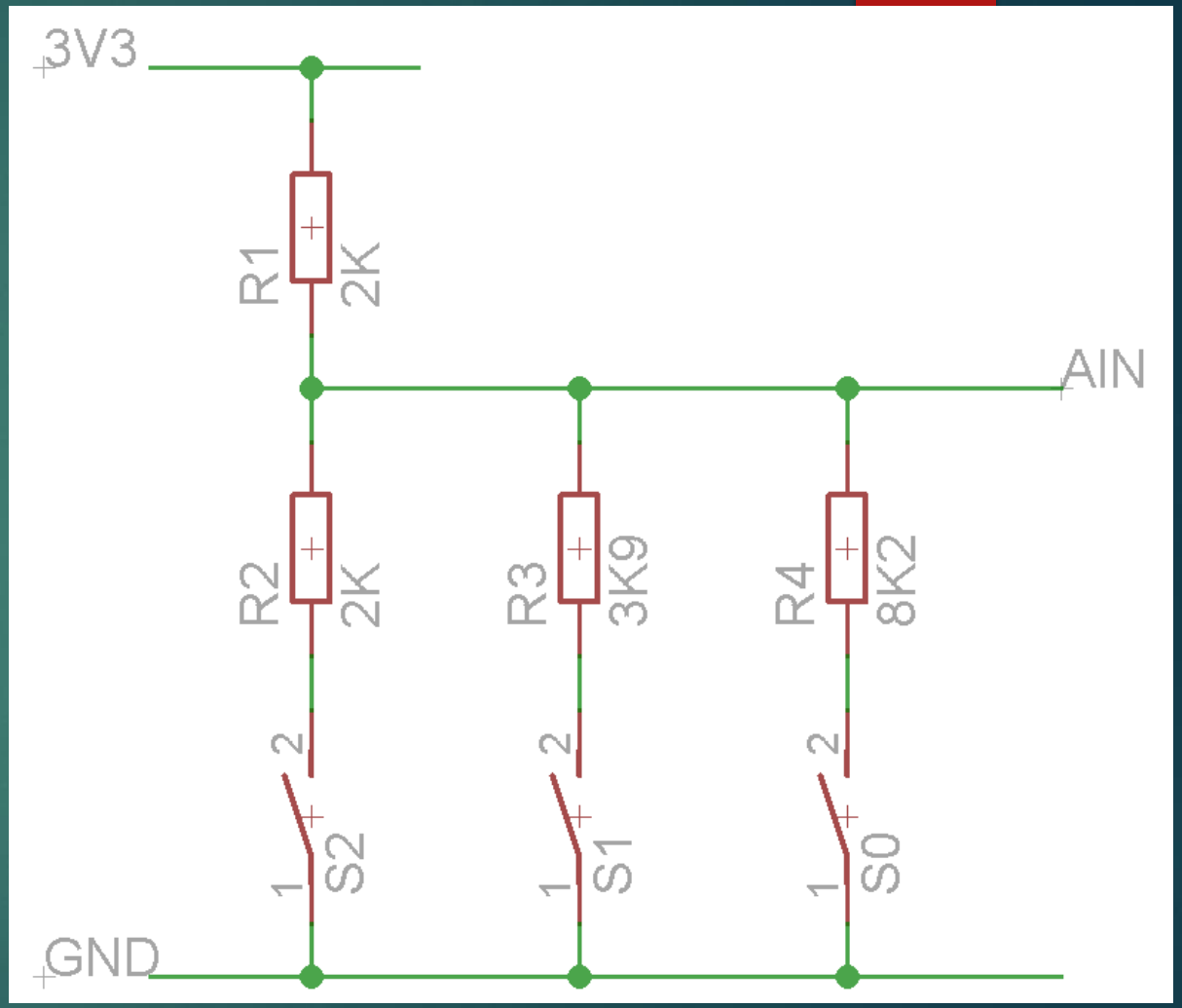
Commander 512 LED avec le 74HC595



Eight shift registers independently control the "ground" for each of the 64 LED pillars. Note how pin 10 from previous shift register is used as input (14) for the next shift register. Arduino shifts all 64 bits for the layer and then supplies (+) on the desired layers via one of the 8 MOSFETs.

Échelle de résistance

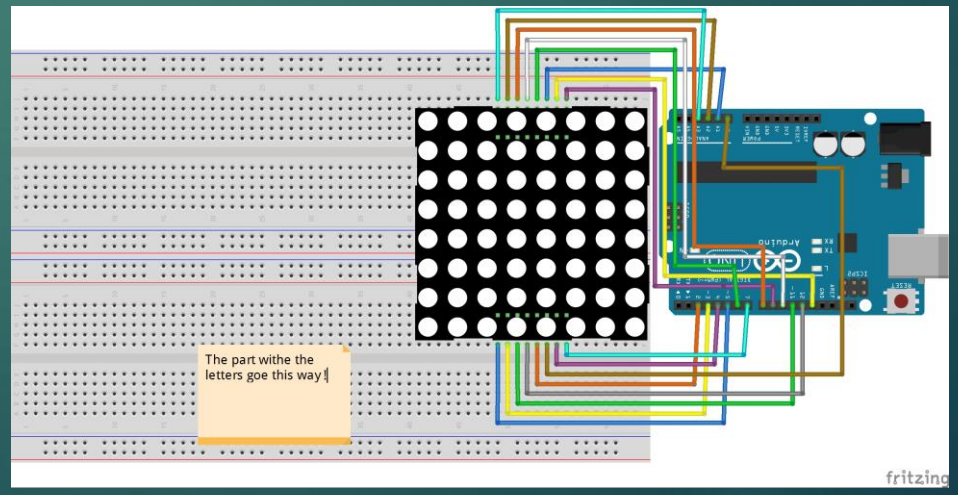
- Principe :
  - Identifier une combinaison d'entrées (S0,S1,S2,…) en fonction du niveau de la tension analogique lue
  - Créer différents diviseurs de tension en fonction du nombre d'interrupteurs fermés
  - Le choix des valeurs de R1,R2,R3,R4 est important pour fixer les valeurs de V_{AIN} non ambigüe
  - SI S0 seul est fermé, VAIN vaut 3,3*R4/(R1+R4)=2,65V
  - Si S1 seul est fermé VAIN vaut 3,3*R3/(R1+R3)=2,18V
  - Si S1 et S2 sont fermés VAIN vaut 3,3*(R3//R4)/(R1+R3//R4)=1,88 V



Matrice de LED

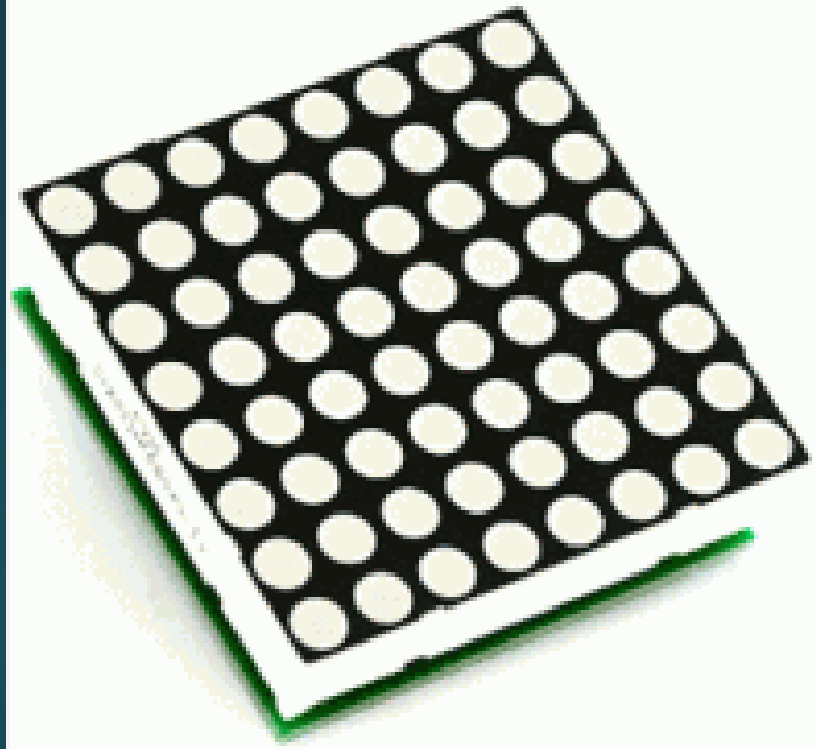
- La matrice des LED est constituée de 8 lignes et 8 colonnes. On peut assimiler la matrice à un tableau 1D de 8 éléments, chaque élément du tableau est codé sur 8 bits.

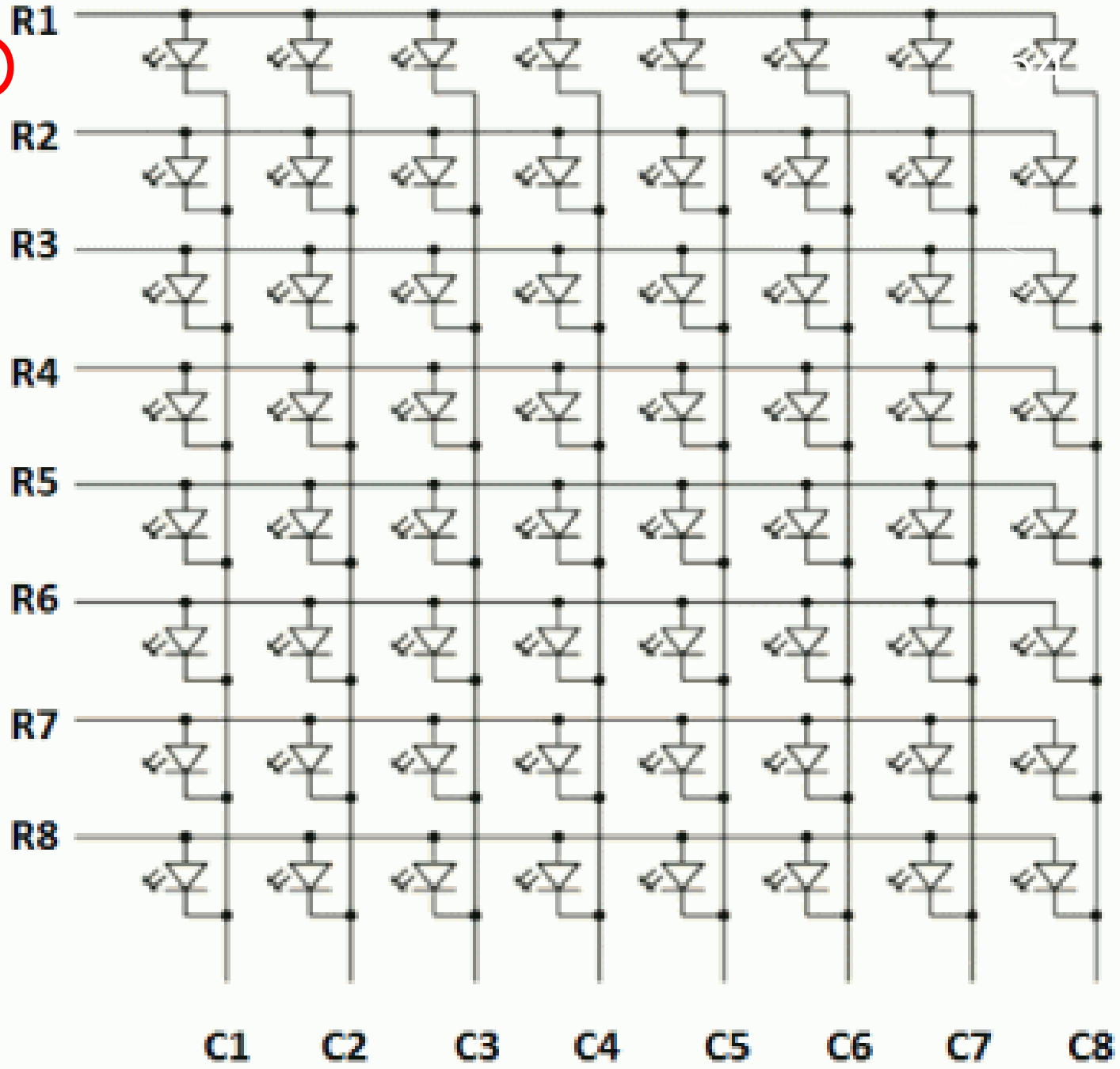
- Pour allumer la LED R1,C1, nous pouvons écrire
- digitalWrite(R1,HIGH);
- digitalWrite(C1,LOW);



Matrice de LED

- Schéma





Circuit de connexion d'une matrice LED avec Arduino Uno

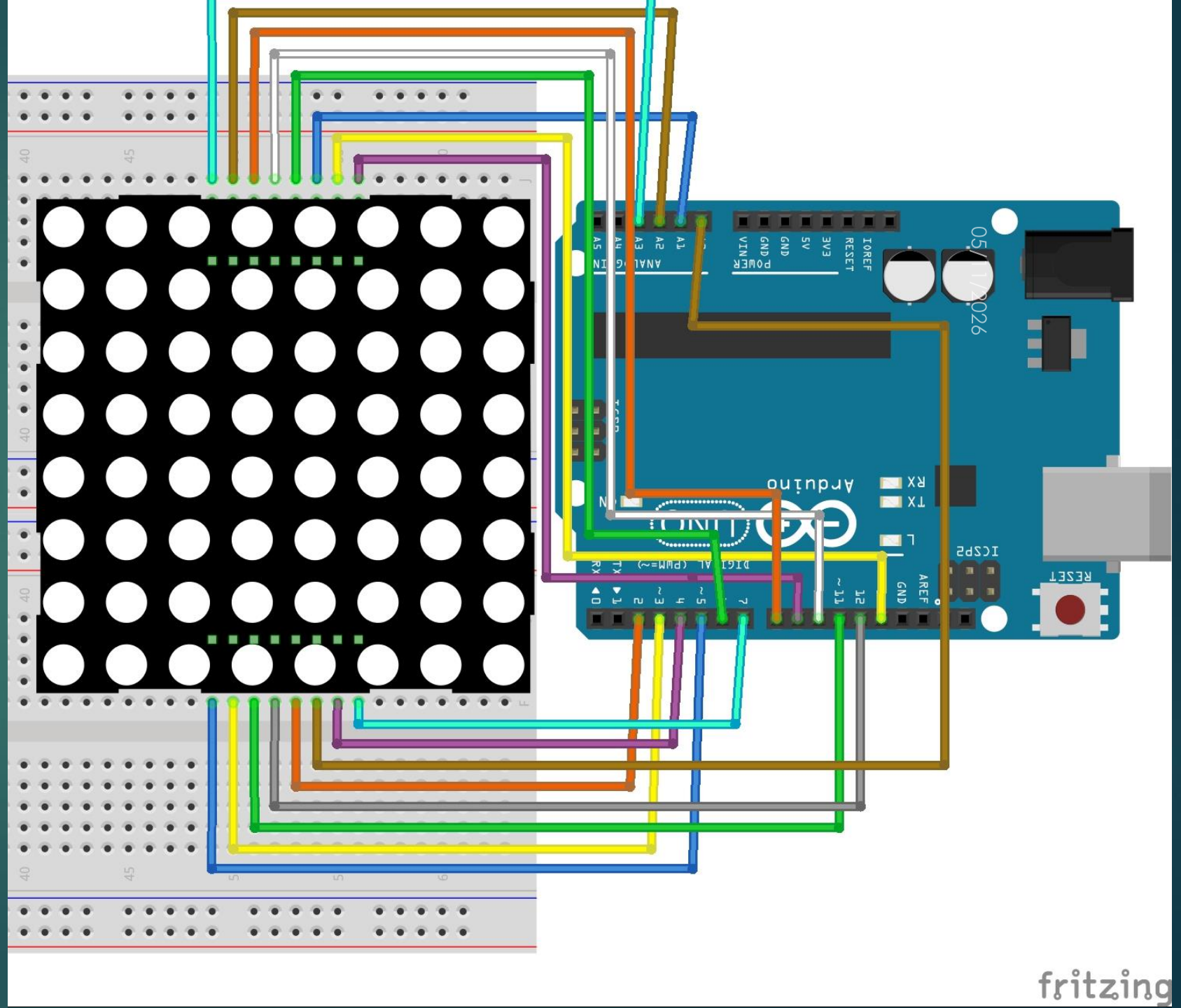
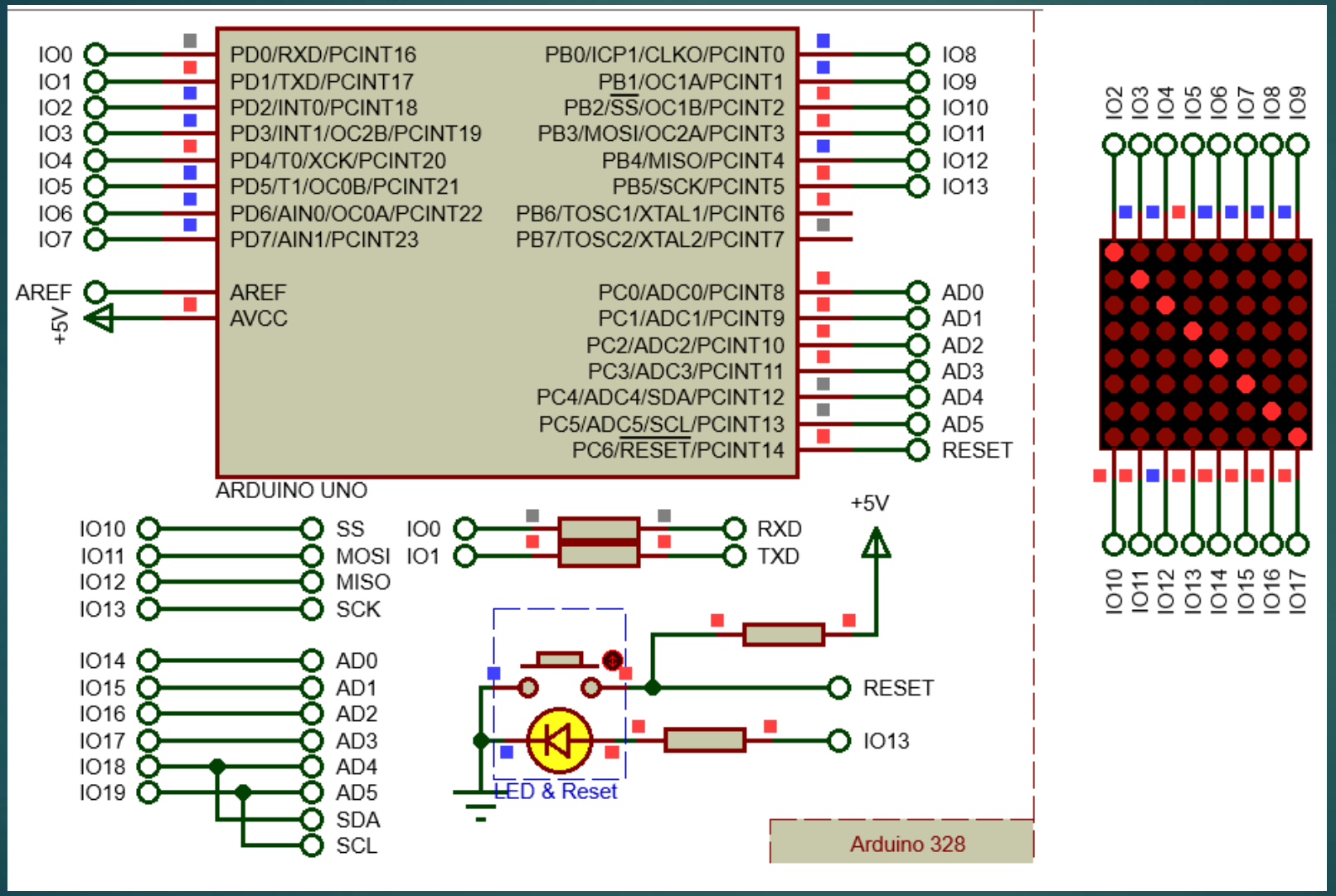


Schéma dans Proteus



Simulation de l'exécution du programme

Code Arduino pour allumer la diagonale


```
const int r[8]= {2,3,4,5,6,7,8,9};
const int
c[8]={10,11,12,13,14,15,16,17};
void setup() {
  // put your setup code here, to run
once:
for (int i=0;i<8;i++)
  {pinMode(r[i],OUTPUT);
  pinMode(c[i],OUTPUT);
  digitalWrite( c[i],HIGH);}
  Serial.begin(115200); //debug

  delay(5000);
}
void allumePixel(byte x,byte y,int
duree)
{
  if (x>0&&x<=8)
    if(y>0 &&y<=8)
        {digitalWrite(r[x-1],HIGH);
        digitalWrite(c[y-1],LOW);
delay(duree);
  digitalWrite(r[x-1],LOW);
        digitalWrite(c[y-1],HIGH);}
       else Serial.println("colonne
erronée :"+String(y));
 else Serial.println("ligne erronée
:"+String(x));
}
```

Code Arduino pour allumer la diagonale

- Fonctions pour allumer un pixel au hasard et fonction pour allumer la diagonale
- Note : la durée d'allumage du pixel est déterminante pour l'effet visuel recherché

```
void diagonale(){
  for(int i=1;i<=8;i++)
  {allumePixel(i,i,5);
  //eteintPixel(i,i);}
}
void aleat(){
  int x=random(8)+1;
  int y=random(8)+1;
  allumePixel(x,y,200);
}
void loop() {

aleat();

}
```

Code Arduino pour allumer un chiffre

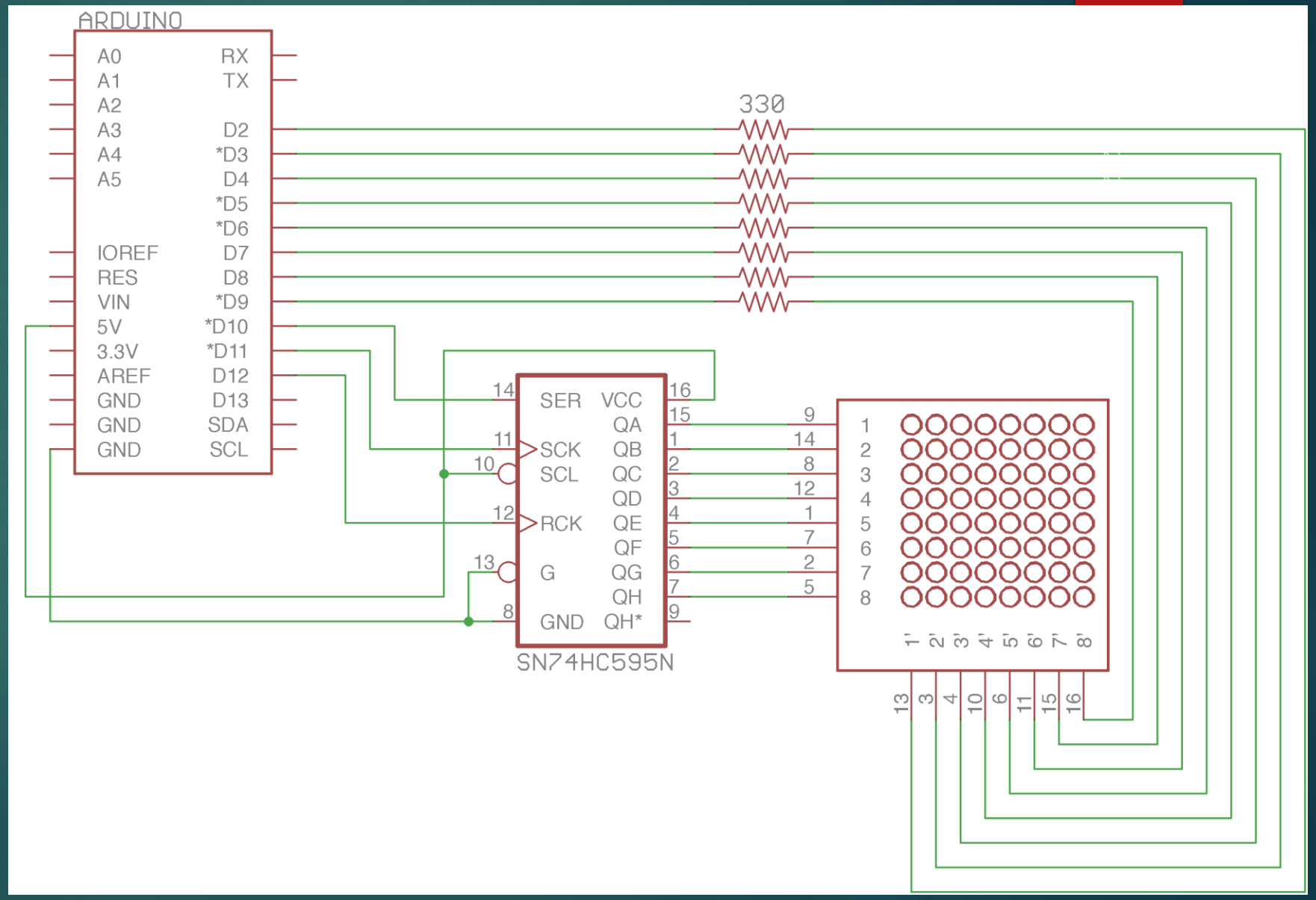
- Principe : la liste des pixels à allumer est stocker dans un tableau contenant huit octets.
- Chiffres0={0xFF,0x81,0x81,0x81,0x81,0x81,0x81,0xFF};

```
void ecritC(byte t[8]){

for (int i=0;i<=7;i++){
PORTD=t[i];
//PORTB=((PORTB &
0x80)>>7|PORTB<<1); //rotation à
gauche
//PORTB=((PORTB &
0x01)<<7|PORTB>>1);  //rotation à
droite
digitalWrite(c[7-i],LOW);  //mise à
0 de la colonne
delay(5);
digitalWrite(c[7-i],HIGH); //remise
à 1 de la colonne
}
}
```

Diminuer le nombre de lignes

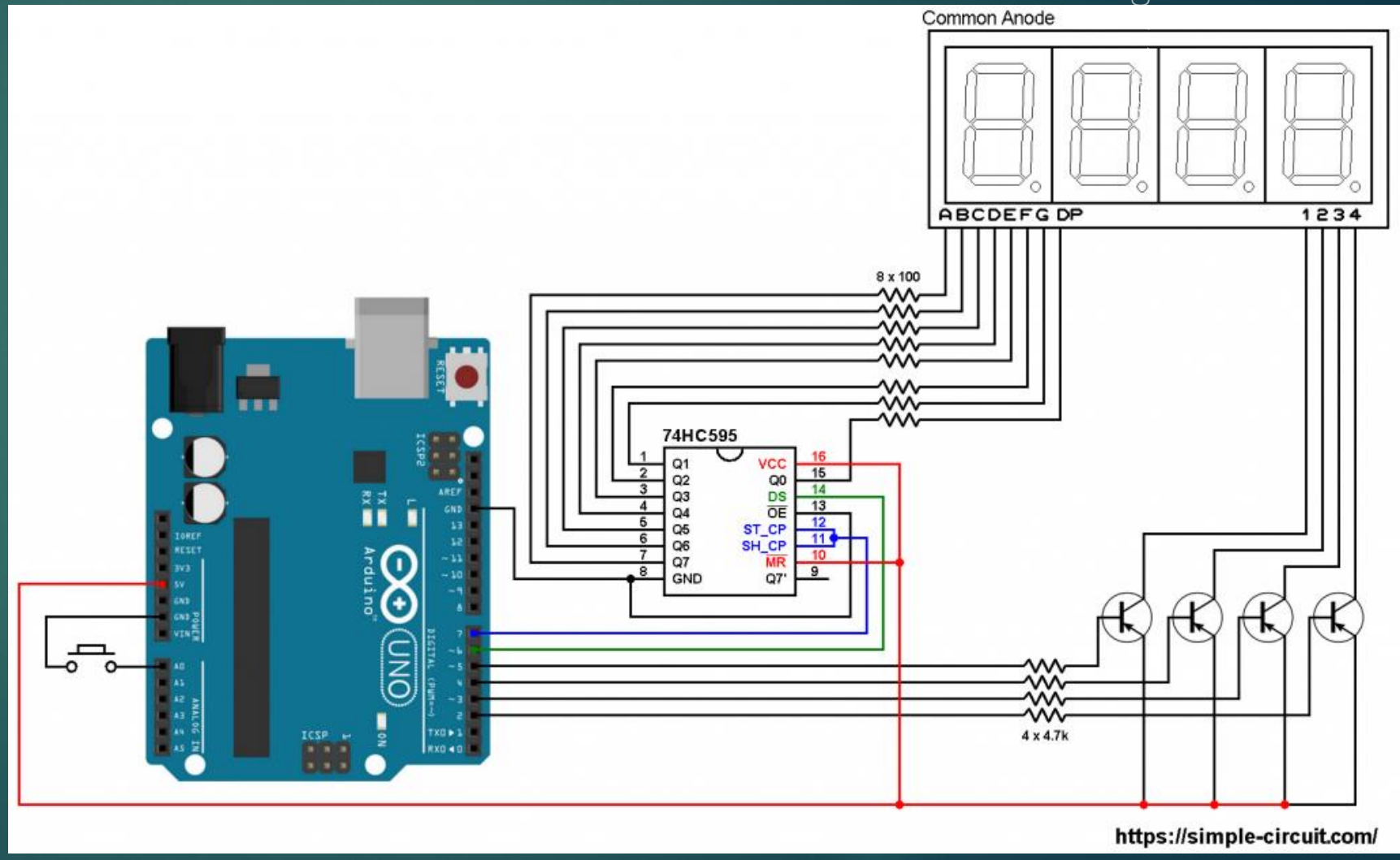
Le 74HC595 permet de réduire le nombre de fil nécessaires pour relier la carte Arduino à la matrice de Led



Afficheur 7 segments Arduino avec un circuit de registre à décalage 74HC595 avec Timer

Hardware Required:

- Arduino UNO board
- 4-Digit **common anode** 7-segment display
- 74HC595 shift register —-> datasheet
- 4 x PNP transistor (2SA1015, 2S9015, 2N3906 …)
- 8 x 100 ohm resistor
- 4 x 4.7k ohm resistor
- Push button
- Breadboard
- Jumper wires



Afficheur 7 segments Arduino avec un circuit de Timer et registre à décalage 74HC595


```
/*
* 7-segment display with 74HC595 shift register
* 4-Digit counter example.
* Common anode 7-segment display is used.
* This is a free software with NO WARRANTY.
* https://simple-circuit.com/
*/

// counter button definition
#define button    A0

// shift register pin definitions
#define clockPin  7   // clock pin
#define dataPin   6   // data pin

// common pins of the four digits definitions
#define Dig1    5
#define Dig2    4
#define Dig3    3
#define Dig4    2

// variable declarations
byte current_digit;
int  count = 0;
void disp(byte number, bool dec_point = 0);

void setup()
{
  pinMode(button, INPUT_PULLUP);
  pinMode(Dig1, OUTPUT);
  pinMode(Dig2, OUTPUT);
  pinMode(Dig3, OUTPUT);
  pinMode(Dig4, OUTPUT);
  pinMode(clockPin, OUTPUT);
  pinMode(dataPin, OUTPUT);

  disp_off();  // turn off the display

  // Timer1 module overflow interrupt configuration
  TCCR1A = 0;
  TCCR1B = 1;  // enable Timer1 with prescaler = 1 ( 16 ticks each 1 µs)
  TCNT1  = 0;  // set Timer1 preload value to 0 (reset)
  TIMSK1 = 1;  // enable Timer1 overflow interrupt
}
```

Afficheur 7 segments Arduino avec un circuit de Timer et registre à décalage 74HC595


```
 // Timer1 module overflow interrupt configuration
 TCCR1A = 0;
 TCCR1B = 1;  // enable Timer1 with prescaler = 1 ( 16 ticks each 1 µs)
 TCNT1  = 0;  // set Timer1 preload value to 0 (reset)
 TIMSK1 = 1;  // enable Timer1 overflow interrupt
}

ISR(TIMER1_OVF_vect)   // Timer1 interrupt service routine (ISR)
{
 disp_off();  // turn off the display

 switch (current_digit)
 {
  case 1:
   disp(count / 1000);   // prepare to display digit 1 (most left)
   digitalWrite(Dig1, LOW);  // turn on digit 1
   break;

  case 2:
   disp( (count / 100) % 10 );   // prepare to display digit 2
   digitalWrite(Dig2, LOW);     // turn on digit 2
   break;

  case 3:
   disp( (count / 10) % 10 );   // prepare to display digit 3
   digitalWrite(Dig3, LOW);    // turn on digit 3
   break;

  case 4:
   disp(count % 10);   // prepare to display digit 4 (most right)
   digitalWrite(Dig4, LOW);  // turn on digit 4
  }

 current_digit = (current_digit % 4) + 1;
}

// main loop
void loop()
{
 if(digitalRead(button) == 0)
 {
  count++;  // increment 'count' by 1
  if(count > 9999)
   count = 0;
  delay(200);  // wait 200 milliseconds
 }
}
```

Afficheur 7 segments Arduino avec un circuit de Timer et registre à décalage 74HC595


```
void disp(byte number, bool dec_point)
{
 switch (number)
 {
  case 0:  // print 0
   shiftOut(dataPin, clockPin, MSBFIRST, 0x02 |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 1:  // print 1
   shiftOut(dataPin, clockPin, MSBFIRST, 0x9E |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 2:  // print 2
   shiftOut(dataPin, clockPin, MSBFIRST, 0x24 |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 3:  // print 3
   shiftOut(dataPin, clockPin, MSBFIRST, 0x0C |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 4:  // print 4
   shiftOut(dataPin, clockPin, MSBFIRST, 0x98 |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 5:  // print 5
   shiftOut(dataPin, clockPin, MSBFIRST, 0x48 |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 6:  // print 6
   shiftOut(dataPin, clockPin, MSBFIRST, 0x40 |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 7:  // print 7
   shiftOut(dataPin, clockPin, MSBFIRST, 0x1E |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 8:  // print 8
   shiftOut(dataPin, clockPin, MSBFIRST, !dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
   break;

  case 9:  // print 9
   shiftOut(dataPin, clockPin, MSBFIRST, 0x08 |
!dec_point);
   digitalWrite(clockPin, HIGH);
   digitalWrite(clockPin, LOW);
 }
}
 void disp_off()
{
  digitalWrite(Dig1, HIGH);
  digitalWrite(Dig2, HIGH);
  digitalWrite(Dig3, HIGH);
  digitalWrite(Dig4, HIGH);
}

// end of code.
```

05/12/2026

Utilisation du Circuit MAX7219

- Focntionnalités
- Brochage
- Applications typiques

Utilisation du Circuit MAX7219

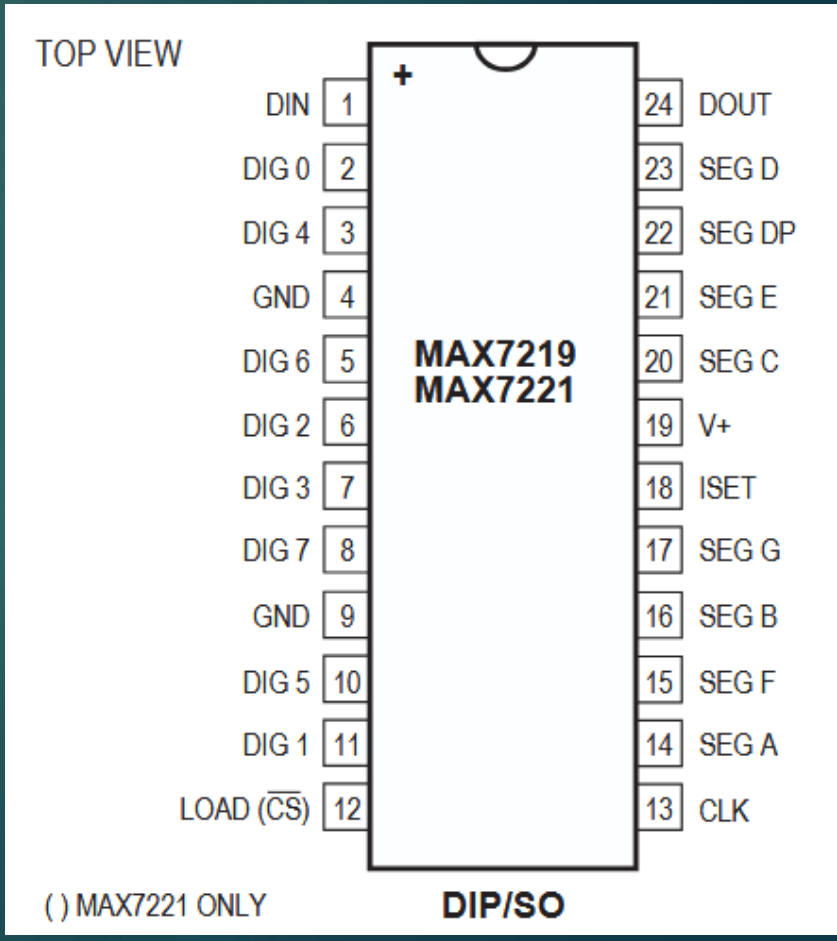
- Focntionnalités
- Brochage
- Applications typiques

Bar-Graph Displays
7-Segment Displays
Industrial Controllers
Panel Meters
LED Matrix Displays

Utilisation du Circuit MAX7219

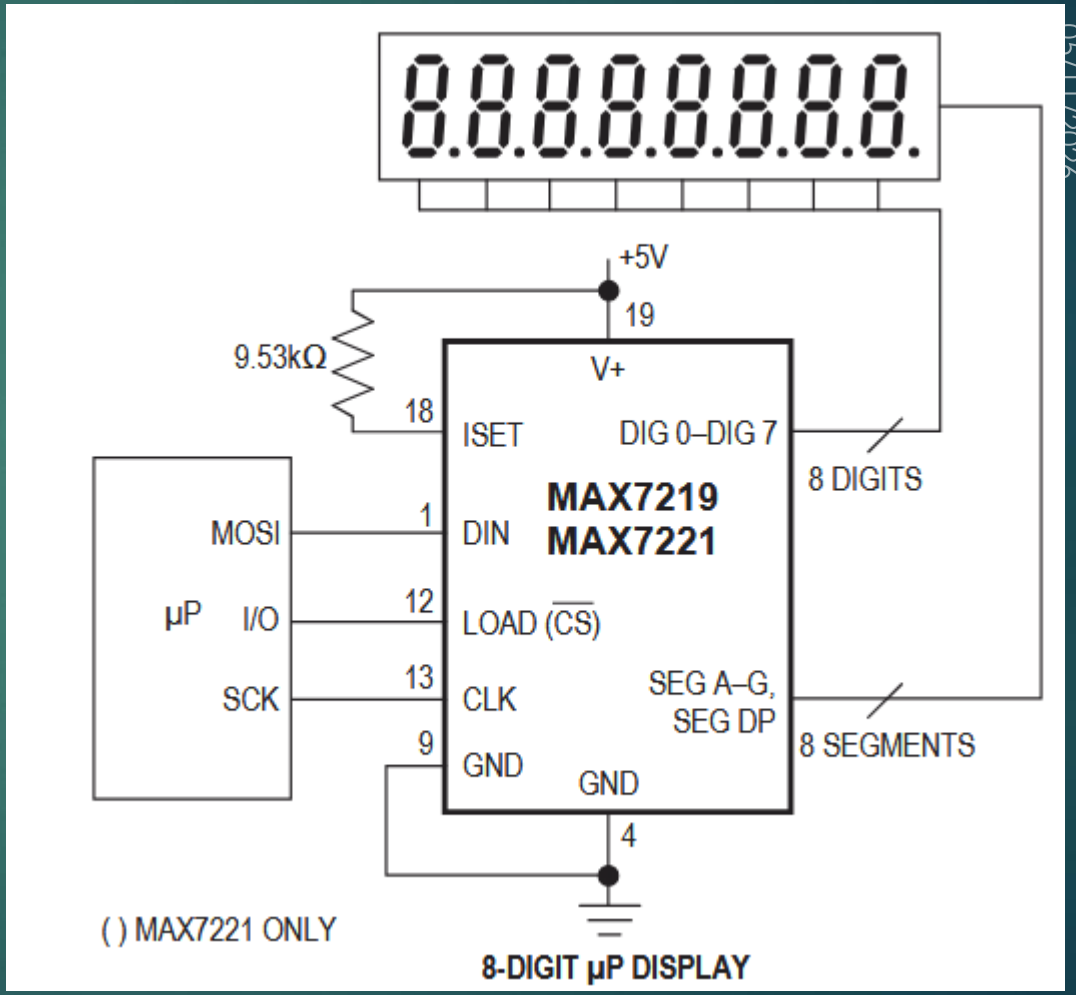
- Brochage

PIN	NAME	FUNCTION
1	DIN	Serial-Data Input. Data is loaded into the internal 16-bit shift register on CLK's rising edge.
2, 3, 5–8, 10, 11	DIG 0–DIG 7	Eight-digit drive lines that sink current from the display common cathode. The MAX7219 pulls the digit outputs to V+ when turned off. The MAX7221's digit drivers are high-impedance when turned off.
4, 9	GND	Ground. Both GND pins must be connected.
12	LOAD (MAX7219)	Load-Data Input. The last 16 bits of serial data are latched on LOAD's rising edge.
	$\overline{CS}$ (MAX7221)	Chip-Select Input. Serial data is loaded into the shift register while $\overline{CS}$ is low. The last 16 bits of serial data are latched on $\overline{CS}$'s rising edge.
13	CLK	Serial-Clock Input. 10MHz maximum rate. On CLK's rising edge, data is shifted into the internal shift register. On CLK's falling edge, data is clocked out of DOUT. On the MAX7221, the CLK input is active only while $\overline{CS}$ is low.
14–17, 20–23	SEG A–SEG G, DP	Seven Segment Drives and Decimal Point Drive that source current to the display. On the MAX7219, when a segment driver is turned off it is pulled to GND. The MAX7221 segment drivers are high-impedance when turned off.
18	ISET	Connect to V_{DD} through a resistor (R_{SET}) to set the peak segment current (Refer to *Selecting R_{SET} Resistor and Using External Drivers* section).
19	V+	Positive Supply Voltage. Connect to +5V.
24	DOUT	Serial-Data Output. The data into DIN is valid at DOUT 16.5 clock cycles later. This pin is used to daisy-chain several MAX7219/MAX7221's and is never high-impedance.



Utilisation du Circuit MAX7219

- Focntionnalités
- Brochage
- Applications typiques

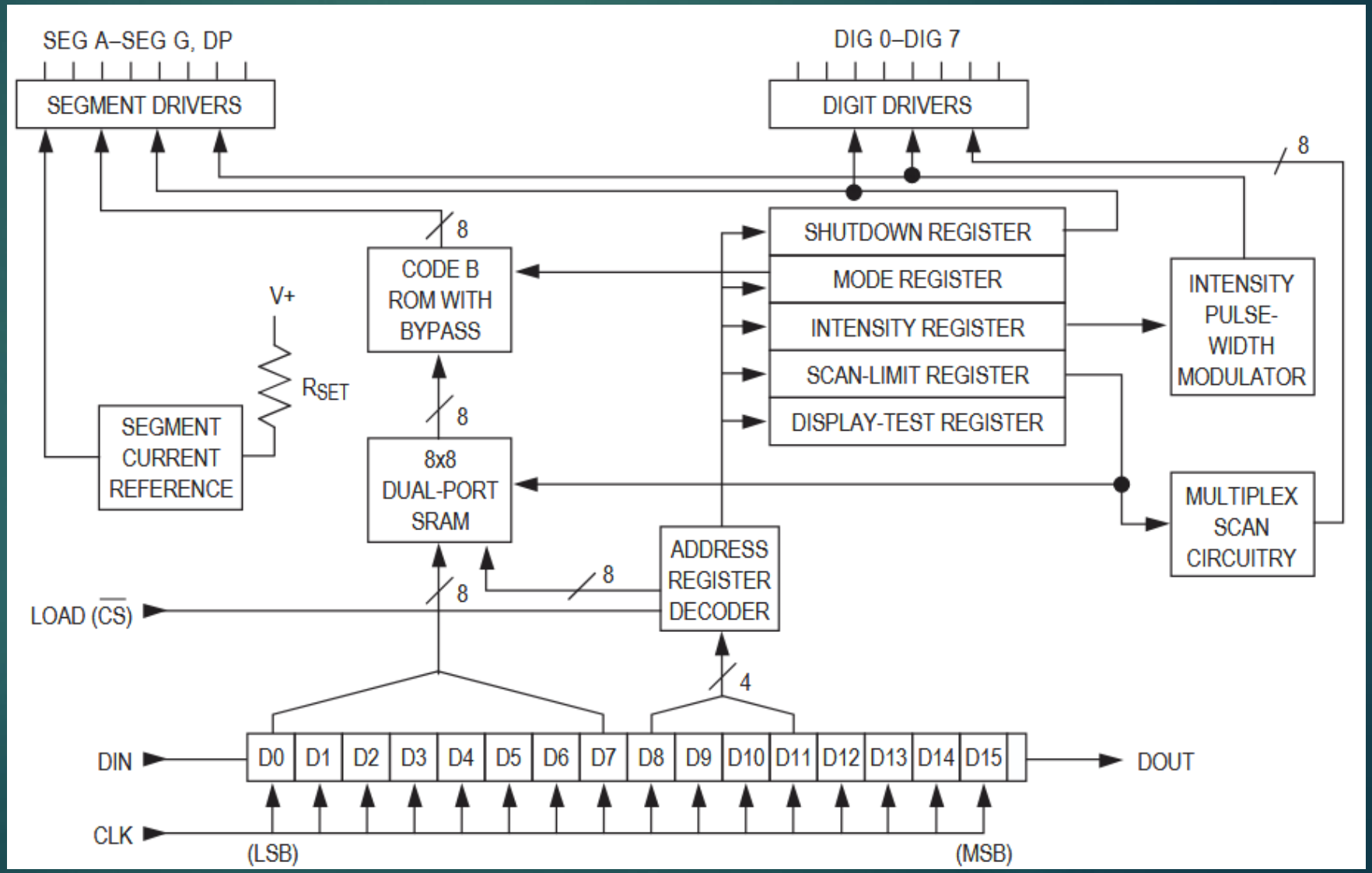


Utilisation du Circuit MAX7219

- Focntionnalités
- Brochage
- Applications typiques

Utilisation du Circuit MAX7219

- Fonctionnement interne



Description du circuit MAX7219

- Description générale

Les MAX7219/MAX7221 sont des pilotes d'affichage compacts à cathode commune, avec entrées/sorties série, qui connectent des microprocesseurs (µP) à des afficheurs LED numériques à 7 segments (jusqu'à 8 chiffres), des afficheurs à barres graphiques ou 64 LED individuelles.

La puce intègre un décodeur BCD code-B, un circuit de balayage multiplexé, des pilotes de segments et de chiffres, ainsi qu'une mémoire RAM statique 8x8 stockant chaque chiffre. Une seule résistance externe est nécessaire pour régler le courant de segment de toutes les LED.

Le MAX7221 est compatible avec SPI™, QSPI™ et MICROWIRE™, et dispose de pilotes de segments à vitesse de balayage limitée pour réduire les interférences électromagnétiques. Une interface série 4 fils pratique permet de connecter tous les µP communs.

Chaque chiffre peut être adressé et mis à jour individuellement sans réécrire l'ensemble de l'affichage. Les MAX7219/MAX7221 permettent également à l'utilisateur de sélectionner le décodage code-B ou l'absence de décodage pour chaque chiffre.

Description du circuit MAX7219

- **Modes d'adressage série Pour le MAX7219**

Les données série DIN, envoyées par paquets de 16 bits, sont décalées vers le registre à décalage interne 16 bits à chaque front montant de CLK, quel que soit l'état de LOAD.

Pour le MAX7221, CS doit être à l'état bas pour synchroniser les données en entrée ou en sortie. Les données sont ensuite verrouillées dans le registre de chiffres ou de contrôle sur le front montant de LOAD/CS.

LOAD/CS doit passer à l'état haut simultanément ou après le 16è front montant, mais avant le front montant suivant, sous peine de perte de données.

Les données DIN sont propagées dans le registre à décalage et apparaissent sur DOUT 16,5 cycles d'horloge plus tard.

Les données sont synchronisées sur le front descendant de CLK.

Les bits de données sont étiquetés D0 à D15.

D8 à D11 contiennent l'adresse du registre. D0 à D7 contiennent les données, et D12 à D15 sont les bits « sans importance ».

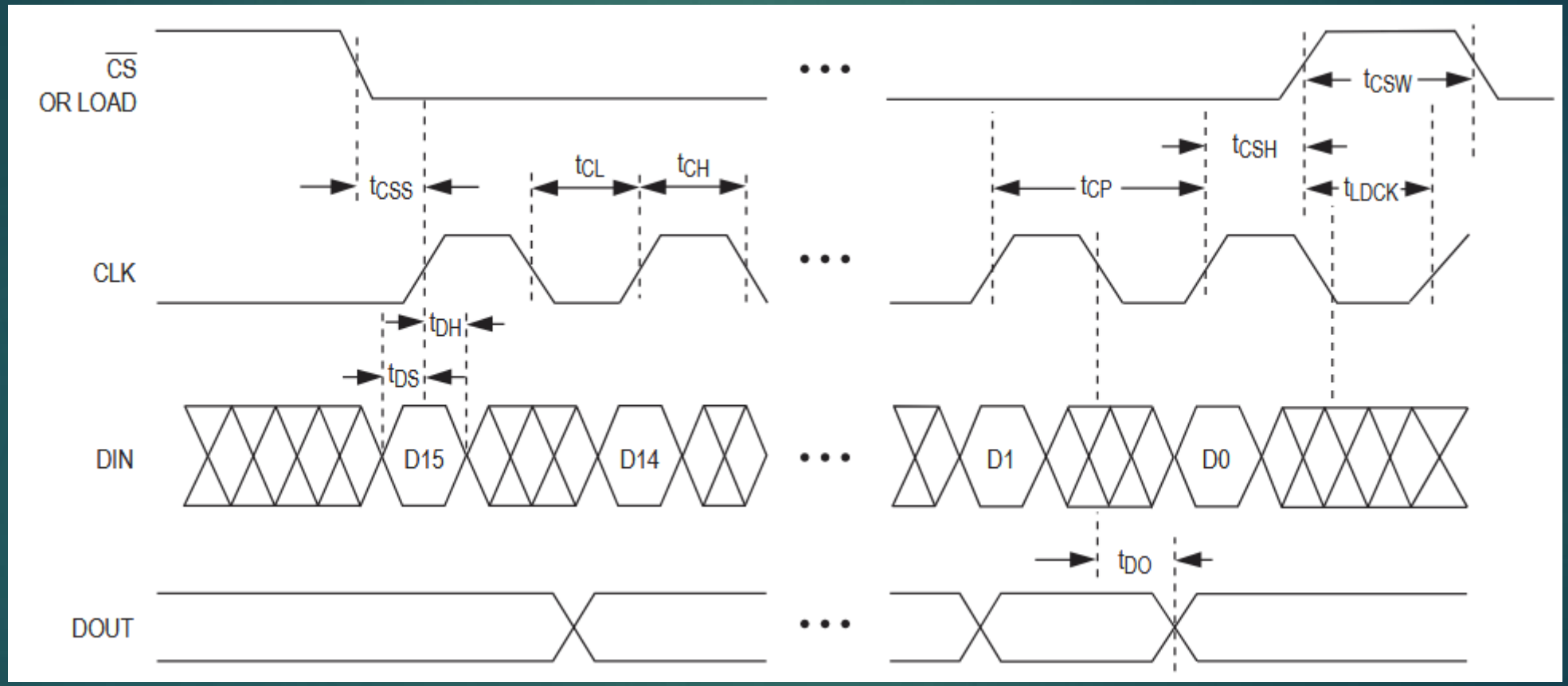
Le premier bit reçu est D15, le bit de poids fort (MSB).

Description du circuit MAX7219

- **Registres de chiffres et de contrôle**
- Le tableau 2 répertorie les 14 registres de chiffres et de contrôle adressables.
- Les registres de chiffres sont réalisés avec une SRAM double port 8x8 intégrée. Ils sont adressés directement afin que les chiffres individuels puissent être mis à jour et conserver les données tant que V+ dépasse généralement 2 V.
- Les registres de contrôle comprennent :
  - le mode de décodage,
  - l'intensité d'affichage,
  - la limite de balayage (nombre de chiffres balayés),
  - l'arrêt
  - et le test d'affichage (toutes les LED allumées).

Description du circuit MAX7219

- **Chronograme**

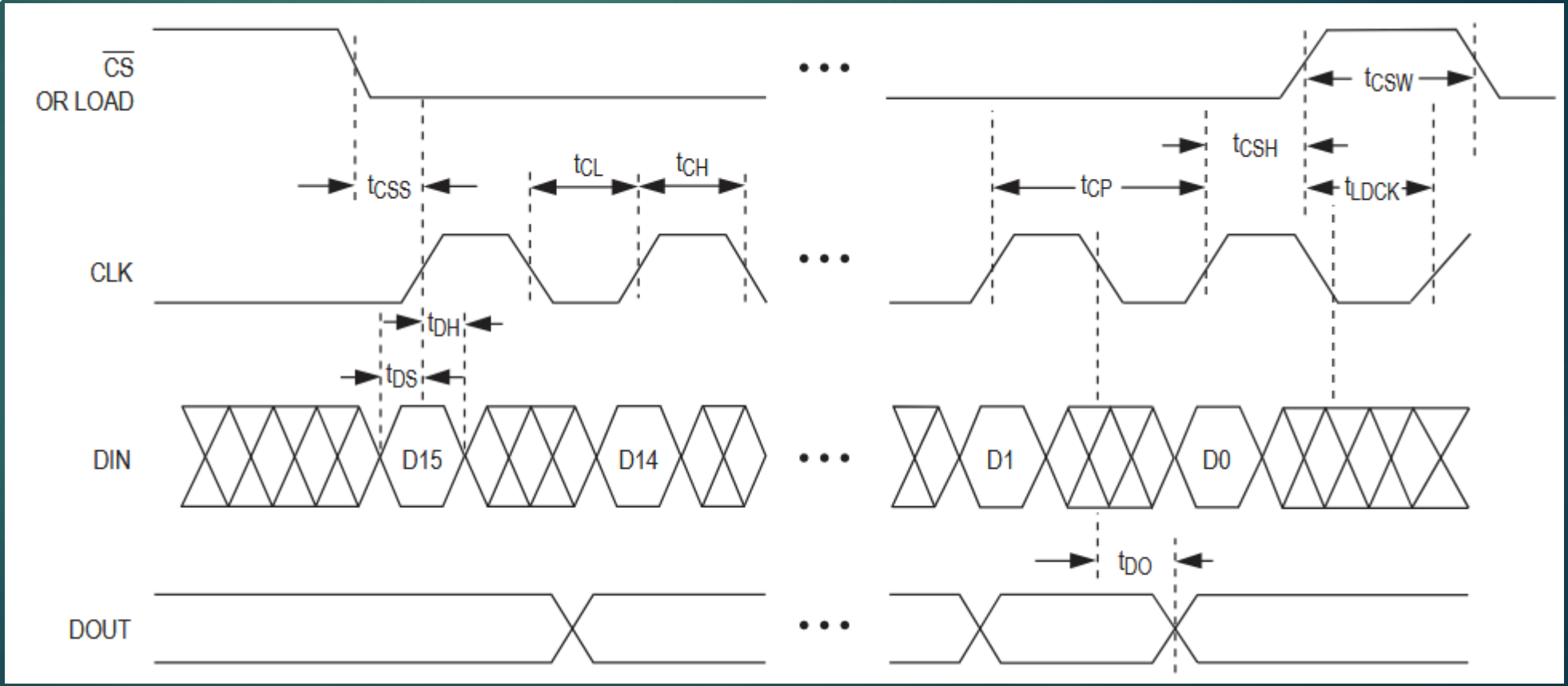


Description du circuit MAX7219

- **Format des données**

D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
X	X	X	X	ADDRESS				MSB	DATA						LSB

- **Chronograme**



Description du circuit MAX7219

- **Table d'adressage des registres**

REGISTER	ADDRESS					HEX CODE
	D15–D12	D11	D10	D9	D8	
No-Op	X	0	0	0	0	0xX0
Digit 0	X	0	0	0	1	0xX1
Digit 1	X	0	0	1	0	0xX2
Digit 2	X	0	0	1	1	0xX3
Digit 3	X	0	1	0	0	0xX4
Digit 4	X	0	1	0	1	0xX5
Digit 5	X	0	1	1	0	0xX6
Digit 6	X	0	1	1	1	0xX7
Digit 7	X	1	0	0	0	0xX8
Decode Mode	X	1	0	0	1	0xX9
Intensity	X	1	0	1	0	0xXA
Scan Limit	X	1	0	1	1	0xXB
Shutdown	X	1	1	0	0	0xXC
Display Test	X	1	1	1	1	0xXF

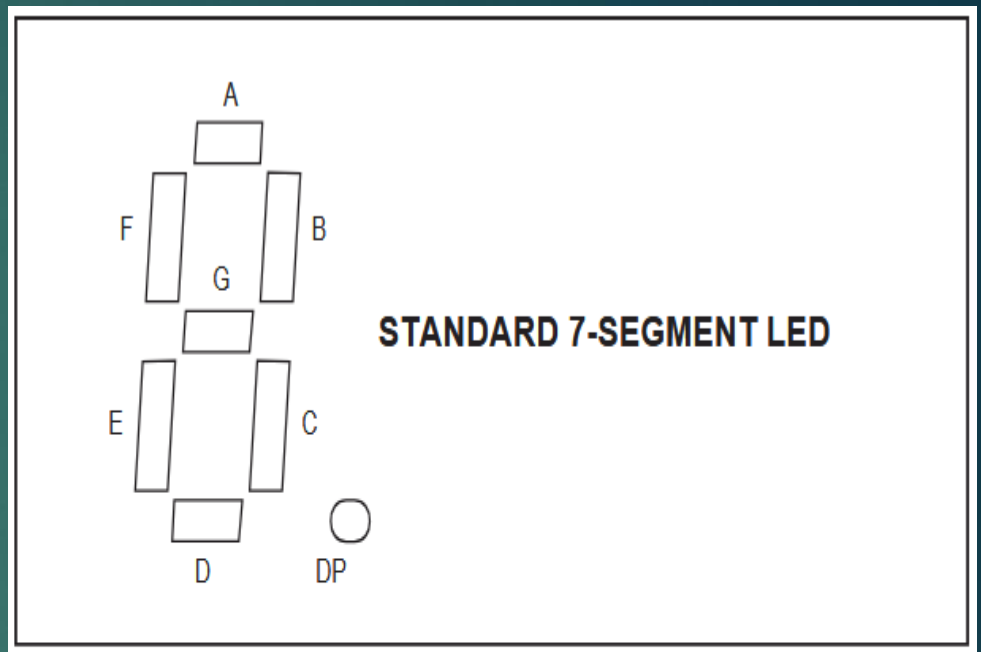
Description du circuit MAX7219

- **Registre du mode de décodage**
- Le registre du mode de décodage définit le code BCD B (0-9, E, H, L, P et -) ou l'absence de décodage pour chaque chiffre.
- Chaque bit du registre correspond à un chiffre. Un niveau logique haut sélectionne le décodage du code B(binaire), tandis qu'un niveau logique bas contourne le décodeur. Le tableau ci-dessous présente des exemples de format du registre de contrôle du mode de décodage. Lorsque le mode de décodage du code B est utilisé, le décodeur examine uniquement le quartet inférieur des données dans les registres de chiffres (D3 à D0), ignorant les bits D4 à D6.
- D7, qui définit le point décimal (SEG DP), est indépendant du décodeur et est en logique positive (D7 = 1 active le point décimal).

DECODE MODE	REGISTER DATA								HEX CODE
	D7	D6	D5	D4	D3	D2	D1	D0	
No decode for digits 7–0	0	0	0	0	0	0	0	0	0x00
Code B decode for digit 0 No decode for digits 7–1	0	0	0	0	0	0	0	1	0x01
Code B decode for digits 3–0 No decode for digits 7–4	0	0	0	0	1	1	1	1	0x0F
Code B decode for digits 7–0	1	1	1	1	1	1	1	1	0xFF

Description du circuit MAX7219

- **Registre du mode de décodage**
- Le registre du mode de décodage définit le code BCD B (0-9, E, H, L, P et -) ou l'absence de décodage pour chaque chiffre.
- Lorsque l'absence de décodage est sélectionnée, les bits de données D7 à D0 correspondent aux lignes de segment du MAX7219/MAX7221. Voir figure ci-contre



	REGISTER DATA							
	D7	D6	D5	D4	D3	D2	D1	D0
Corresponding Segment Line	DP	A	B	C	D	E	F	G

Description du circuit MAX7219

- **Contrôle d'intensité et masquage inter-chiffres**
- Les MAX7219/MAX7221 permettent de contrôler la luminosité de l'écran grâce à une résistance externe (RSET) connectée entre V+ et ISET.
- Le courant de crête provenant des pilotes de segment est nominalement 100 fois supérieur au courant entrant dans ISET. Cette résistance peut être fixe ou variable pour permettre le réglage de la luminosité depuis la face avant. Sa valeur minimale est de 9,53 kΩ, ce qui fixe généralement le courant de segment à 40 mA.
- La luminosité de l'écran peut également être contrôlée numériquement grâce au registre d'intensité.
- Le contrôle numérique de la luminosité de l'écran est assuré par un modulateur de largeur d'impulsion interne, commandé par le quartet inférieur du registre d'intensité.
- Le modulateur ajuste le courant moyen du segment en 16 étapes, d'un maximum de 31/32 à 1/32 du courant de crête défini par RSET (15/16 à 1/16 sur le MAX7221).
- Le tableau 7 présente le format du registre d'intensité. Le temps minimal de suppression inter-chiffres est fixé à 1/32 de cycle.

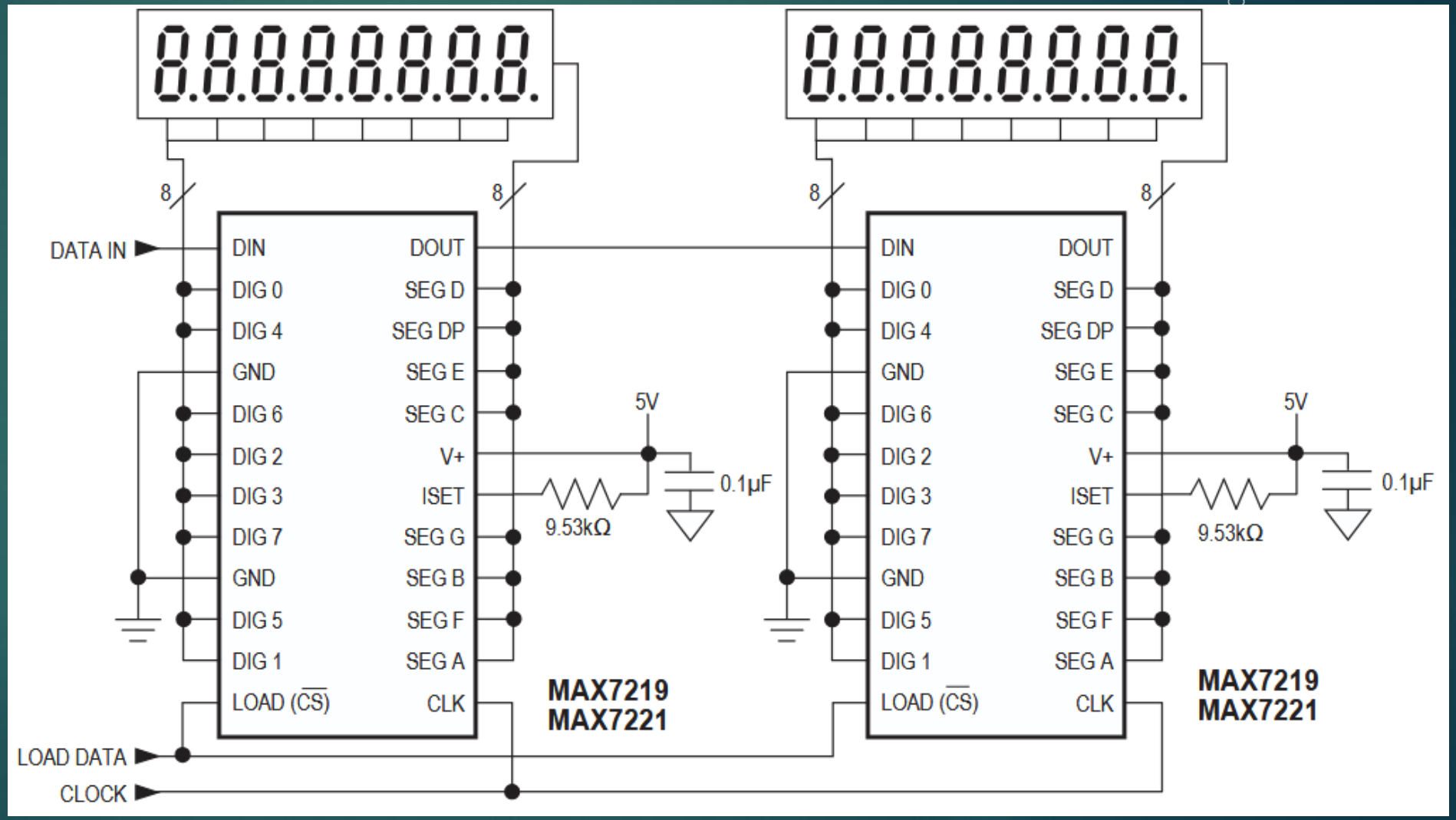
Description du circuit MAX7219

- **Contrôle d'intensité et masquage inter-chiffres**
- Le tableau ci-contre présente le format du registre d'intensité.
- Le temps minimal de suppression inter-chiffres est fixé à 1/32 de cycle.

DUTY CYCLE		D7	D6	D5	D4	D3	D2	D1	D0	HEX CODE
MAX7219	MAX7221									
1/32 (min on)	1/16 (min on)	X	X	X	X	0	0	0	0	0xX0
3/32	2/16	X	X	X	X	0	0	0	1	0xX1
5/32	3/16	X	X	X	X	0	0	1	0	0xX2
7/32	4/16	X	X	X	X	0	0	1	1	0xX3
9/32	5/16	X	X	X	X	0	1	0	0	0xX4
11/32	6/16	X	X	X	X	0	1	0	1	0xX5
13/32	7/16	X	X	X	X	0	1	1	0	0xX6
15/32	8/16	X	X	X	X	0	1	1	1	0xX7
17/32	9/16	X	X	X	X	1	0	0	0	0xX8
19/32	10/16	X	X	X	X	1	0	0	1	0xX9
21/32	11/16	X	X	X	X	1	0	1	0	0xXA
23/32	12/16	X	X	X	X	1	0	1	1	0xXB
25/32	13/16	X	X	X	X	1	1	0	0	0xXC
27/32	14/16	X	X	X	X	1	1	0	1	0xXD
29/32	15/16	X	X	X	X	1	1	1	0	0xXE
31/32	15/16 (max on)	X	X	X	X	1	1	1	1	0xXF

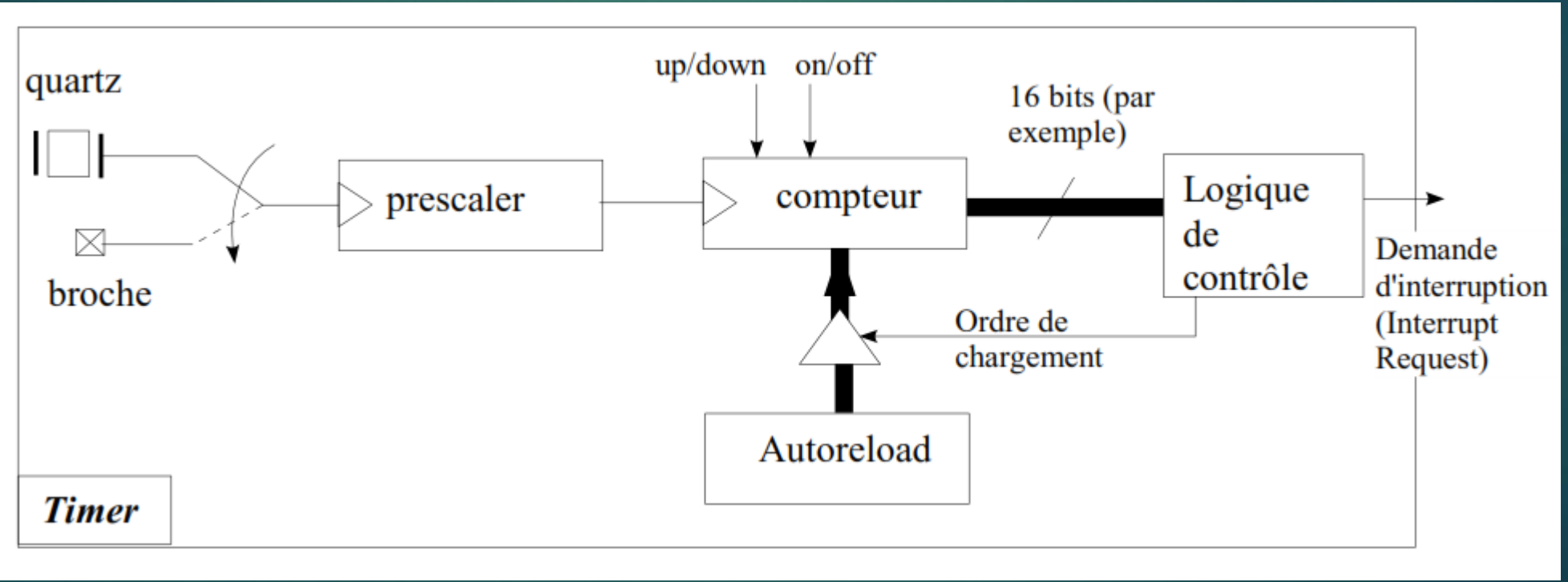
Utilisation du Circuit MAX7219

- Applications typiques :

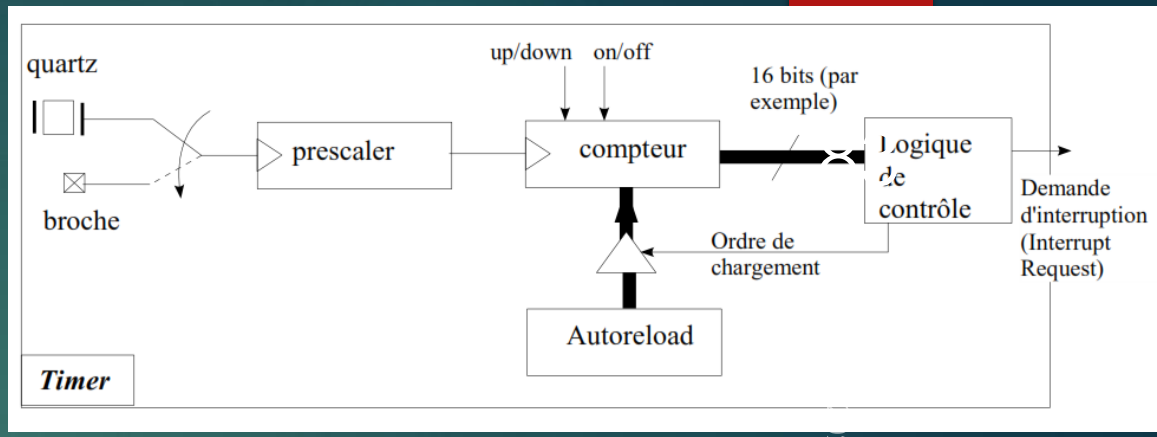


Interface périphérique Timer

- Principe



Interface périphérique Timer

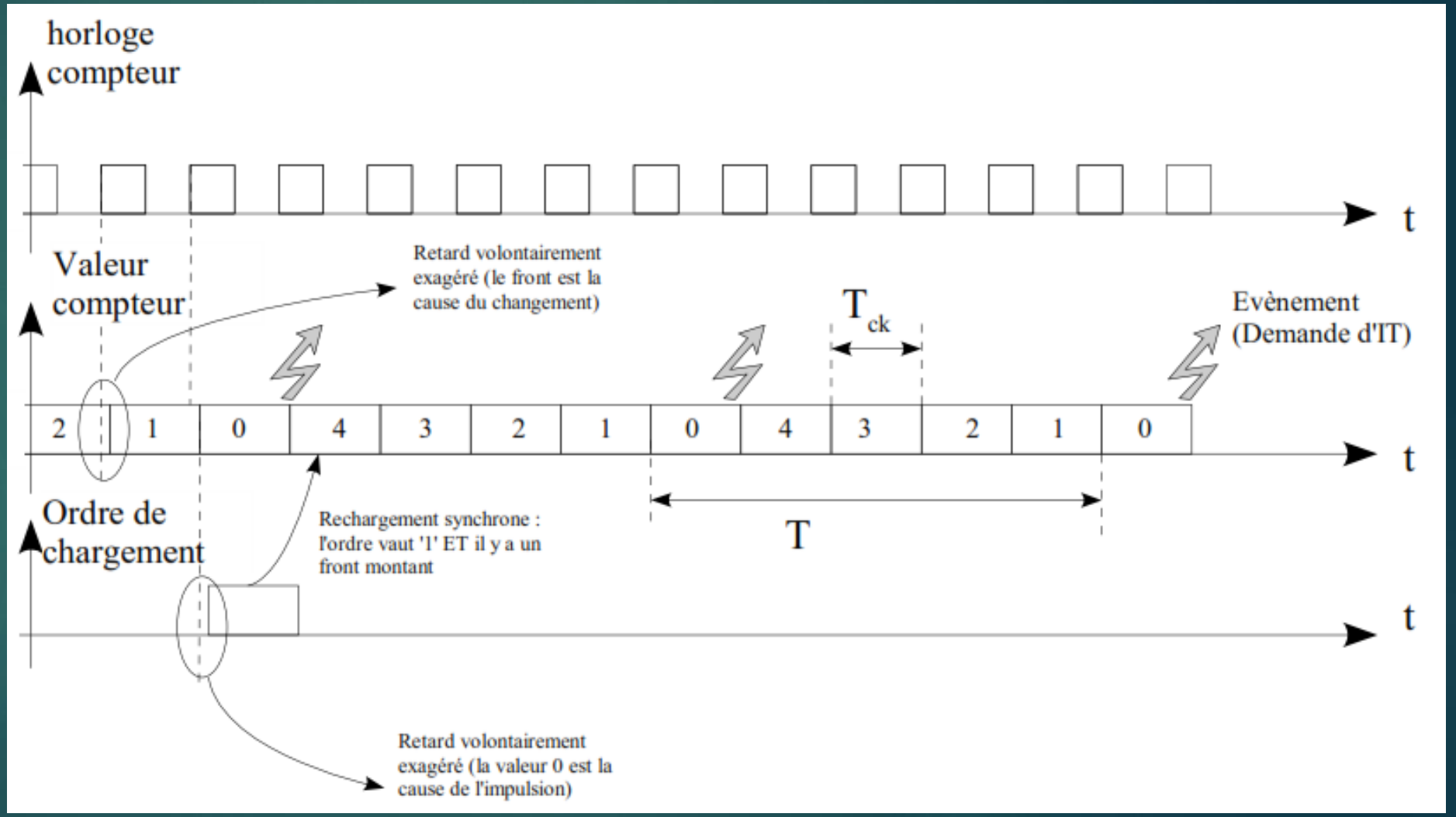


- Principe
- Le Timer/compteur est un périphérique utilisé pour mesurer les durées ou pour compter les impulsions externes.
- En mode Timer il compte les impulsions d'un oscillateur de référence (à quartz) de fréquence connue. Le module prescaler est un prédiviseur configurable qui fournit les impulsions d'horloge au module compteur,
- Le module compteur peut incrémenter (ou décrémenter) la valeur de son registre interne à chaque impulsion reçue. La taille du registre indique la durée maximale que le compteur peut mesurer, ou le nombre d'impulsions max qu'il peut compter
- La logique de contrôle permet de détecter la valeur maxi du registre du compteur pour générer un signal de dépassement (overflow) qui peut aussi générer une interruption Timer
- En fixant la valeur initial du registre compteur et sachant la durée entre deux implusions du prescaler, il est possible de calculer le temps écoulé lorsque le compteur arrive à la valeur maxi et génère le signal d'interruption
- Le circuit d'autorelaod permet de recharger le registre du compteur pour un nouveau comptage/mesure de durée lorsque la valeur maxi est atteinte.

Interface périphérique Timer

- Chronogramme

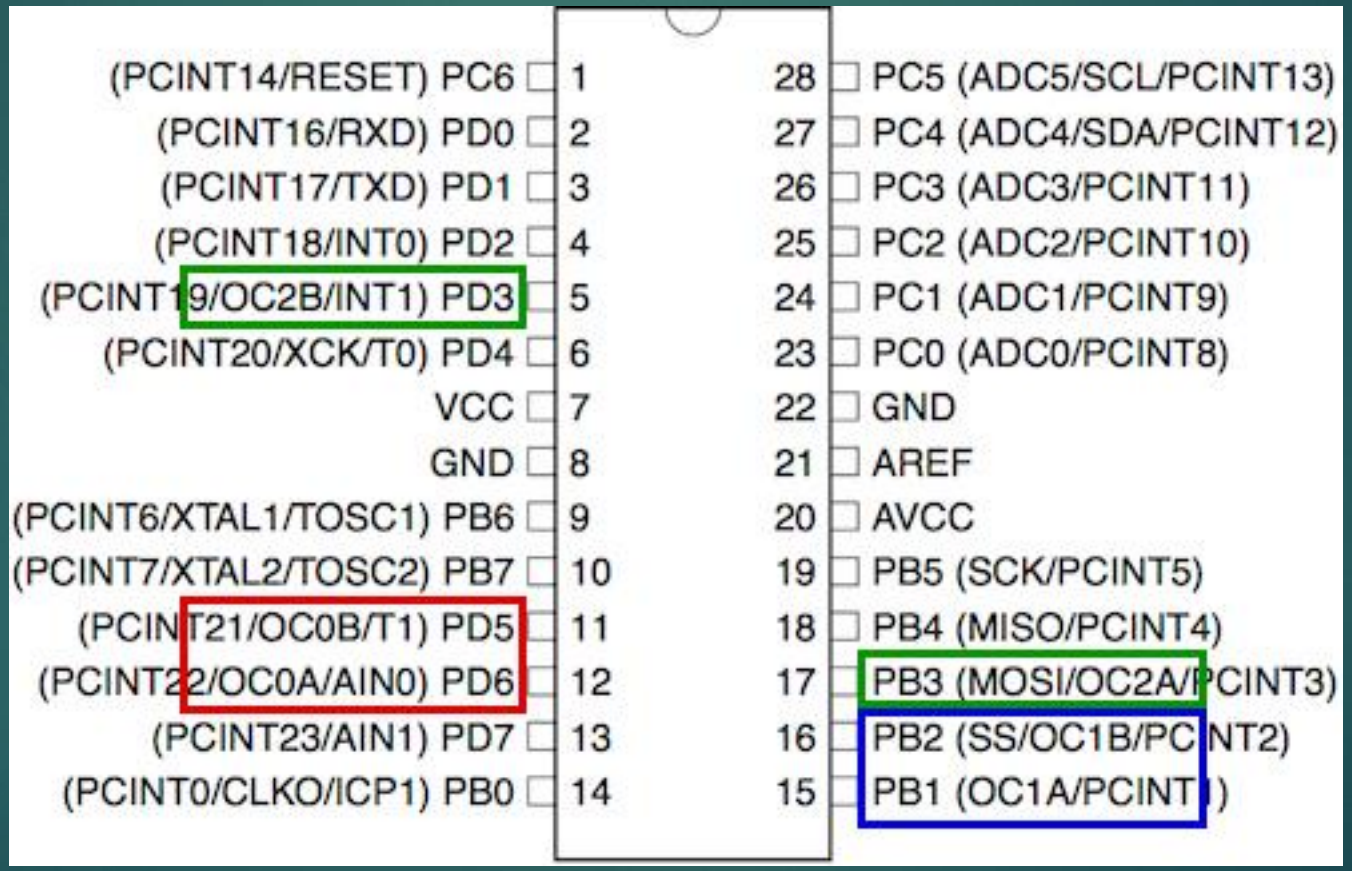
- *On suppose un mode de décomptage (down) et une valeur d'autoreload de 4 et qu'au départ, le compteur a la valeur 2.*



Utilisation de Timer sous Arduino

- Direcctement par accès au registre du périphérique
- en utilisant les fonctions de l'API Arduino

Broche des Timers sur ATMEGA 328



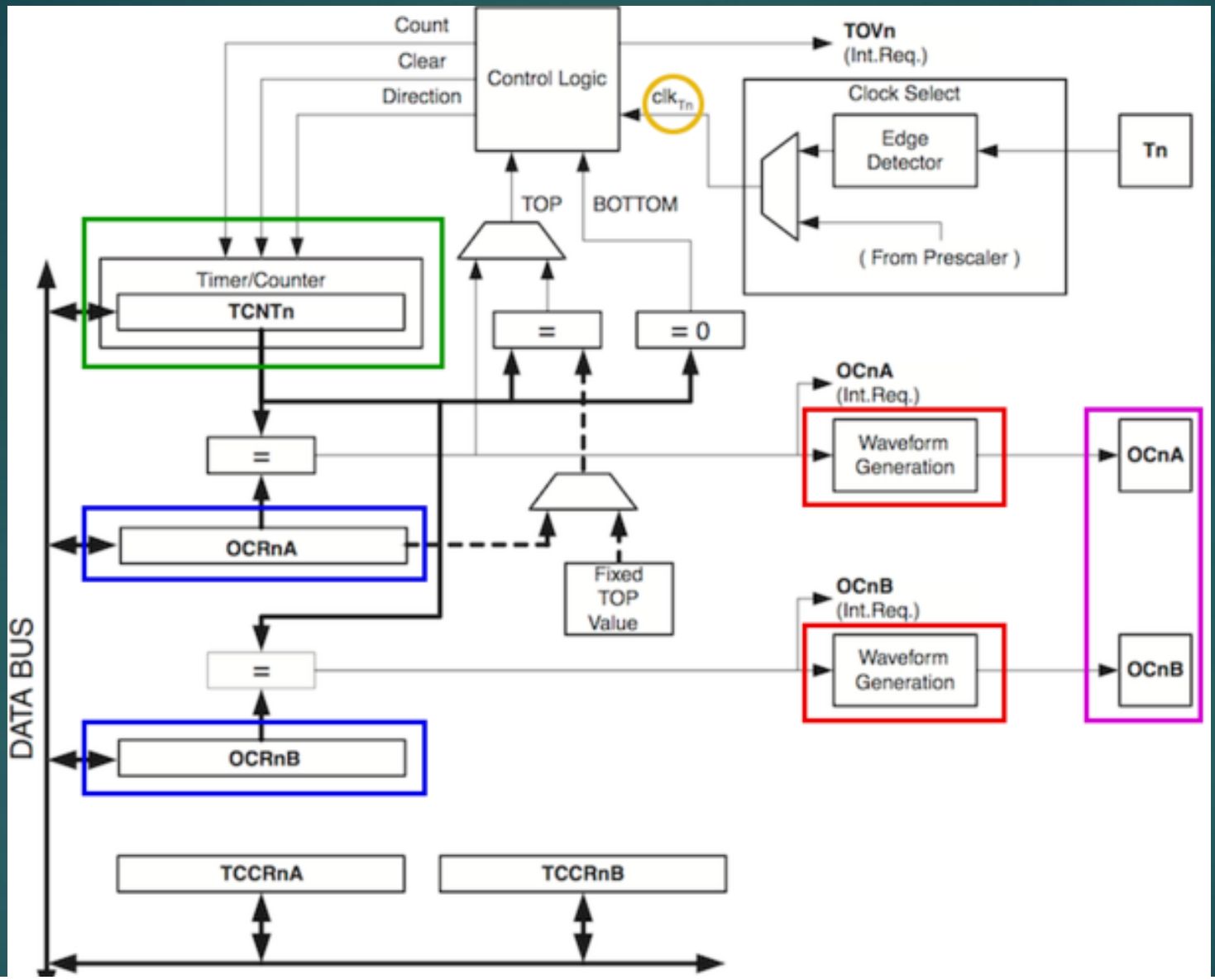
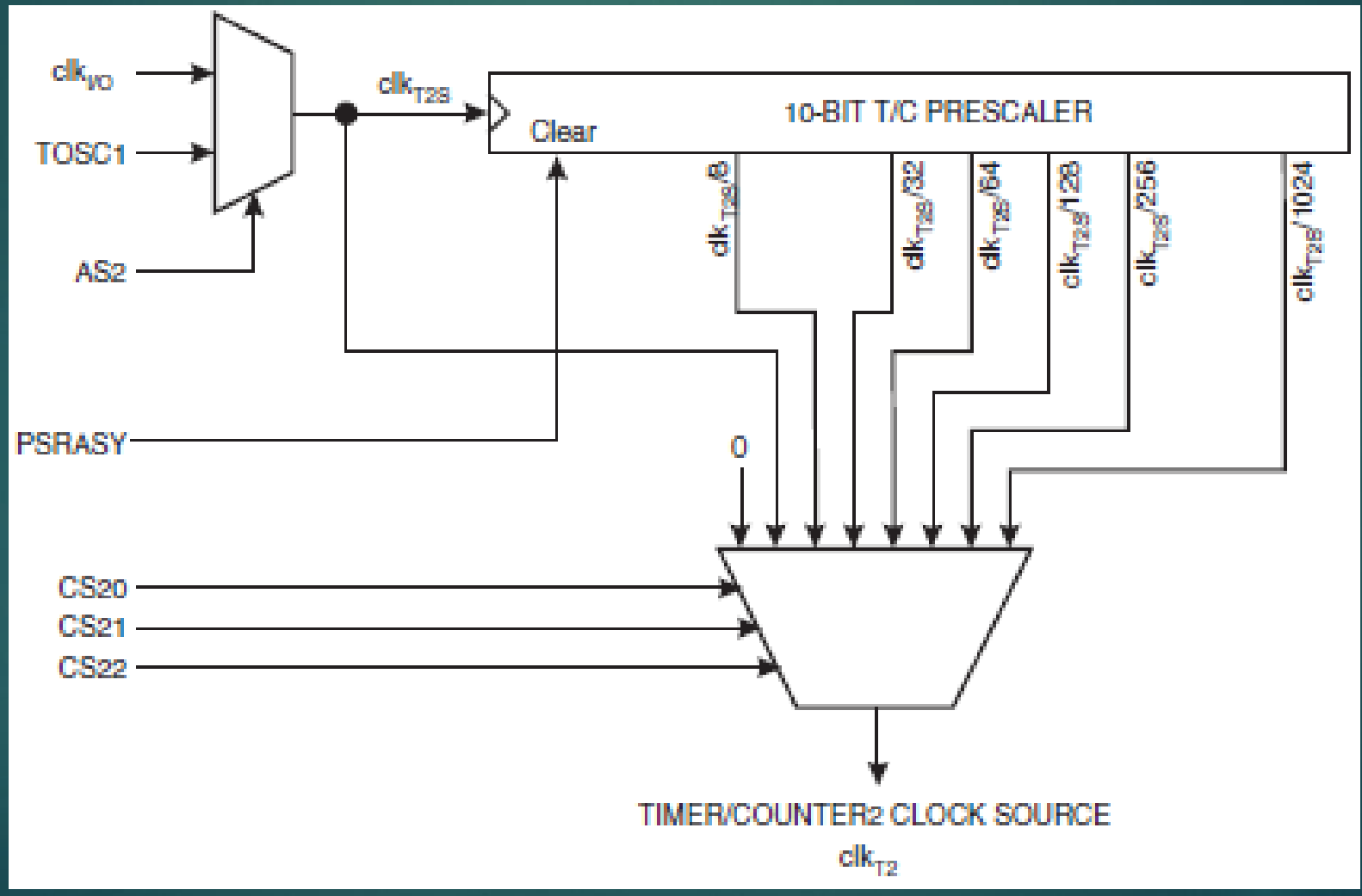
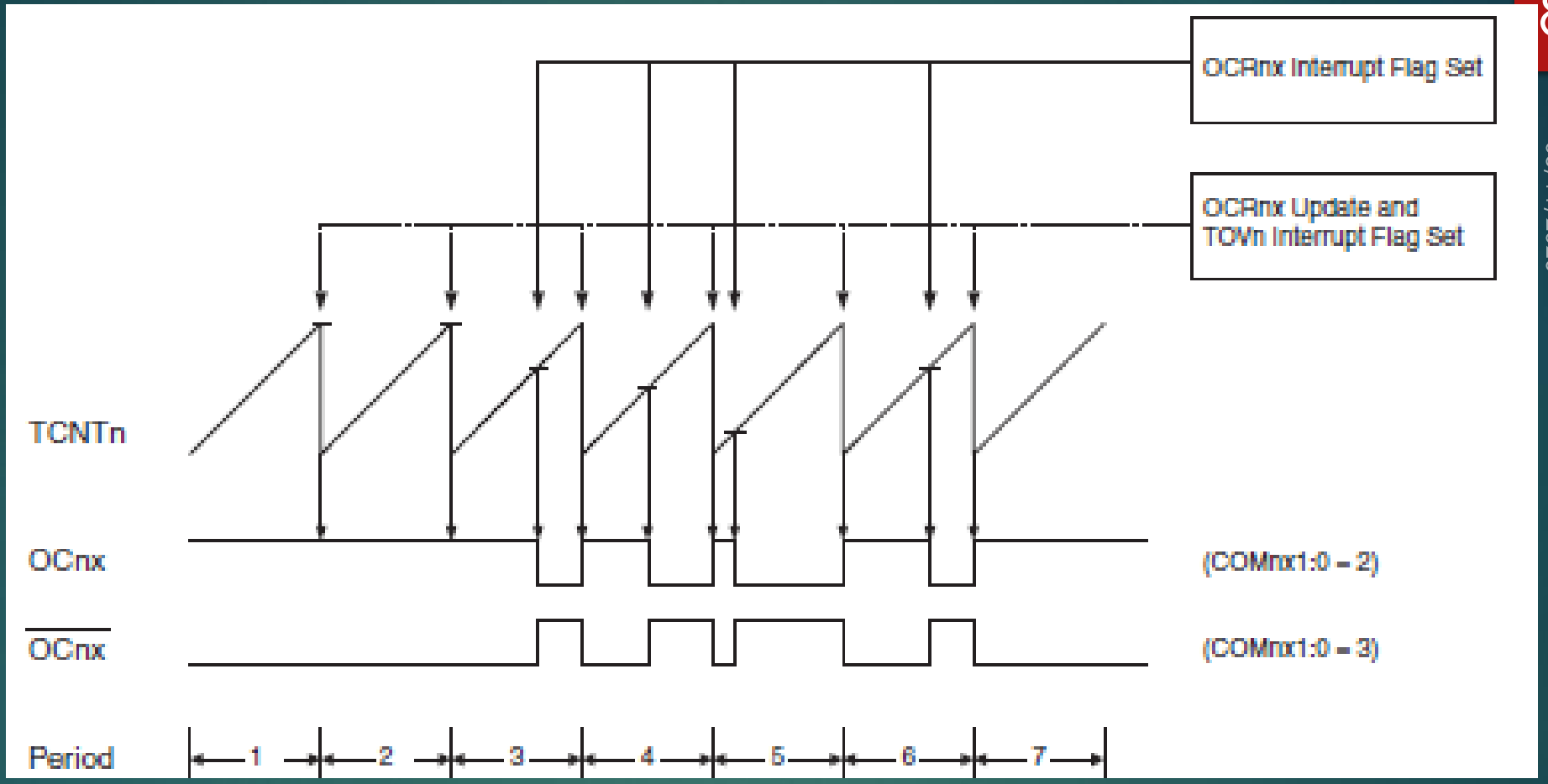


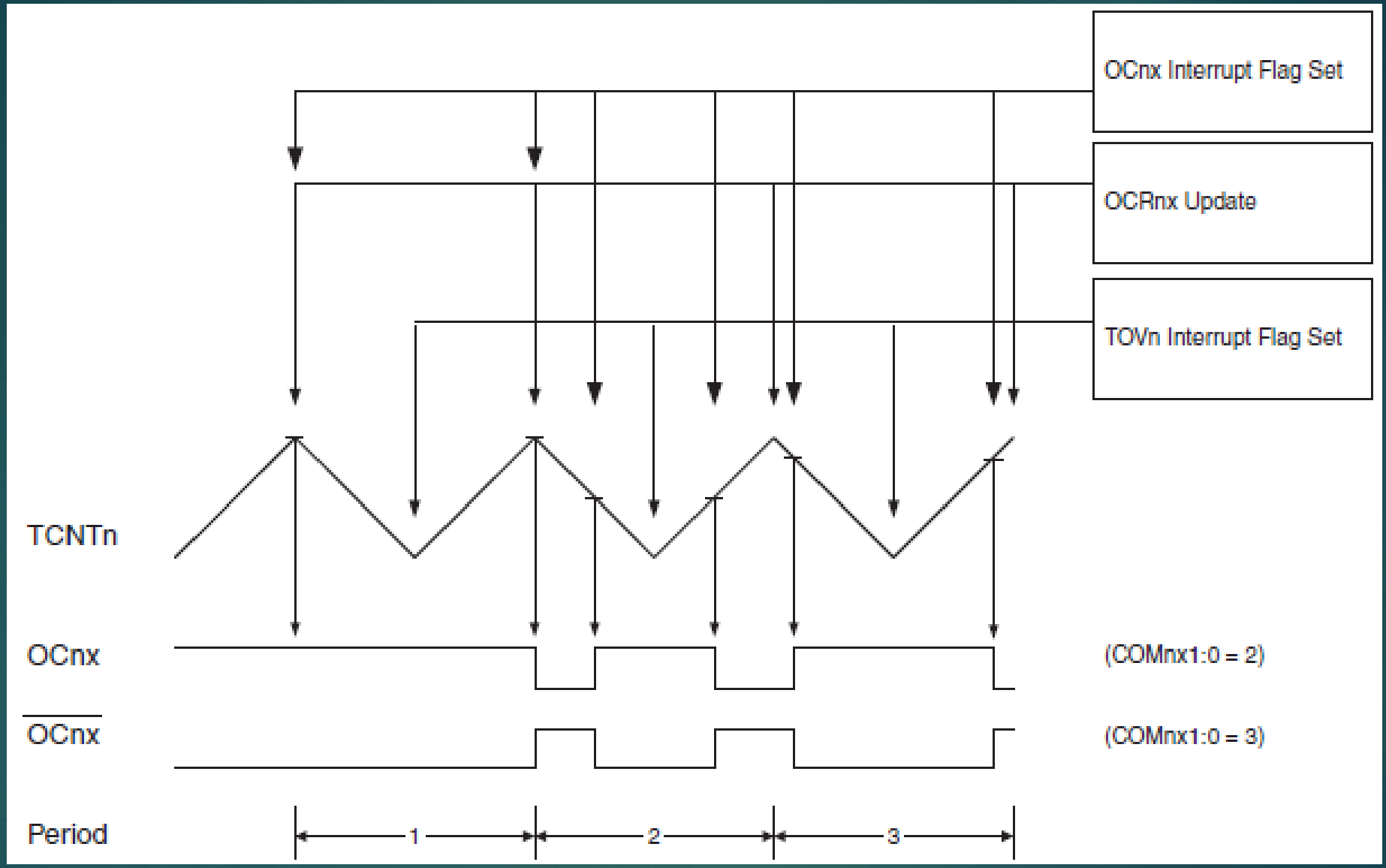
Figure 1 Architecture du bloc timer à l'intérieur du microcontrôleur ATmega328P

Pré-diviseur (Prescaler) Timer2





Fast PWM Mode, Timing Diagram



Phase Correct PWM Mode, Timing Diagram

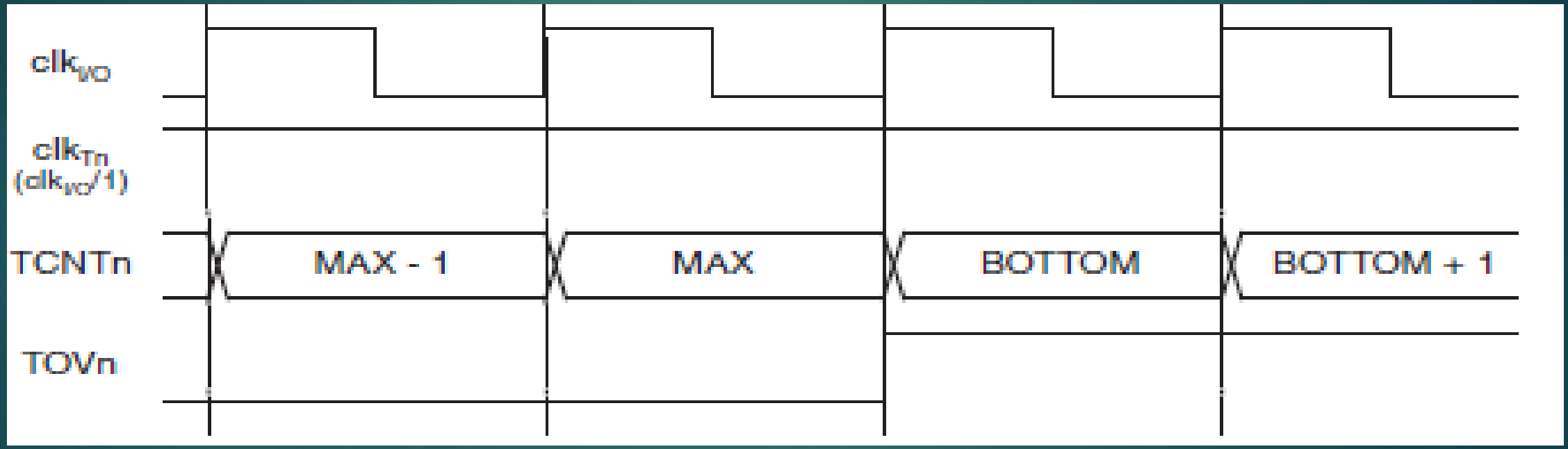
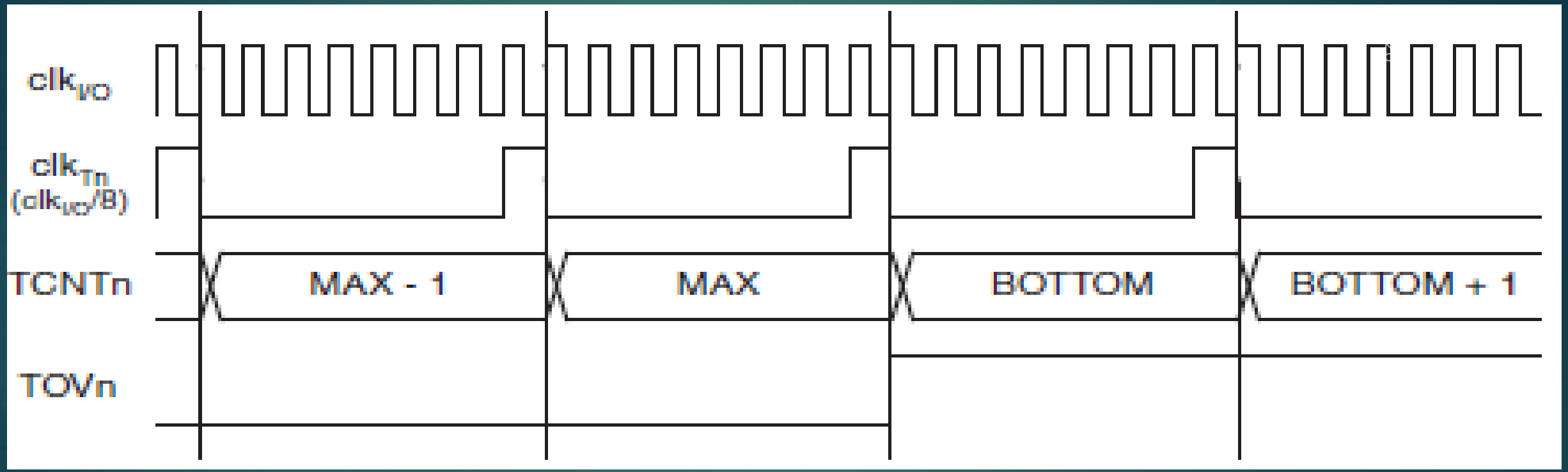


Diagramme de minuterie / compteur, sans pré-diviseur

Timer/Counter Timing Diagram, with Prescaler (fclk_I/O/8)



Chronogramme Minuterie / compteur, avec Pré-diviseur (fclk_I / O / 8)

Les registres utilisés par les Timers

Timer 0	Timer 1	Timer 2	Rôle
TCNT0	TCNT1L	TCNT2	Valeur du Timer (bit 0 à 7)
-	TCNT1H	-	Valeur du Timer (bit 8 à 15)
TCCR0A	TCCR1A	TCCR2A	Registre de contrôle du Timer/Counter
TCCR0B	TCCR1B	TCCR2B	Registre de contrôle du Timer/Counter
-	TCCR1C	-	Registre de contrôle
OCR0A	OCR1AL	OCR2A	Output Compare (bit 0 à 7)
-	OCR1AH	-	Output Compare (bit 8 à 15)
OCR0B	OCR1BL	OCR2B	Output Compare (bit 0 à 7)
-	OCR1BH	-	Output Compare (bit 8 à 15)
-	ICR1L	-	Input Capture (bit 0 à 7)
-	ICR1H	-	Input Capture (bit 8 à 15)
TIMSK0	TIMSK1	TIMSK2	Interrupt Mask
TIFR0	TIFR1	TIFR2	Interrupt Flag

Timers/compteurs

- Les modules timers peuvent être configure pour compter les impulsions sur une broche
- Une interruption peut être générée lors du débordement du Timer
- Le Timer peut générer une interruption après comparaison d'une valeur particulière
- Le Timer peut capturer un évènement sur la broche ICP du µC ATMEGA 328

Exercice Timer

- On dispose d'une horloge de 40MHz en entrée d'un Timer 16 bits (H_ref). Le Prescaler est sur 16 bits. On veut génerer une impulsion périodique de 0,5s.
- Quelle valeur donner au Prescaler, ainsi qu'à l'auto-reload pour parvenir à cela, sachant que l'on veut une fréquence la plus grande possible sur le compteur (cela revient à dire une valeur d'autoreload maximale).

Exercice Timer (correction)

15/11/2022

- La valeur maximale d'autorelaod est de 2^{16} = 65536. La période correspondante en entrée de comptage est donc 0,5s / 65536 = 7,629µs.
- A partie d'une fréquence de 40MHz, le Presceler doit donc donner une fréquence de 1/7,629 µs = 131 072 Hz.
- Le Prescaler vaudra donc P = 40MHz / 131 072 Hz =305,17.
- Il faut arrondir à l'entier supérieur. En effet, dans le sens inverse (arrondi inférieur) la fréquence en entrée de comptage serait trop forte => Autorelaod saturé.
- Prenons donc P=306 => Freq_Cpt = H_ref / P.
- La valeur de l'autoreload est telle que : Auto_Rel*T_Cpt = 0,5s => Auto_Rel/ Freq_Cpt = 0,5s soit Auto_Rel = 0,5* Freq_Cpt = 0,5 * H_ref / P
- **AN: Auto_Rel = 0,5*40M/306 = 65359,477, arrondi à 65359, ce qui donne une période exacte de 499,99635 ms, à la précision du quartz près !**
- Finalement, pour le réglage on placera 306 -1 = 305 dans le prescaler et 65359-1 = 65358 dans l'Autorelaod, eu égard aux comptage synchrone expliqué ci-dessus

TCCR2A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0
(0xB0)	COM2A1	COM2A0	COM2B1	COM2B0	–	–	WGM21	WGM20
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

TCCR2B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0
(0xB1)	FOC2A	FOC2B	–	–	WGM22	CS22	CS21	CS20
Read/Write	W	W	R	R	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

COM2x1:0: Compare Match Output x Mode

WGM2:0: Waveform Generation Mode

FOC2x: Force Output Compare x

CS22:0: Clock Select

Configuration du registre TCCR2

Comapre Output Mode : Non PWM Mode		
COM2B1	COM2B0	Description
0	0	Fonctionnement normal , OC2B déconnecté..
0	1	Bascule OC2B en cas de correspondance Compare Match
1	0	réinitialise OC2B lors de la correspondance Compare Match
1	1	Active OC2B lors de la correspondance Compare Match

Comapre Output Mode : Fast PWM Mode		
COM2B1	COM2B0	Description
0	0	Fonctionnement normal , OC2B déconnecté..
0	1	Réservé
1	0	RAZ OC2B sur la valeur de correspondance, RAU de OC2B sur BAS, (Mode non inversé).
1	1	RAU de OC2B sur la correspondance TCNTn=OCRn, RAZ de OC2B en BAS (TCNTn=0), (mode inversion).

Description	CS22 CS21 CS20	**T(µs)**
Timer2 arrêté	0 0 0 (=0)	
pas de division	0 0 1 (=1)	0,0625
division par 8	0 1 0 (=2)	0,5
division par 32	0 1 1 (=3)	2
division par 64	1 0 0 (=4)	4
division par 128	1 0 1 (=5)	8
division par 256	1 1 0	16
division par 1024	1 1 1	64

Exemple Clignotement sans interruption 1/2

le programme, écrit avec le langage Arduino :

```
/*
 * Clignotement d'une DEL (LED en anglais) à 1 Hz par timer 2.
 */
const byte Led = 13; // Pour utiliser la LED du module
#define LedToggle digitalWrite (Led, !digitalRead(Led))

void setup ()
{
pinMode (Led, OUTPUT);
bitClear (TCCR2A, WGM20); // WGM20 = 0
bitClear (TCCR2A, WGM21); // WGM21 = 0
TCCR2B = 0b00000110; // Clock / 256 soit 16 micro-s et WGM22 = 0
TIFR2 = 0b00000001; // TOV2
TCNT2 = 256 - 250; // Chargement du timer à 6
}
```

Exemple Clignotement sans interruption 2/2

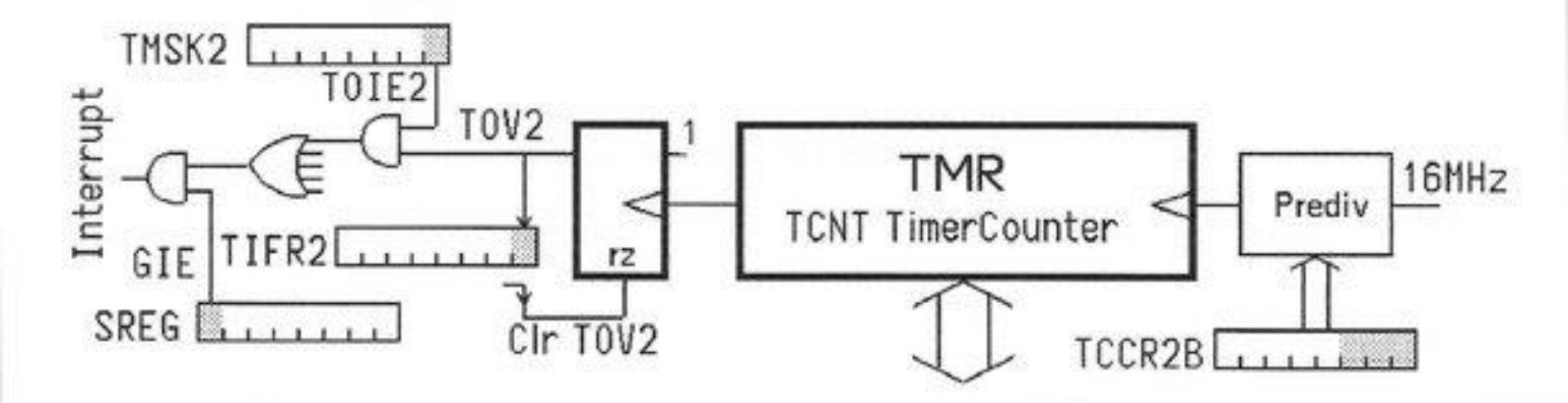

```
byte varCompteur = 0; // La variable globale compteur

void loop () {
if (bitRead (TIFR2, 0) == 1) { // Flag TOV2 mis à 1 ?
TCNT2 = 256 - 250; // Rechargement du timer à 6
bitSet (TIFR2, TOV2); // Remise à zéro du flag TOV2
(voir texte)
if (varCompteur++ > 125) { // Incrémentation et a
atteint 125 ?
varCompteur = 0; // On recommence un nouveau cycle
LedToggle; // Inversion de la LED
}
}
}
```

Architecture de pilotage d'un timer

- le signal d'horloge de 16 MHz arrive au prédiviseur qui divise la fréquence en fonction des bits 0 à 2 du registre **TCCR2B**. Le timer **TCNT** compte ainsi moins rapidement. Lorsqu'il déborde, une bascule est positionnée à 1 : c'est le flag ***TOV2*** que l'on retrouve dans le bit 0 du registre **TIFR2**. Pour autoriser une interruption par le timer, il faut que le flag ***TOIE2*** (Timer/Counter2 Overflow Interrupt Enable) qui est le bit 0 du registre **TIMSK2** (Timer/Counter2 Interrupt Mask Register), soit à 1 ; c'est ce qu'on appelle une autorisation d'interruption locale. Pour que notre timer déclenche une interruption, il faut qu'il y soit autorisé (***TOIE2*** à 1) ET qu'il déborde (***TOV2*** à 1), ces deux conditions arrivant sur une porte ET en amont de la porte OU. Mais comme on l'a dit plus haut, il faut aussi que les interruptions soient autorisées de façon générale : c'est le rôle du flag ***GIE*** , le bit 7 du registre **SREG** (Status Register) qui alimente une porte ET en aval de la porte OU.

Architecture de pilotage d'un timer avec Interruption



Architecture de pilotage d'un timer

Exemple Timer Interrupt


```
// Clignotement d'une LED (mot anglais pour DEL) à 1 Hz par une
// interruption en provenance du timer 2,
// pendant que le programme principal fait
// clignoter une autre LED (période 10s)

const byte Led = 13; // Pour utiliser la LED du module
const byte Led2 = 5; // Led clignotante du programme principal
#define LedToggle digitalWrite (Led, !digitalRead(Led))
#define Led2Toggle digitalWrite (Led2, !digitalRead(Led2))

void setup () {
pinMode (Led, OUTPUT);
pinMode (Led2, OUTPUT);
cli(); // Désactive l'interruption globale
bitClear (TCCR2A, WGM20); // WGM20 = 0
bitClear (TCCR2A, WGM21); // WGM21 = 0
TCCR2B = 0b00000110; // Clock / 256 soit 16 micro-s et WGM22 = 0
TIMSK2 = 0b00000001; // Interruption locale autorisée par TOIE2
sei(); // Active l'interruption globale
}
```

Exemple Timer Interrupt suite


```
byte varCompteur = 0; // La variable compteur

// Routine d'interruption
ISR(TIMER2_OVF_vect) {
TCNT2 = 256 - 250; // 250 x 16 µS = 4 ms
if (varCompteur++ > 125) { // 125 * 4 ms = 500 ms (demi-
période)
varCompteur = 0;
LedToggle;
}
}

void loop () {
// Mettre ici le programme. Exemple :
Led2Toggle;
delay (5000); // Demi-période de la deuxième LED
}
```

Exemple Timer trois tâches


```
// trois tâches de clignotement avec les Timers

const byte Led = 13; // Led pour la tâche 1
const byte Led2 = 5; // Led pour la tâche 2
const byte Led3 = 6; // Led pour la tâche 3
void setup () {
pinMode (Led, OUTPUT);
pinMode (Led2, OUTPUT);
pinMode (Led3, OUTPUT);
cli(); // Désactive l'interruption globale
bitClear (TCCR2A, WGM20); // WGM20 = 0
bitClear (TCCR2A, WGM21); // WGM21 = 0
TCCR2B = 0b00000110; // Clock / 256 soit 16 micro-s et WGM22 = 0
TIMSK2 = 0b00000001; // Interruption locale autorisée par TOIE2
sei(); // Active l'interruption globale
}
```

Trois tâches avec Timer


```
byte varCompteur = 0; // La variable compteur
byte cpt_t2=0, cpt_t3 = 0;//Le compteur pour les tâches

// Routine d'interruption
ISR(TIMER2_OVF_vect) {
TCNT2 = 256 - 250; // 250 x 16 µS = 4 ms
if (varCompteur++ > 125) { // 125 * 4 ms = 500 ms (demi-période)
varCompteur = 0;
Tache_1();
if (cpt_t2++ == 3) {Tache_2();cpt_t2=0; }
if (cpt_t3++ == 5) {Tache_3();cpt_t3=0; }
}
}
void Tache_1()  {digitalWrite (Led, !digitalRead(Led));}
void Tache_2()  {digitalWrite (Led2, !digitalRead(Led2));}
void Tache_3()  {digitalWrite (Led3, !digitalRead(Led3));}

void loop () {
// Mettre ici le programme. Exemple :
//delay (5000); // Demi-période de la deuxième LED
}
```

Utilisation de millis()

- La fonction millis() renvoie le nombre de milliseconde écoulées depuis le démarrage de la carte Arduino. Elle permet de mesurer une durée écoulée
- Exempe
-

```
//programme tempo pour utiliser millis()
 unsigned long t1,t2;
boolean tempo(unsigned int duree){
t2=millis();
if ((t2-t1)>=duree) {t1=t2;
return true;}
return false;}
void setup() {  Serial.begin(9600);
t1=millis();//initialiser la tempo }
void loop() {  int x;
 if (tempo(1000)) x++; //incrémente x toutes les 1000 ms
Serial.println("nombre de secondes =" + String(x)); }
```

Utilisation de millis()

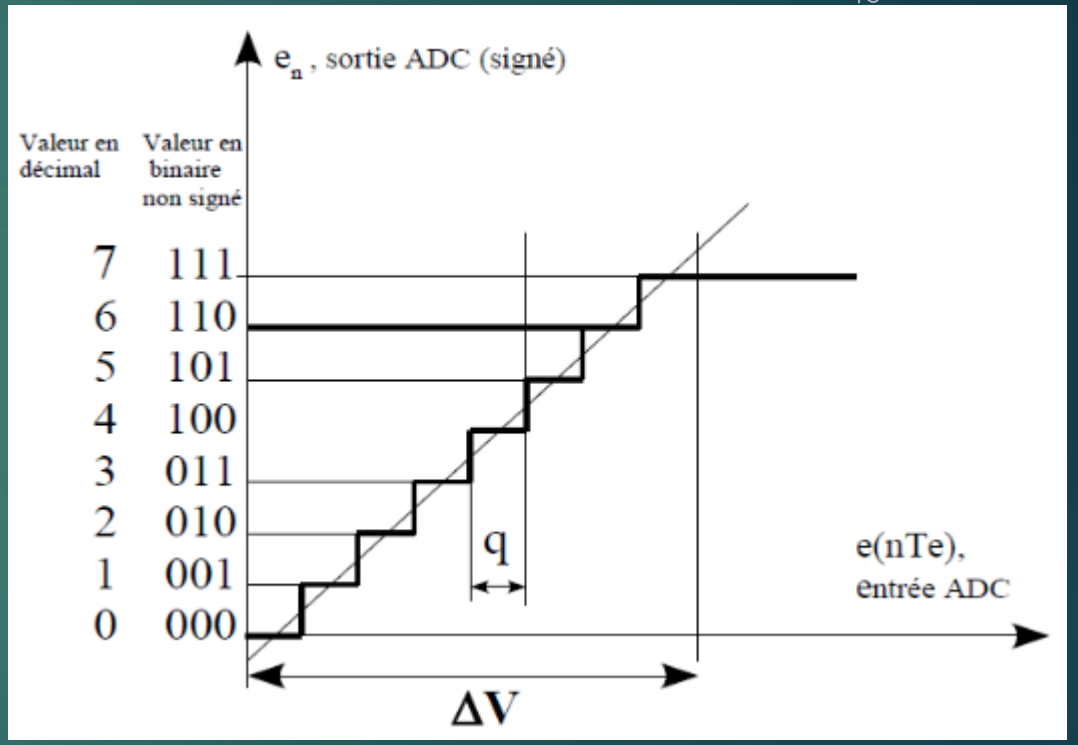
- Il est pratique de créer une classe permattant d'instancier facilement des variables temporisation en utilisant millis()
- Exempe

```
class temps //pour mesurer les durées en ms
{   unsigned long tc;
public:
unsigned long duree()
{  return (millis()-tc);   }
void reset()
{   tc = millis();   }
temps()
{  reset(); }
};
```

```
//programme tempo pour utiliser la classe
temps()
 temps t1,t2;
 #define pin_led 13
int vled=0;
void setup() {  Serial.begin(9600);
pinMode(pin_led,OUTPUT);
t1.reset();t2.reset();//initialiser la tempo }
void loop() {  int x;
 if (t1.duree()==1000)) {x++;t1.reset();}
//incrémente x toutes les 1000 ms
Serial.println("nombre de secondes =" +
String(x));
if (t2.duree()==500) {vled=~vled;t2.reset();}
digitalWrite(pin_led,vled);
}
```

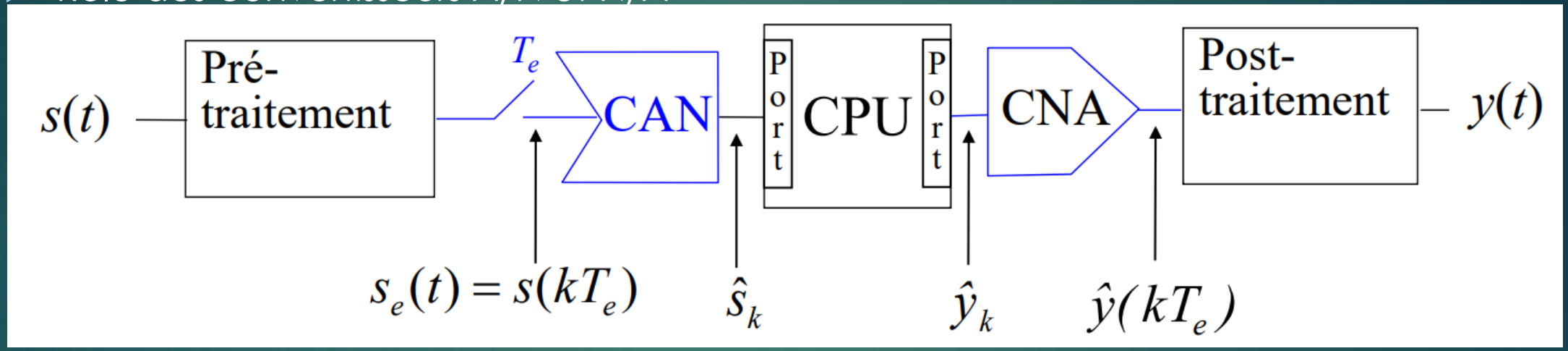
Conversion Analogique Numérique CAN

- Un CAN est un circuit électronique qui a pour rôle de fournir un nombre binaire à l'image d'une tension analogique. Une tension possède par définition une infinité de valeurs possibles, alors que le nombre binaire fourni, lui, est forcément limité en quantité de valeurs.
- La caractéristique typique d'un ADC (3 bits) est donnée ci-contre :



Conversion Analogique Numérique CAN et CNA

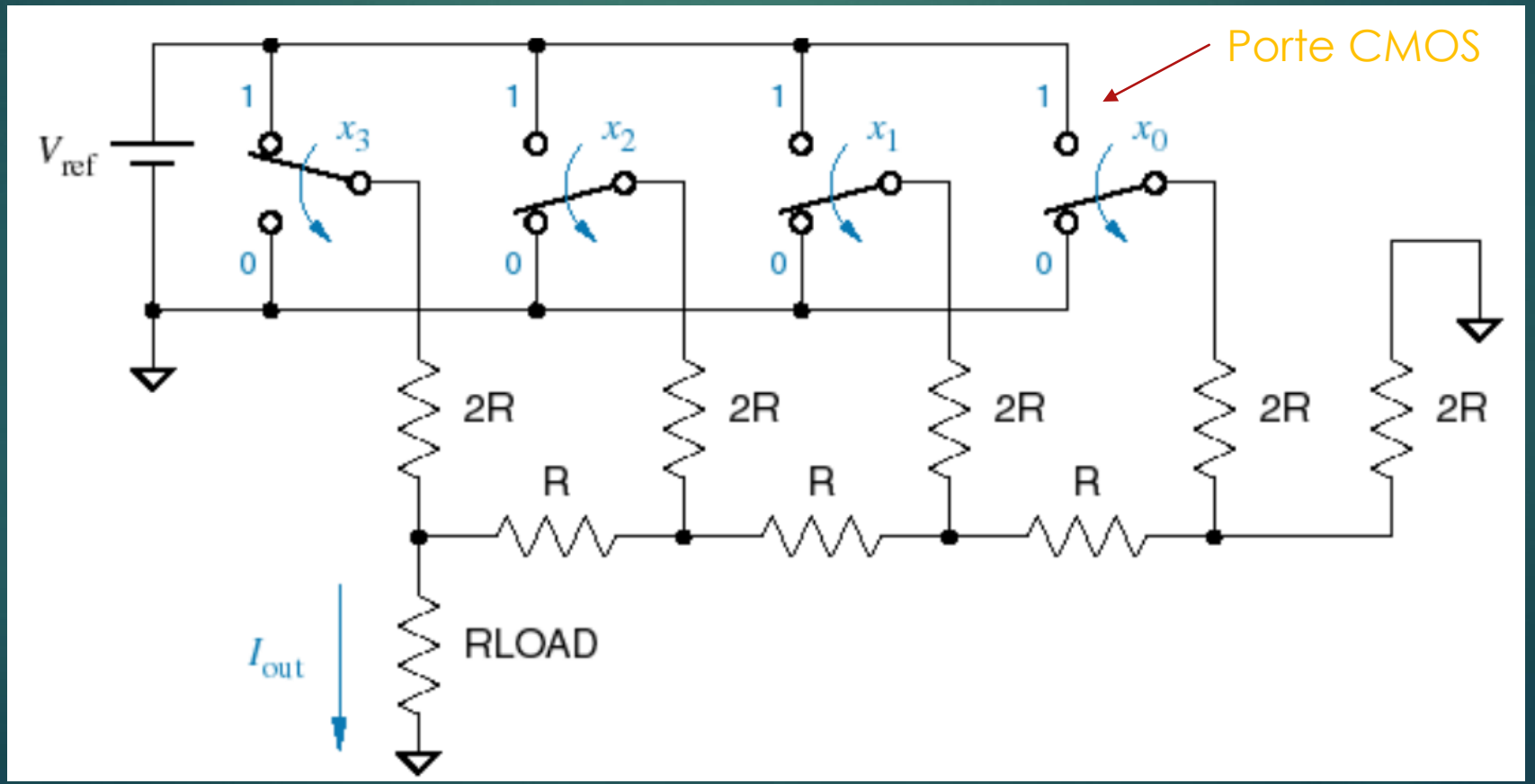
- Rôle des convertisseurs A/N et N/A



- Assurent la conversion de format entre des données d'E/S analogiques et le CPU du microcontrôleur
- Les blocs de prétraitement et de post-traitement gèrent les problèmes d'interface entre les e/s et les convertisseurs

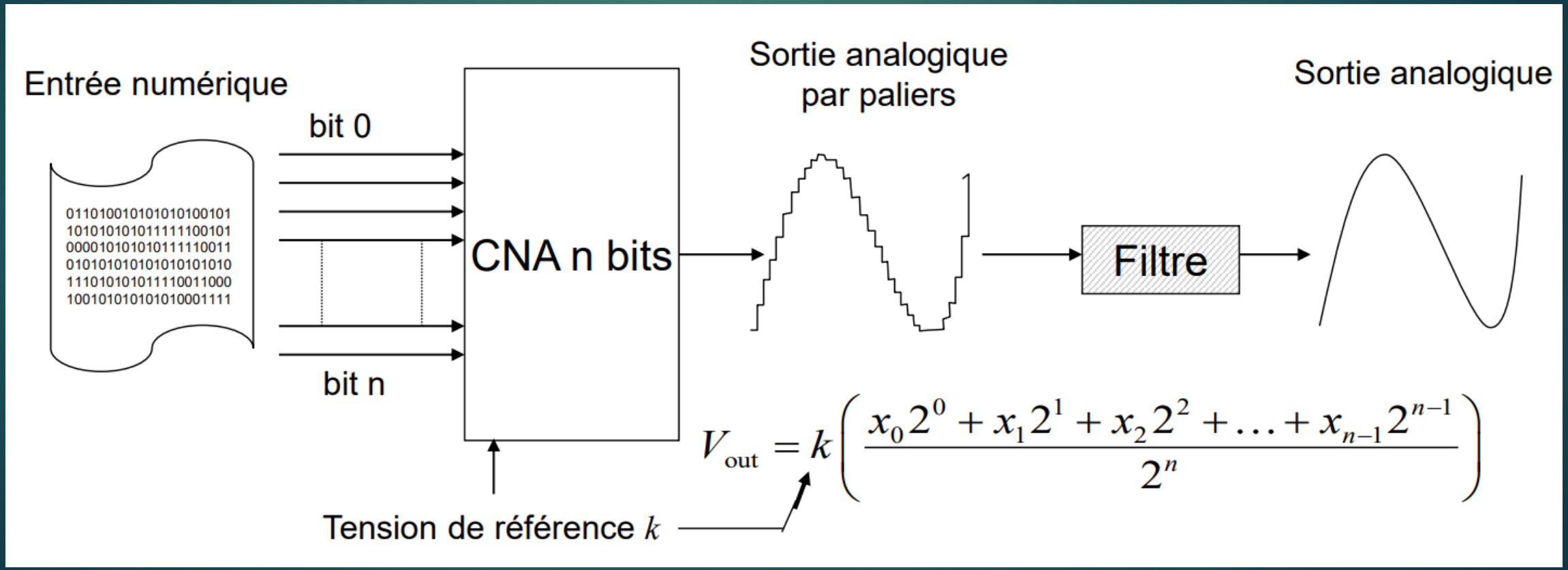
CNA

- Convertit un code binaire en une tension ou courant continu d'amplitude équivalente
  - En pratique un filtre est requis après la sortie
  - CAN à réseau R-2R



CNA

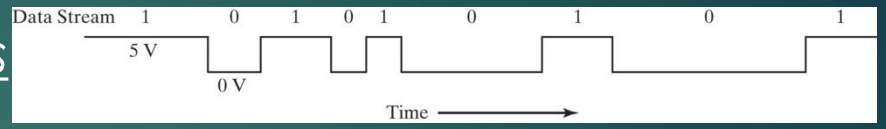
- Convertit un code binaire en une tension ou courant continu d'amplitude équivalente
  - En pratique un filtre est requis après la sortie



Communication Série

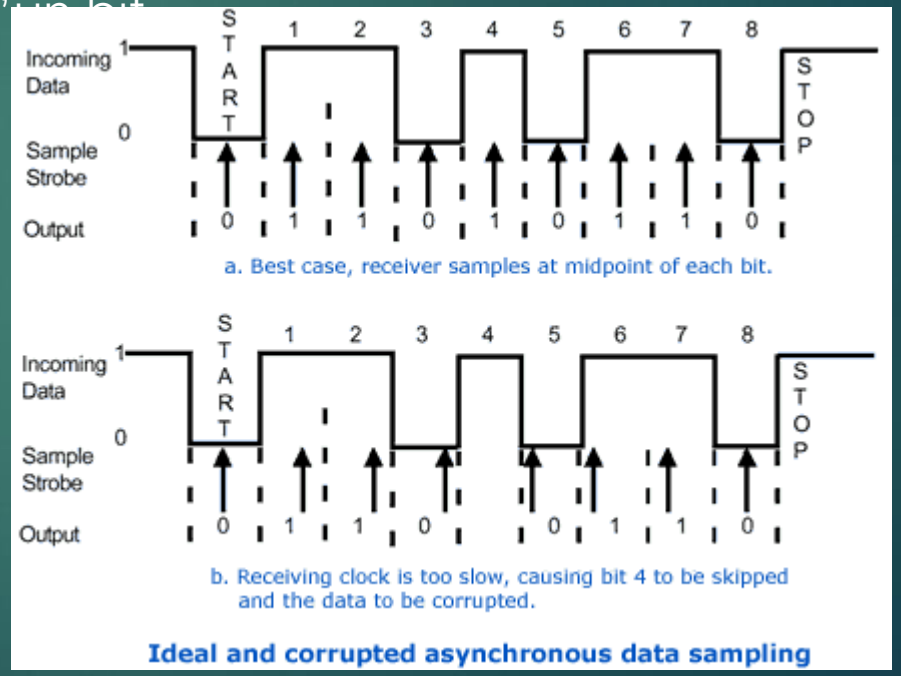
Concepts de communication série

- Serie → un bit à la fois
- En utilisant *les niveaux logiques*
- Nécessité de partage des informations temporelles entre l'emetteur et le récepteur
  - Transitions entre bits, durée d'un bit



Introduction to Mechatronics, Figure 7.2 p. 107.

- Deux types de protocoles série
  - Synchrone (shared clock)
    - Ex. SPI, I²C
  - Asynchrone (no shared clock)
    - Sender and receiver maintain independent clocks
      - Ex. RS-232, USB



http://www.quatech.com/support/figures/async1.gif

Communication série synchrone

- Caractéristiques de la série synchrone
  - Au moins une connexion pour les données série
  - Une connexion d'horloge
  - Signal de masse partagé
- Protocoles usuels
  - **S**erial **P**eripheral **I**nterface (**SPI**, développé par Freescale)
  - **I**nter-**I**ntegrated **C**ircuit bus (**I²C**, développé par Philips (NXT))
- Arduino supporte
  - I²C and SPI

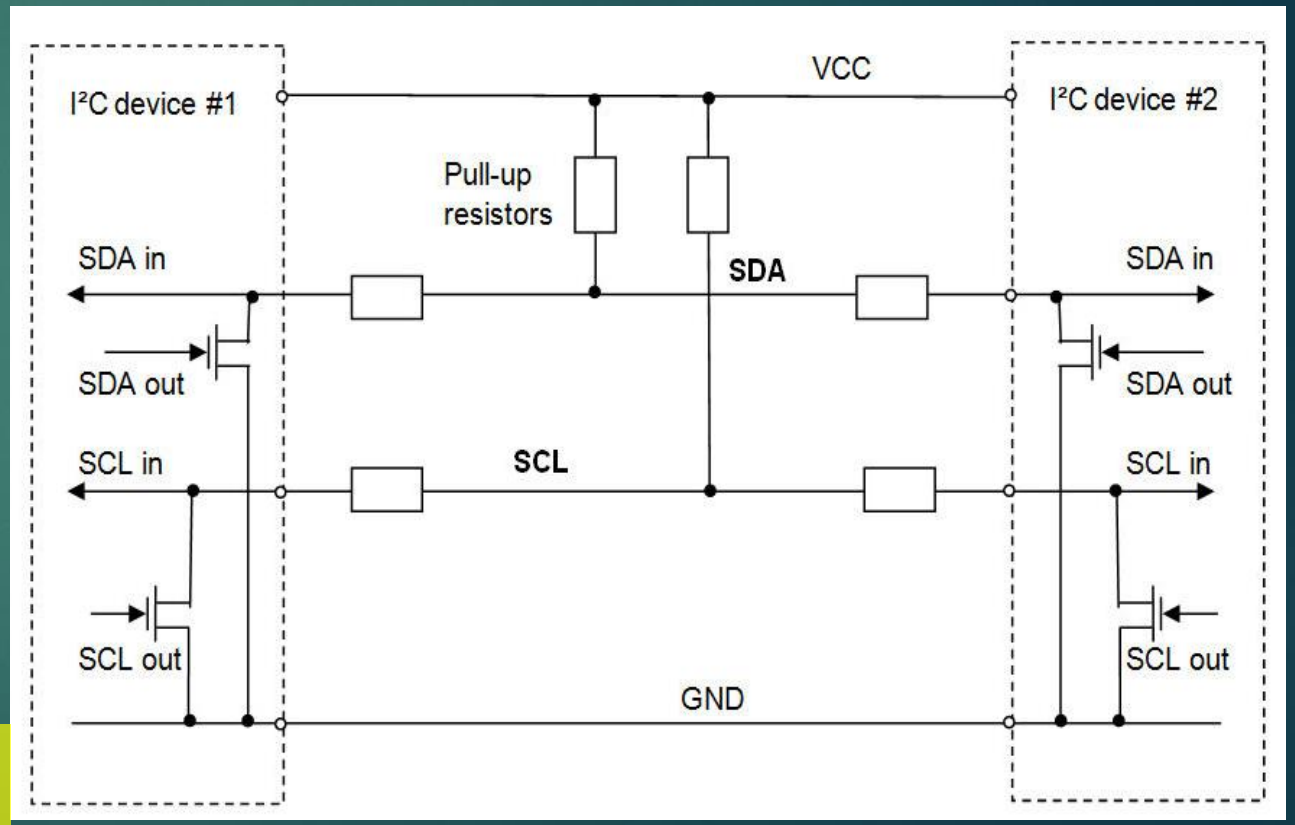
Communication série synchrone

- Les normes I2C (Inter-Integrated Circuit) et SPI (Serial Peripheral Interface) ont été créées pour fournir un moyen simple de transférer des informations numériques entre des capteurs et des microcontrôleurs.
- Les bibliothèques Arduino pour I2C et SPI facilitent l'utilisation de ces deux protocoles. Le choix entre I2C et SPI est en général déterminé par les périphériques que l'on souhaite connecter. Certains appareils offrent les deux standards, mais habituellement un périphérique ou une puce ne supporte qu'une seule des deux normes.

I2C

- La communication I2C utilise 2 fils :
  - Horloge
  - Données
- Elle fonctionne en half-duplex : émission et réception se font en alternance
- Les vitesses sont :
  - Standard : 100 Kbits/s
  - Fast : 400 Kbits/s
  - Fast plus : 1 MB/s
  - High-speed : 3,4 Mb/s
  - Ultra High-speed : 5 Mb/s
- Principe de communication :
  - Maître / Esclave
  - Le maître :
    - Ouvre la communication
    - Désigne l'esclave par son adresse
    - Choisit le sens de communication (émission ou
    - réception) Envoie ou reçoit les données
    - Ferme la communication

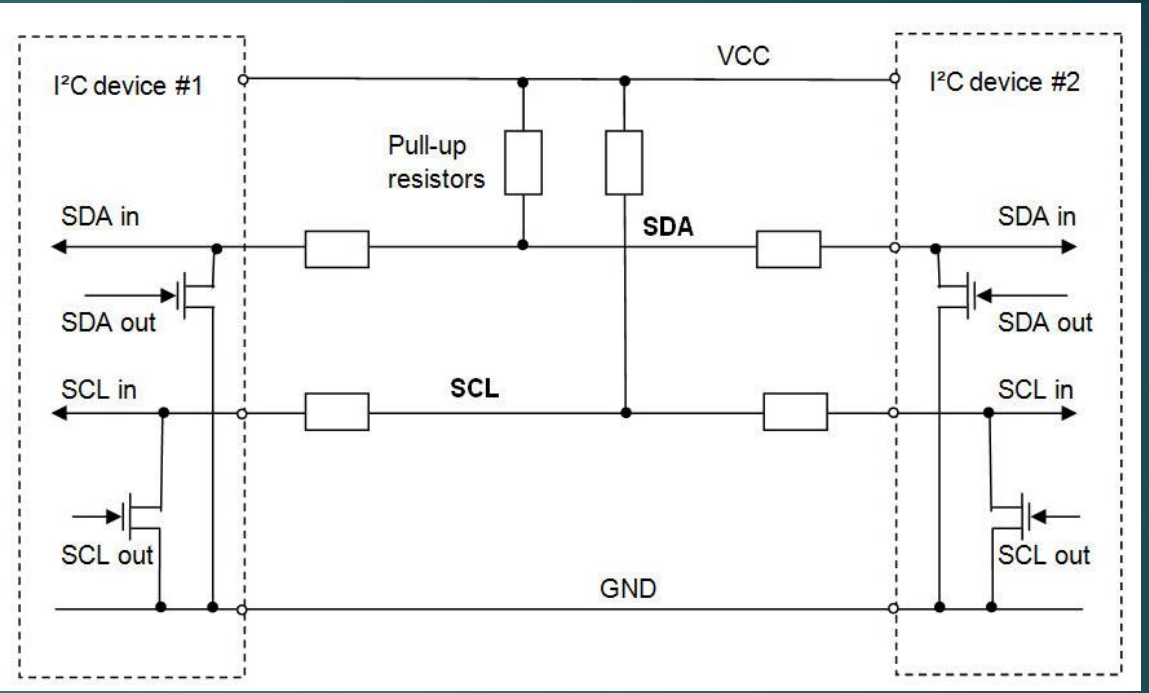
Utiliser des résistances de 1,2k - 2,2k pour Vcc = 3,3 V ou des résistances de 1,8 - 3,3k pour Vcc = 5 V



http://www.byteparadigm.com/kb/article/AA-00255/22/Introduction-to-SPI-and-IC-protocols.html

I²C

- Developed in 1982 by Philips (now NXP)
  - To connect CPU to peripherals in televisions
- Two signal lines, both are '*open drain*', thus pull-up resistors are needed
  - SDA (serial data)
  - SCL (serial clock)
- Devices are either masters or slaves
  - Master is the device that always drives SCL and initiates a transfer
- Each device has an address



http://www.byteparadigm.com/kb/article/AA-00255/22/Introduction-to-SPI-and-IC-protocols.html

Use 1.2k - 2.2k pull up resistors for Vcc = 3.3 V or 1.8 - 3.3k resistors for Vcc = 5 V

I2C

- Chaque capteur/actionneur (esclave) a une adresse (sur 7 ou 10 bits)
  - Cette adresse est fixée par le constructeur
  - Il est parfois possible de choisir une adresse parmi un ensemble prédéfini



- La vitesse d'horloge est définie par le maître (microcontrôleur)
- Le périphérique doit s'y adapter mais peut imposer un ralentissement en forçant la ligne d'horloge à 0 plus longtemps
- Les bits d'acquittement (adresse et données) sont mis à 1 par l'émetteur mais forcés à 0 par le récepteur :
  - Correspondant à cette adresse pour le premier
  - Ayant correctement reçu la donnée pour le second
- L'émetteur teste l'état des ces bits pour savoir si la communication a fonctionné

I2C : Protocole

La connexion est toujours ouverte et fermée par le maître et l'esclave se contente de répondre.

1. Ouverture de connexion :
   - Envoyer un bit de START
   - Attendre qu'il soit envoyé
   - Vérifier que l'envoi du START s'est bien passé (correspond au cas où la ligne ne serait pas disponible : multi maîtres)

2. Envoi de l'adresse :
   - Envoyer l'adresse (7 ou 10 bits) complété à droite par 0 (émission) ou 1 (réception)
   - Attendre qu'elle soit envoyée
   - Regarder si l'esclave a répondu par ACK (l'absence de réponse correspond à un NACK)

I2C : protocole

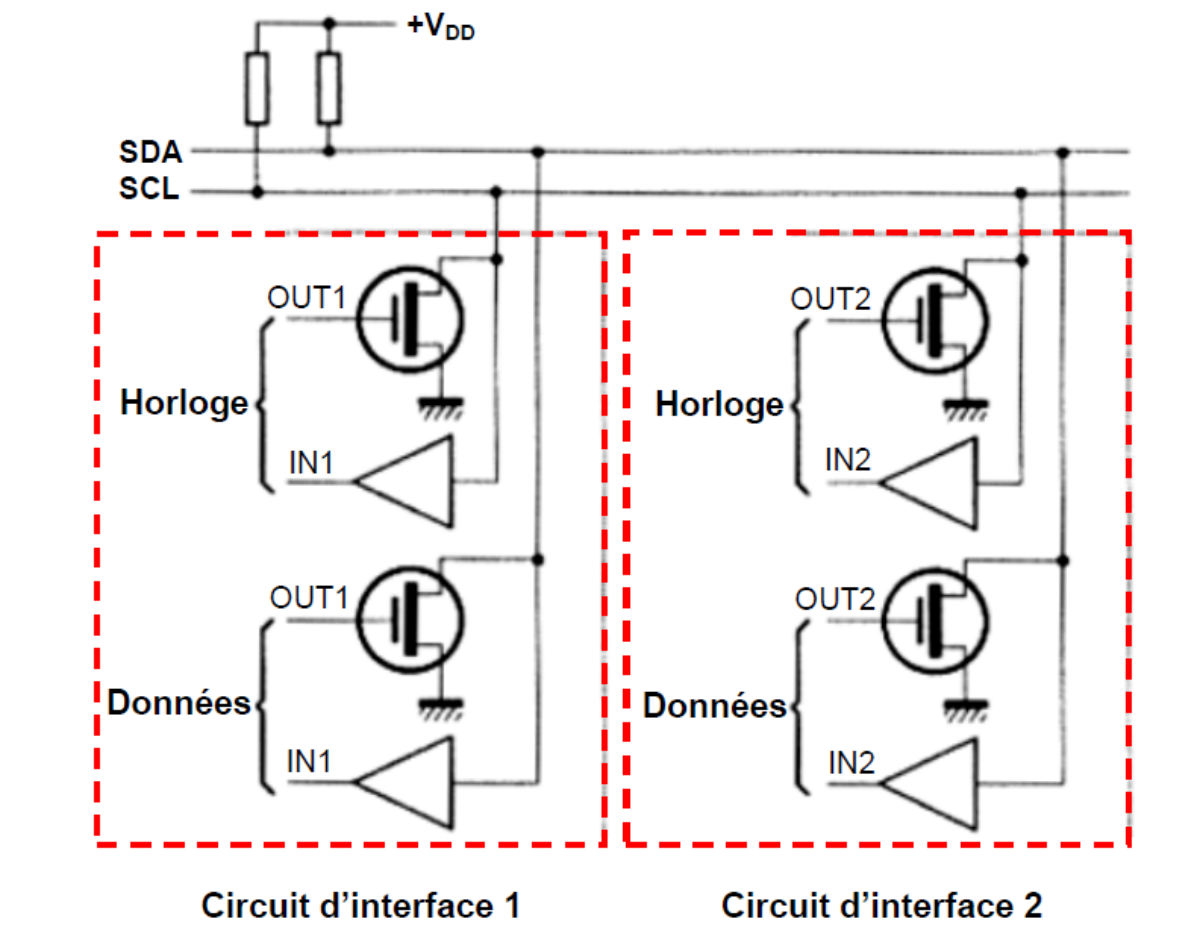
3. Envoi de message :
   - Envoyer un octet
   - Attendre qu'il soit envoyé
   - Regarder si l'esclave a répondu par ACK
   - Recommencer pour chaque octet

OU

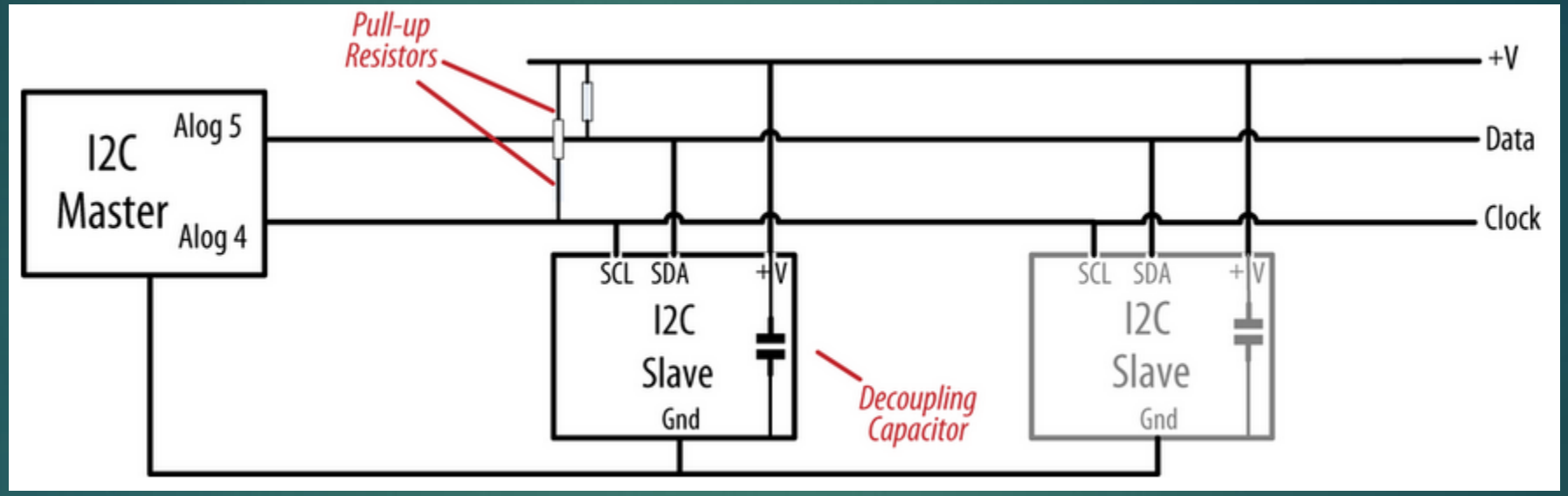
Réception de message :
   - Préparer le périphérique pour répondre
     - ACK si ce n'est pas le dernier octet à recevoir
     - NACK si c'est le dernier octet à recevoir
   - Attendre d'avoir reçu l'octet
   - Récupérer l'octet
   - Recommencer pour chaque octet

3. Fin de connexion :
   - Envoyer un bit de STOP

I2C Circuit d'interface

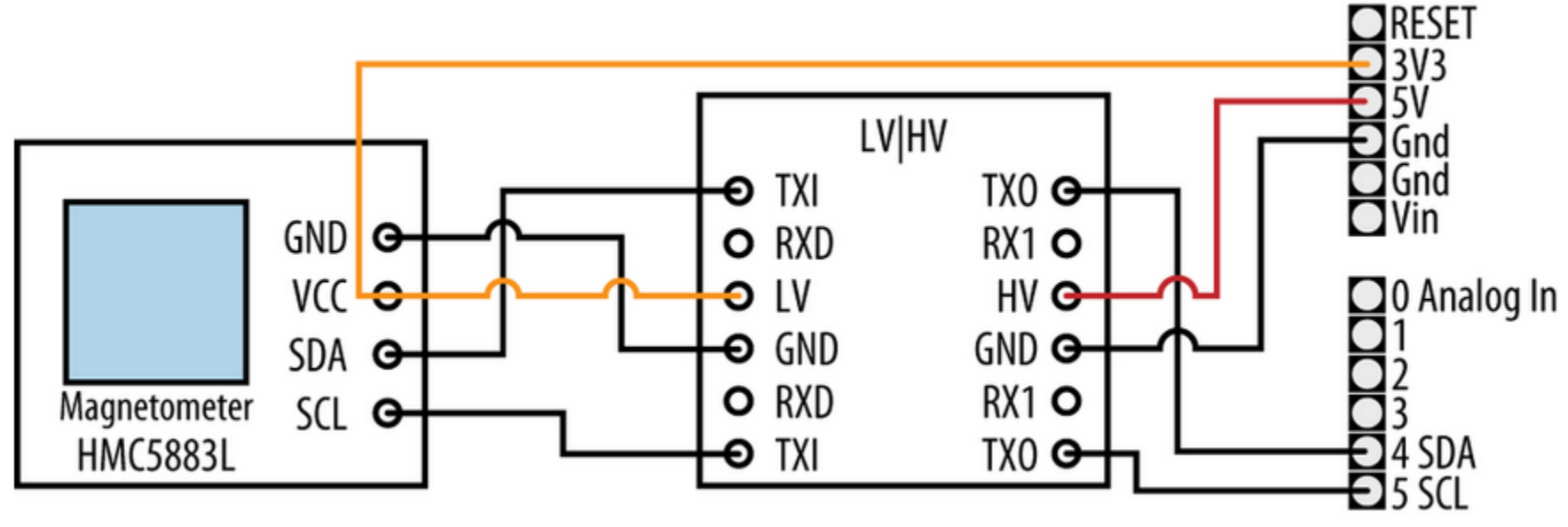


Les broches Analog 4 (SCL) et 5 (SDA) doivent être rappelés à +V et une masse commune est nécessaire



Margolis, M. (2012). Arduino Cookbook, O'Reilly Media, Inc., Sebastopol, CA, p. 422.

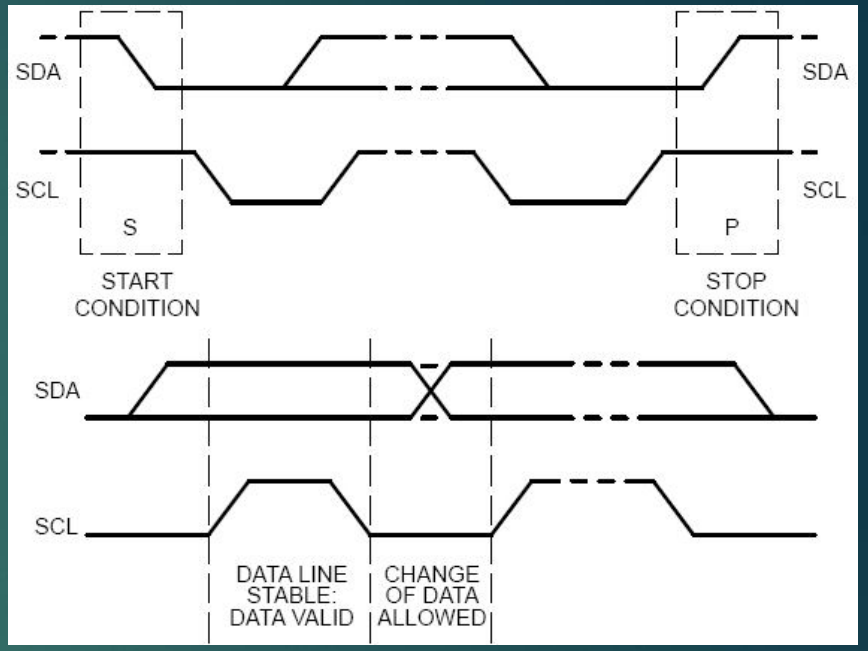
Parfois il faut adapter les niveaux de tesion pour certains capteurs
faire passer le niveau de tension de 5 V (Arduino) à 3,3 V pour certains capteurs



Margolis, M. (2012). Arduino Cookbook, O'Reilly Media, Inc., Sebastopol, CA, p. 423.

Protocole I²C

- Le maître initie le transfert avec un bit START (SDA de haut en bas pendant que SCL est haut).
- Adresse de l'esclave (7 ou 10 bits, 7 étant la valeur la plus courante)
- Type de transfert (1 bit : 0 pour écrire, 1 pour lire)
- Tous les circuits intégrés comparent l'adresse sur le bus à leur propre adresse
  - Si l'adresse correspond, le CI envoie un signal d'ACKNOWLEDGE
  - Si l'adresse ne correspond pas, le CI attend que le bus soit libéré par la conditionSTOP
- Une fois que le maître a reçu le signal ACKNOWLEDGE, il envoie (écrit) ou reçoit (lit) des données.
  - Le récepteur renvoie le signal ACKNOWLEDGE pour chaque octet reçu.
- Le maître conclut le transfert avec le bit STOP



Structure général d'un transfert de deux octets

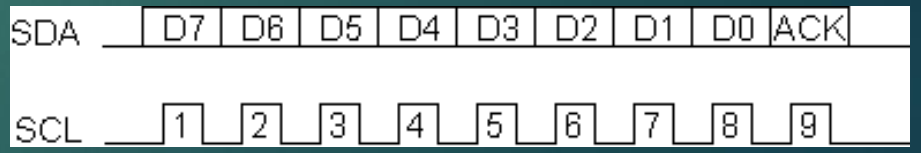
START	Slave address	Rd/nWr	ACK	Data	ACK	Data	ACK	STOP
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

Ecrire 2 octets (les bits grisés sont placés sur le bus par le maître)

START	Slave address	0	0	Data	0	Data	0	STOP
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

Lire 2 octets (les bits grisés sont placés sur le bus par le maître)

START	Slave address	1	0	Data	0	Data	1	STOP
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

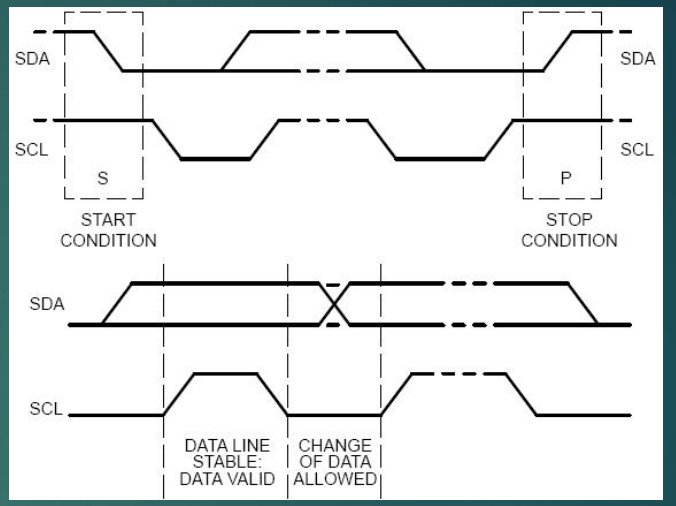


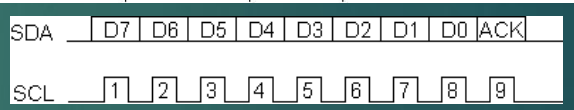
http://www.byteparadigm.com/kb/article/AA-00255/22/Introduction-to-SPI-and-IC-protocols.html

http://www.robot-electronics.co.uk/acatalog/I2C_Tutorial.html

I²C protocol

- Master initiates transfer with a START bit (SDA from high to low while SCL is high
- Slave address (7 or 10 bits, 7 is most common)
- Transfer type (1 bit: 0 to write, 1 to read)
  - All ICs compare address to their address
    - If address matches, device sends an ACKNOWLEDGE signal
    - If address does not match, device waits until bus is released by STOP condition
- Once master receives ACKNOWLEDGE, it then sends (writes) or receives (reads) data
  - Receiver sends back ACKNOWLEDGE for each byte received
- Master concludes the transfer with STOP bit





General structure of a 2-byte transfer

START	Slave address	Rd/nWr	ACK	Data	ACK	Data	ACK	STOP
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

http://www.robot-electronics.co.uk/acatalog/I2C_Tutorial.html

Writing 2-bytes (shaded bits are put on the bus by the master)

START	Slave address	0	0	Data	0	Data	0	STOP
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

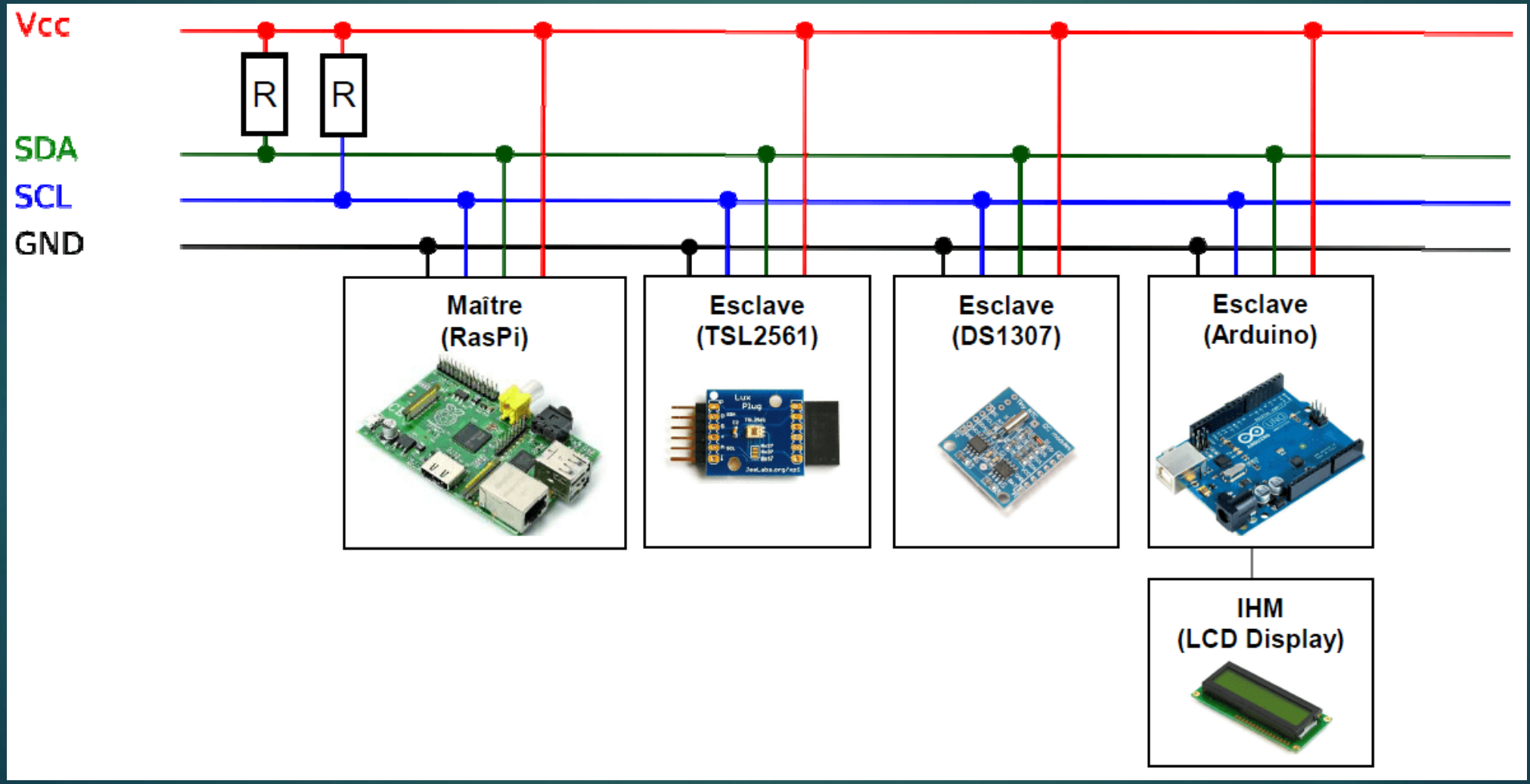
Reading 2-bytes (shaded bits are put on the bus by the master)

START	Slave address	1	0	Data	0	Data	1	STOP
1 bit	7 bits	1 bit	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit

http://www.byteparadigm.com/kb/article/AA-00255/22/Introduction-to-SPI-and-IC-protocols.html

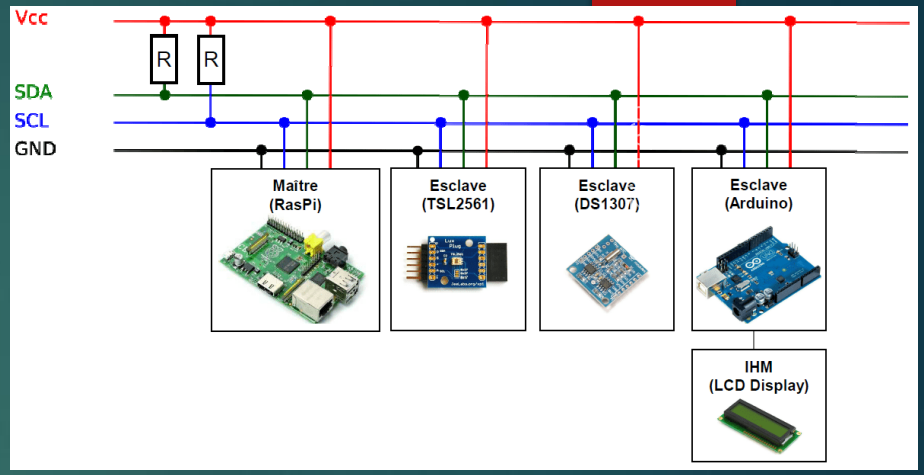
BUS I2C

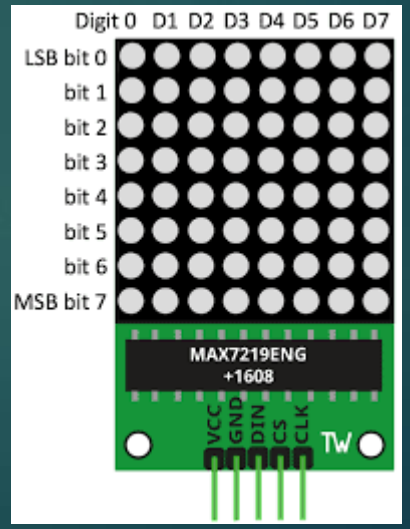
- Les deux connexions du bus I2C se nomment SCL (Serial Clock Line) et SDA (Serial Data line).
- Elles sont disponibles sur une carte standard Arduino en employant la broche analogique 5 pour SCL qui fournit un signal d'horloge, et la broche analogique 4 pour SDA, qui s'occupe du transfert des données (sur la Mega, il faut utiliser la broche 20 pour SDA et la broche 21 pour SCL).
- La carte Uno rev 3 a des broches supplémentaires qui dupliquent les broches 4 et 5.
- Pour le Raspberry Pi type B, il s'agit respectivement des broches GPIO2 et GPIO3 pour SDA et SCL repérées sur le « Pi Cobbler ».
- Un périphérique sur le bus I2C est considéré comme le périphérique maître. Son travail consiste à coordonner le transfert des informations entre les autres périphériques (esclaves) qui sont connectés. Il ne doit y avoir qu'un seul maître qui contrôle les autres composants auxquels il est connecté. La figure ci-dessous montre un maître I2C avec plusieurs esclaves I2C.


Vcc
SDA
SCL
GND
R
R
Maître
(RasPi)
Esclave
(TSL2561)
Esclave
(DS1307)
Esclave
(Arduino)
IHM
(LCD Display)

Projet

Commande d'un afficheur matriciel composé de 4 modules de 64 Leds 8*8.
La carte Afficheur est composée d'une carte Arduino et d'un bloc de 4 afficheur matriciels
La carte de commande est une autre carte Arduino utilisant le bus i2C fonctionnant en maître pour communiquer avec la carte afficheur utilisant le bus i2C fonctionnant en esclave.
L'afficheur pourra afficher des nombres de 4 chiffres entre 0000 et 9999
Les commandes possibles sont : afficher, effacer, clignoter, défiler vers la gauche un message.
TAF :
Proposer une application qui implémente un esclave I2C permettant de fonctionner conformément au protocole.





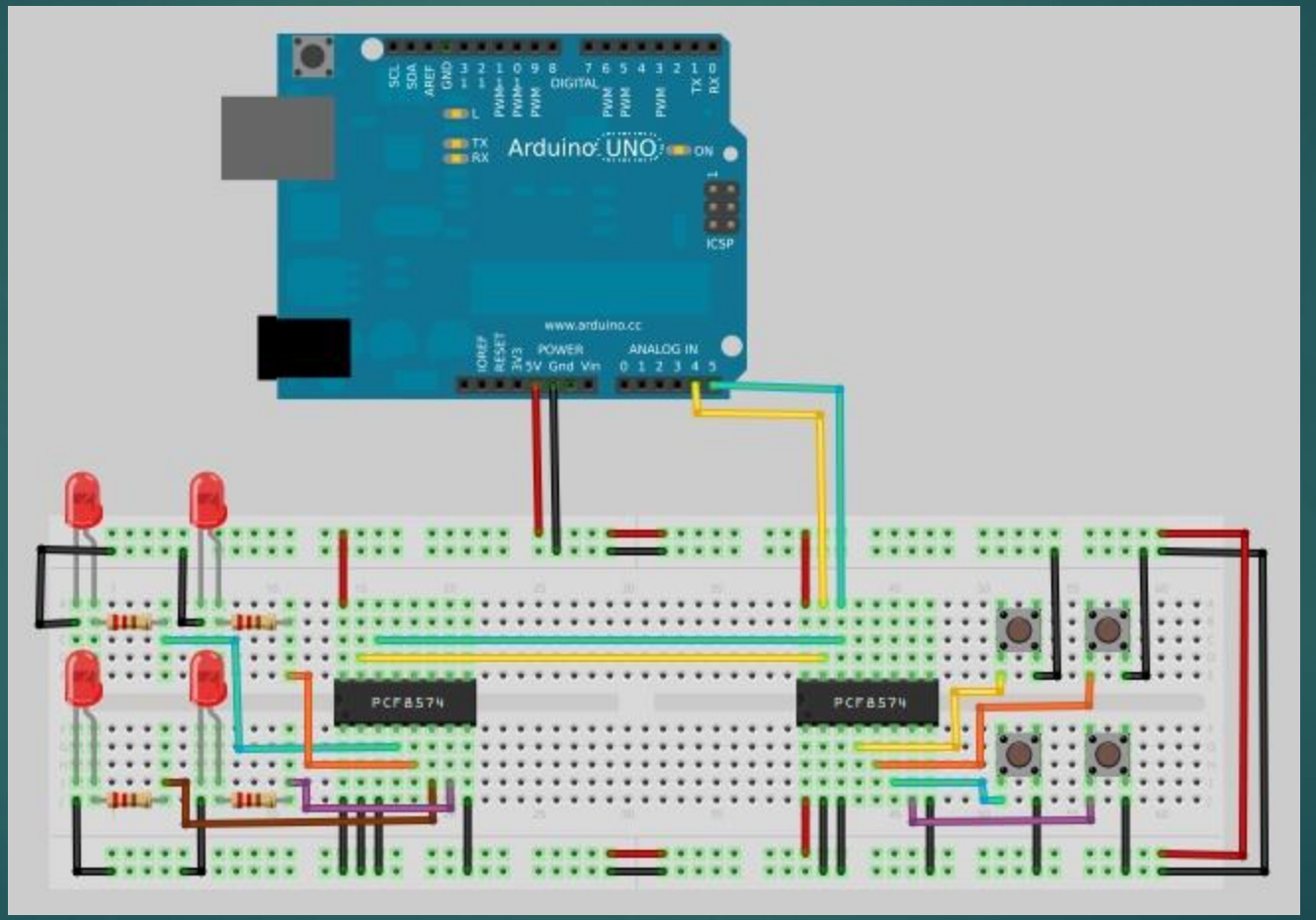
05/11/2026

Many digital components use I2C

- See the list from:

http://forums.parallax.com/showthread.php?99160-List-of-I2C-devices

Les ports d'Arduino peuvent être étendus en utilisant un module I2C port expander (pcf8574)



http://garagelab.com/profiles/blogs/tutorial-arduino-i-o-port-expander-with-pcf8574

The Arduino can be 'expanded' using an I2C port expander device (pcf8574)


```
#include<Wire.h>

byte x=0;
byte y=0;

void setup()
{
    Wire.begin();
}

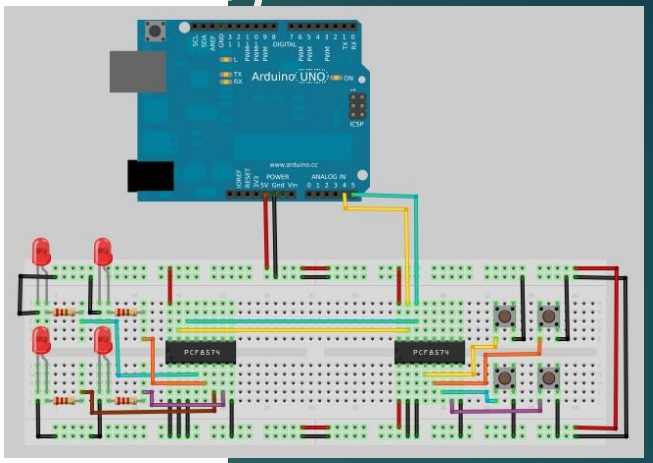
void loop()
{
    Wire.requestFrom(33,1);
    if(Wire.available())      //If the request is available
    {
        x=Wire.receive();       //Receive the data
    }
    if(x<255)                   //If the data is less than 255
    {
        if (x==254) { y = 0; }; //P0
        if (x==253) { y = 2; }; //P1
        if (x==247) { y = 8; }; //P3
        if (x==251) { y = 4; }; //P2
    }
    Wire.beginTransmission(32);     //Begin the transmission to PCF8574
    Wire.send(y);                   //Send the data to PCF8574
    Wire.endTransmission();         //End the Transmission
}
```

ADDRESS

NO. OF BYTES REQUESTED

USE Wire.read();

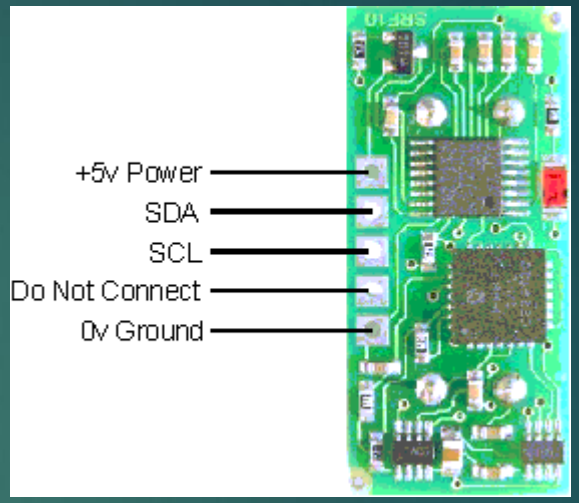
USE Wire.write();



http://garagelab.com/profiles/blogs/tutorial-arduino-i-o-port-expander-with-pcf8574

I²C device – SRF10 Ultrasonic Range Finder

- Devantech SRF10
  - 3 cm – 6 m range
  - 20 Hz max rep rate
  - Range results in in., cm., or μs (time of flight)
  - Always a slave (default address is 0xE0)
  - Four registers:



http://www.robot-electronics.co.uk/htm/srf10tech.htm

Location	Read	Write
0	Software Revision	Command Register
1	Unused (reads 0x80)	Max Gain Register (default 16)
2	Range High Byte	Range Register (default 255)
3	Range Low Byte	N/A

- Three commands:

Command		Action
Decimal	Hex	
80	0x50	Ranging Mode - Result in inches
81	0x51	Ranging Mode - Result in centimeters
82	0x52	Ranging Mode - Result in micro-seconds

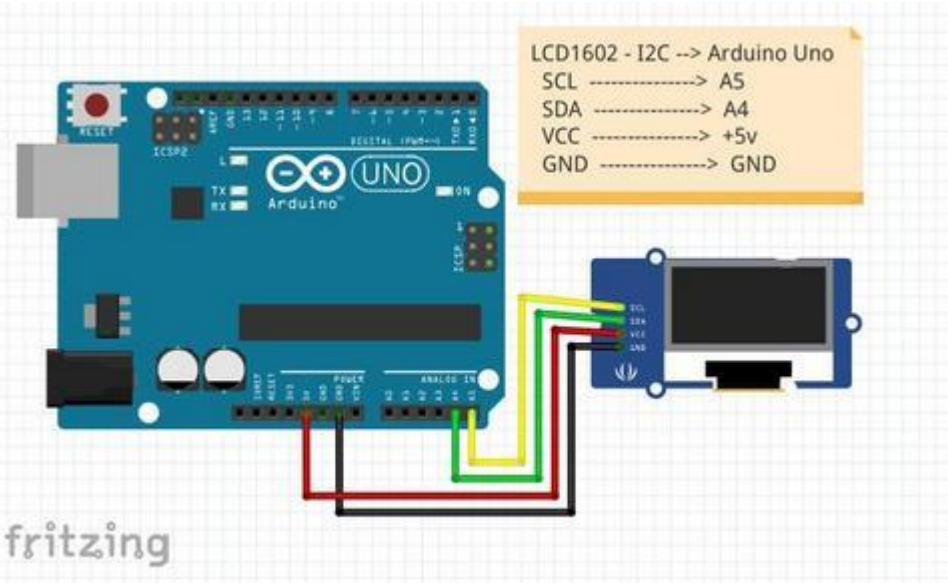
LCD I2C avec Arduino

- Connexion Du LCD 1602 - I2C A L'Arduino Uno

Relier l'afficheur **LCD 1602 - I2C** à l'A

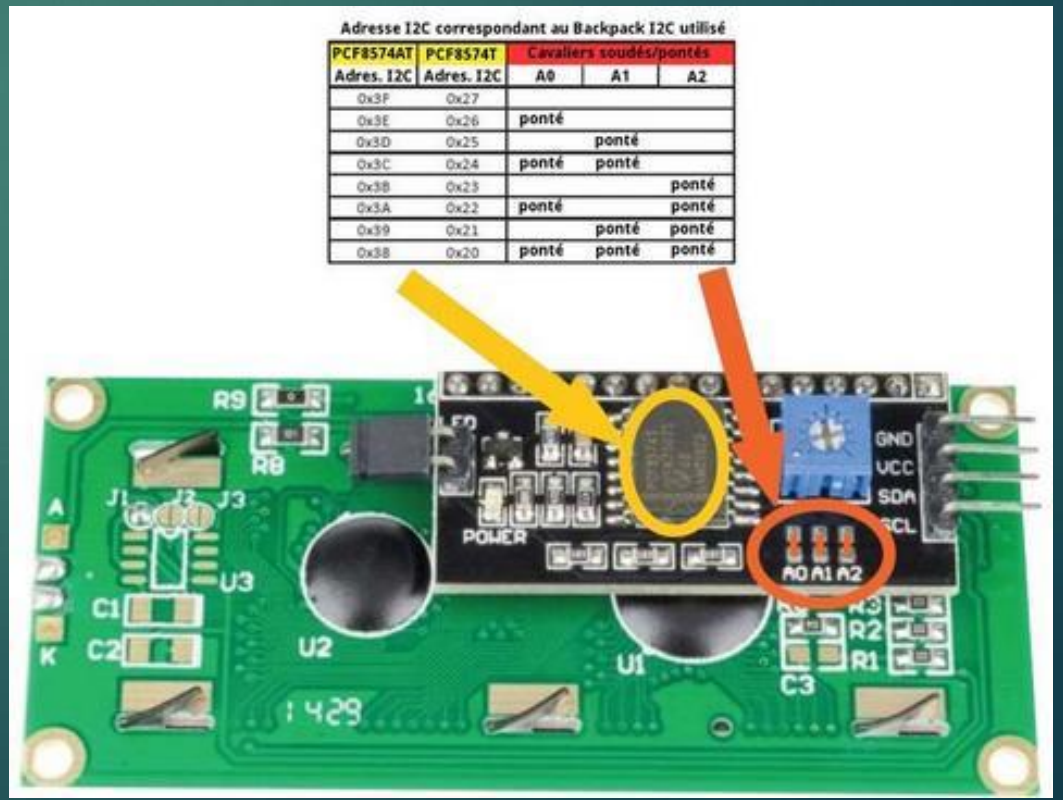
Lcd 1602 I2C ---> Arduino Uno

- **Vcc à la broche +5v # +5v**
- **Gnd à la broche Gnd # Masse**
- **SDA à la broche A4 # Bus Donné**
- **SCL à la broche A5 # Bus Horloge**



Adresse de l'interface I2C

L'interface I2C est un composant électronique du type **PCF8574AT** ou **PCF8574T** qui possède son adresse.



Adresse de l'interface I2C

- Vous pouvez modifier l'adresse suivant vos besoins, en soudant un **cavalier** aux emplacements **A0**, **A1** et **A2** suivant le tableau ci-contre

PCF8574AT	PCF8574T	Cavaliers	Cavaliers	Cavaliers
Adresse I2C	Adresse I2C	A0	A1	A2
0x3f	0x27			
0x3e	0x26	cavalier		
0x3d	0x25		cavalier	
0x3c	0x24	cavalier	cavalier	
0x3b	0x23			cavalier
0x3a	0x22	cavalier		cavalier
0x39	0x21		cavalier	cavalier
0x38	0x20	cavalier	cavalier	cavalier

LCD I2C avec Arduino


```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// LCD address = 0x27
// 16 = Nbre Colonnes
// 2 = Nbre Lignes
LiquidCrystal_I2C lcd(0x27,16,2);  // Remplacer le 20 , 4 par 16 , 2

void setup()
{
  lcd.init();  // initialize the lcd

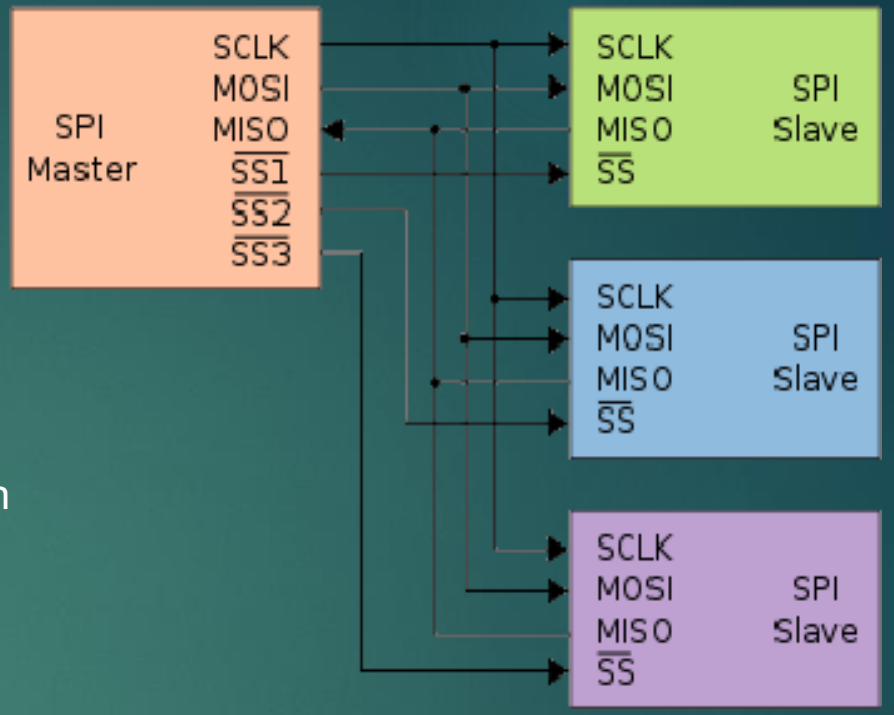
  // Print a message to the LCD.
  lcd.backlight();
  lcd.setCursor(1,0);  // Colonne 1 Ligne 0  // Laisser uniquement les lignes 0 et 1
  lcd.print("Le Blog");
  lcd.setCursor(6,1);  // Colonne 6 Ligne 1  // Supprimer les lignes 2 et 3
  lcd.print("de Claude");
}

void loop()
{
}
```

Projet Distribution automatique de l'eau d'un forage

- En utilisant le DS1307 commander l'ouverture d'une electrovanne permetttant la distribution de l'eau d'un forage tous les jours de 07h-08h puis de 16h-à 18h
- L'application permet de mettre le DS1307 à l'heure par une connexion bluetooth avec le téléphone Android.
- Chaque mois, une alerte est généré pour rappeler à l'utilisateur de mettre le système à l'heure.
- L'electrovanne est pré-actionné par un servo moteur qui déplace mécaniquement une vanne à l'aide d'un pignon.
- Réaliser un prototype de cette application.
- Utiliser la mémoire Eeprom de Arduino Uno pour stocker la date de la dernière mise à jour de l'heure
- Utiliser le module HC06 pour connecter l'interface serie d'Arduino au Bluetooth.

SPI



- La communication SPI utilise 3 fils :
  - Horloge (SCLK)
  - 2 lignes de transmission de données (une dans chaque sens MOSI et MISO)

- Elle fonctionne en full-duplex : émission et réception peuvent se faire simultanément

- La vitesse maximale est de 180 Mbits/s

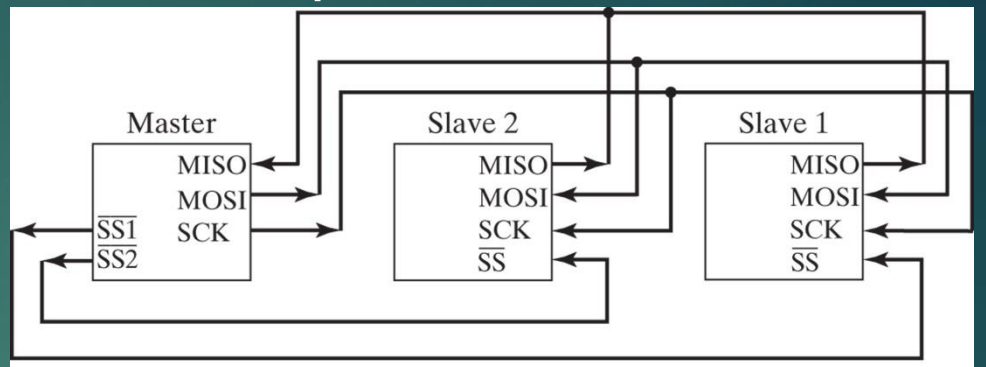
- Principe de communication :
  - Activation de la ligne correspondant au périphérique (SS)
  - Envoi / réception de données sur 8 bits en série (MOSI / MISO)

  Chaque bit est accompagné d'un changement d'état de la ligne d'horloge (SCLK)

- Il faut utiliser des lignes en sortie numérique pour désigner le périphérique (SS1, …). Et les mettre à 0 ou à 1 en fonction du périphérique désigné.

SPI interface and protocol

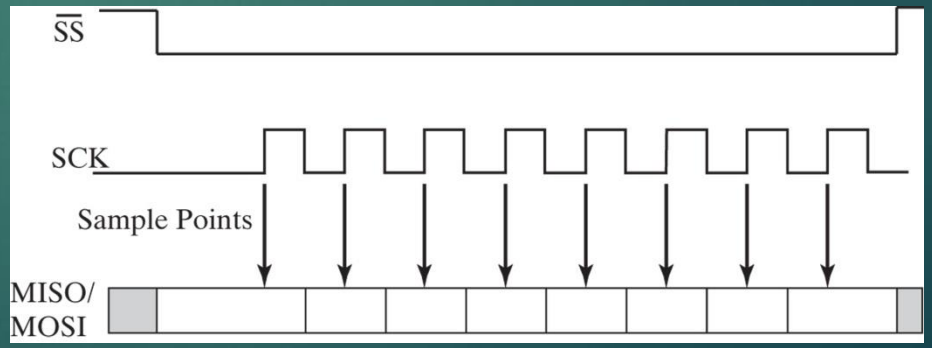
- At least one 'master' and 'slave' needed
- Master controls:
  - Unidirectional data line, MOSI
    - Master Out, Slave In (data from the master to the slave)
  - Shared clock line, SCK (synchronizes the data transfer)
  - Slave select line(s), SS*
    - Which slave to be addressed
- Slave controls:
  - Unidirectional data line, MISO
    - Master In, Slave Out (data from slave to the master)
    - Shared by slaves
    - Non-selected slaves, "tri-state" their MISO outputs
- SPI is a 'data exchange' protocol
  - As a bit is clocked out of the master and received by the slave, the slave clocks out a bit that is received by the master



Signals

Device	MISO	MOSI	SCK	SSx
Master	Input	Output	Output	Output
Slave	Output	Input	Input	Input

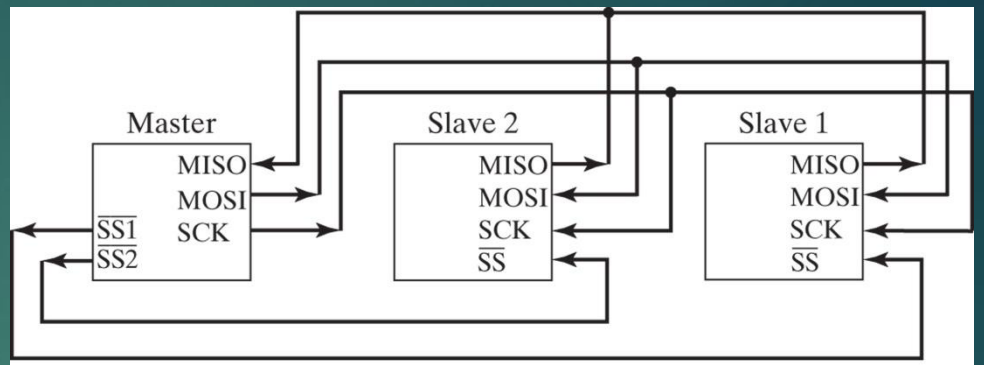
Introduction to Mechatronics, Figure 7.4 p. 109.



Introduction to Mechatronics, Figure 7.5 p. 110.

Protocole et Interface SPI

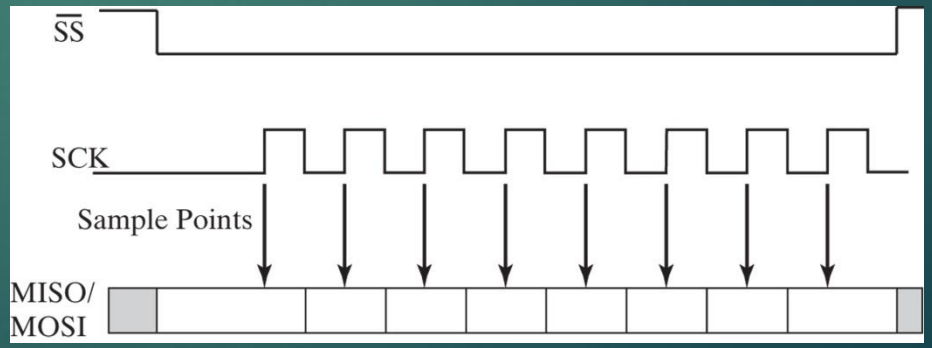
- Il faut au moins un "maître" et un "esclave".
- Le maître contrôle :
  - Ligne de données unidirectionnelle, MOSI Master Out, Slave In (données du maître à l'esclave)
  - Ligne d'horloge partagée, SCK (synchronise le transfert de données)
  - Ligne(s) de sélection de l'esclave, SS*Sélection de l'esclave à qui s'adresser
- L'esclave contrôle :
  - Ligne de données unidirectionnelle, MISO Entrée du maître, sortie de l'esclave (données de l'esclave au maître)Partagée par les esclaves
  - Les esclaves non sélectionnés, mettent leurs sorties MISO en haute impédance
  - SPI est un protocole d'échange de données
  - Lorsqu'un bit est émis par le maître et reçu par l'esclave, l'esclave émet un bit qui est reçu par le maître.



Signals

Device	MISO	MOSI	SCK	SSx
Master	Input	Output	Output	Output
Slave	Output	Input	Input	Input

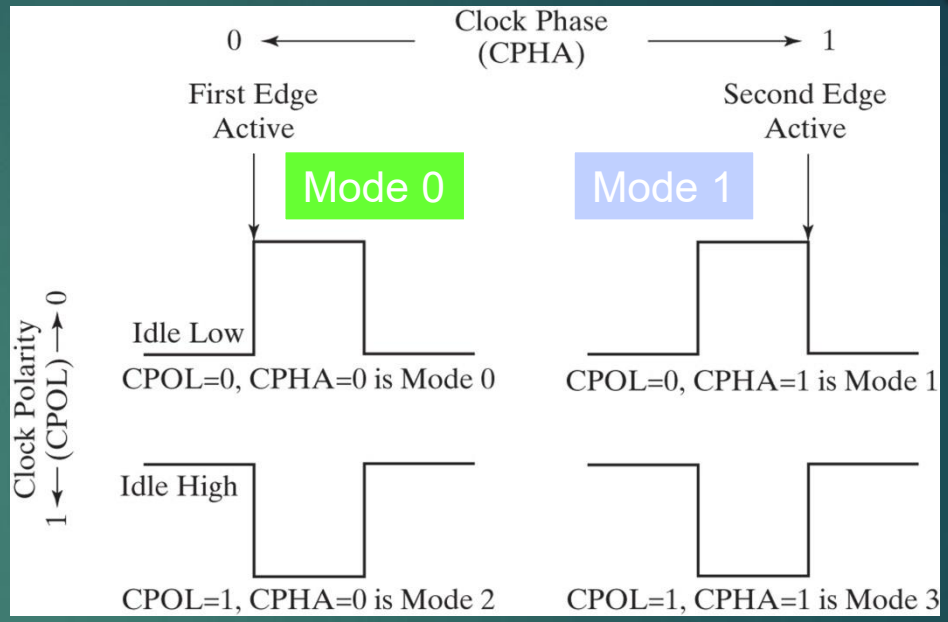
Introduction to Mechatronics, Figure 7.4 p. 109.



Introduction to Mechatronics, Figure 7.5 p. 110.

SPI data transfer modes

- Four transfer modes
  - Mode 0: SCK idle low, rising edge
  - Mode 1: SCK idle low, falling edge
  - Mode 2: SCK idle high, falling edge
  - Mode 3: SCK idle high, rising edge
    - Used most often



Introduction to Mechatronics, Figure 7.6 p. 111.

Many digital components use SPI

- See the (partial) list of SPI devices:

http://www.mct.net/faq/spi.html

Commande d'une Matrice LED par protocole SPI

- The MAX7219 LED display driver communicates with the Arduino through SPI (Serial Peripheral Interface). To learn more about this data protocol, please see this page on the Arduino website.
- With an SPI interface there is always one master device (the Arduino) that controls the peripheral devices (also known as slaves). You can control the display either through the Arduino's AVR microcontroller hardware SPI interface or three arbitrary digital pins (software SPI).
- The hardware SPI pins (MOSI, MISO, and SCK) are at a specific location on each Arduino board. This interface is faster than using software SPI, but you will need to use the following fixed output pins:

Commande d’une Matrice LED par protocole SPI

- Le pilote d'écran LED MAX7219 communique avec l'Arduino par l'intermédiaire de l'interface SPI (Serial Peripheral Interface).
- Avec une interface SPI, il y a toujours un dispositif maître (l'Arduino) qui contrôle les dispositifs périphériques (également appelés esclaves). Vous pouvez contrôler l'écran soit par l'interface SPI matérielle du microcontrôleur AVR de l'Arduino, soit par trois broches numériques arbitraires (SPI logicielle).
- Les broches SPI matérielles (MOSI, MISO et SCK) se trouvent à un emplacement spécifique sur chaque carte Arduino. Cette interface est plus rapide que l'interface SPI logicielle, mais vous devrez utiliser les broches de sortie fixes suivantes:

- Pour contrôler les écrans MAX7219, il suffit d'effectuer trois connexions :MOSI (Master Out Slave In) connecté à DIN - La ligne maître envoyant les données aux périphériques.SCK (Serial Clock) connecté à CLK - Les impulsions d'horloge qui synchronisent la transmission des données générées par le maître.SS (Slave Select) connecté à CS - La broche de chaque périphérique que le maître peut utiliser pour activer et désactiver des périphériques spécifiques.Vous pouvez relier plusieurs écrans en guirlande pour créer un grand écran en connectant DOUT du premier écran à DIN de l'écran suivant. VCC, GND, CLK et CS sont partagés par tous les écrans.Traduit avec www.DeepL.com/Translator (version gratuite)

05/11/2026

RFID (Radio-Frequency Identification)

03/11/2026

- La RFID appartient à un groupe de technologies appelées Automatic Identification and Data Capture (AIDC). Les méthodes AIDC identifient automatiquement les objets, collectent des données les saisissent directement dans des systèmes informatiques avec une intervention humaine minime. Les méthodes RFID utilisent des ondes radio pour y parvenir. À un niveau simple, les systèmes RFID se composent de trois composants: une étiquette RFID « RFID tag », **un lecteur RFID** et une antenne.
- **Les étiquettes RFID contiennent un circuit intégré et une antenne**, qui permettent de transmettre des données au lecteur RFID. Le lecteur convertit ensuite les ondes radio en une forme de données plus utilisable. Les informations collectées à partir des étiquettes sont ensuite transférées via une interface de communication vers un système informatique hôte pour les traitements ultérieurs

RFID Applications

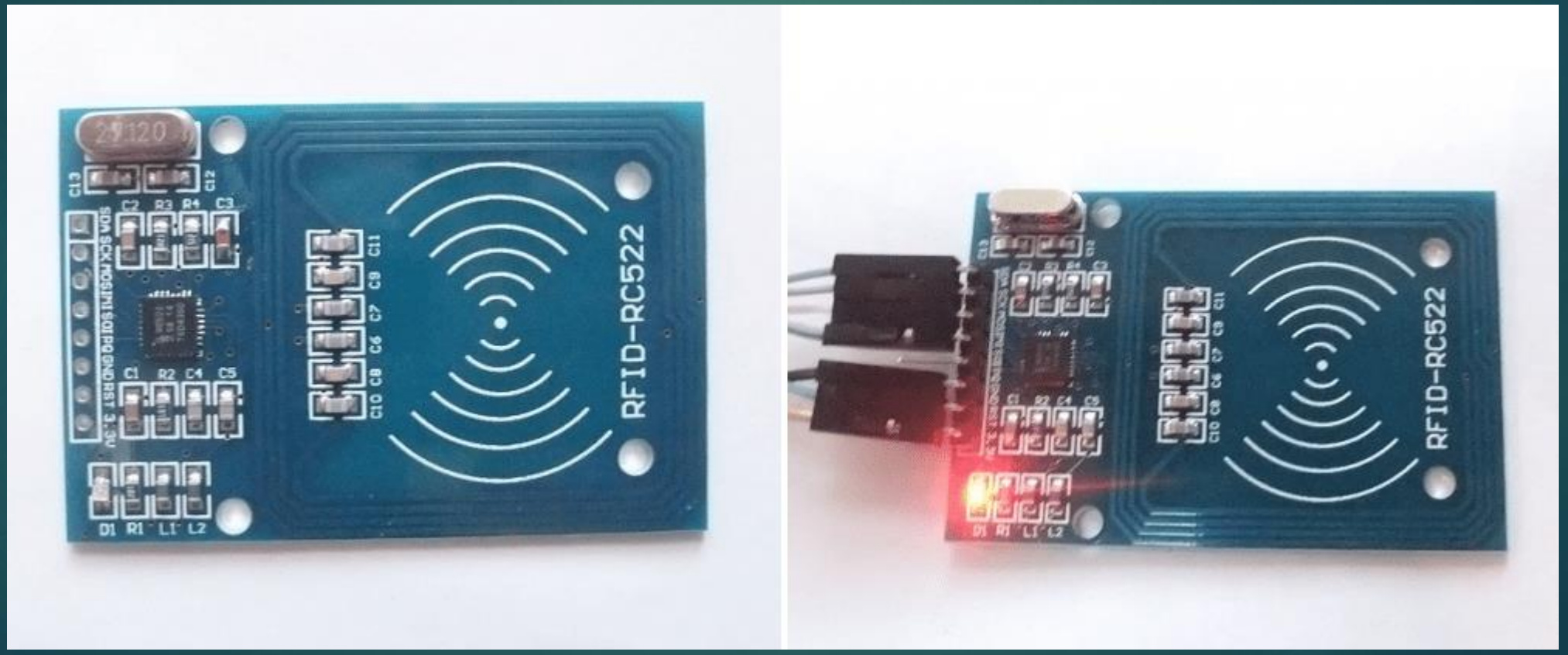
- Gestion de l'inventaire
- Suivi des actifs
- Suivi du personnel
- Contrôle de l'accès aux zones restreintes
- Badge d'identification
- Gestion de la chaîne logistique
- Prévention de la contrefaçon (par exemple dans l'industrie pharmaceutique)

Module RFID MFC522

- Le MFRC522 est un circuit intégré de lecture / écriture hautement intégré pour la communication sans contact à 13,56 MHz
- Basé sur le circuit MFRC522
- Fréquence de fonctionnement: 13,56 MHz
- Tension d'alimentation: 3.3V
- Courant: 13-26mA
- Portée de lecture: Environ 3 cm avec la carte et le porte-clés fournis
- Interface de communication : SPI
- Taux de transfert de données maximum: 10 Mbit / s
- Dimensions: 60mm × 39mm

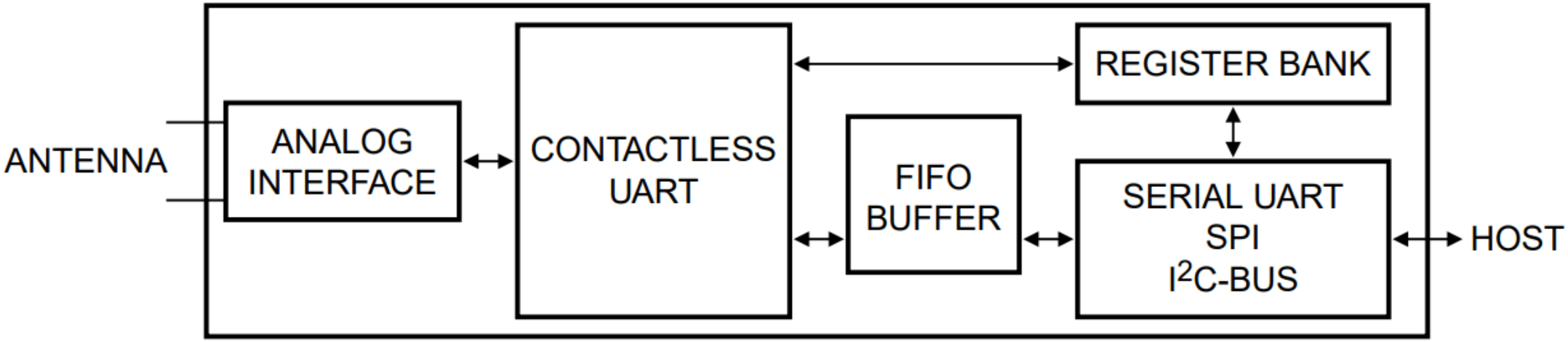
Module RFID MFC522

- Image



Module RFID MFC522

- Schéma bloc

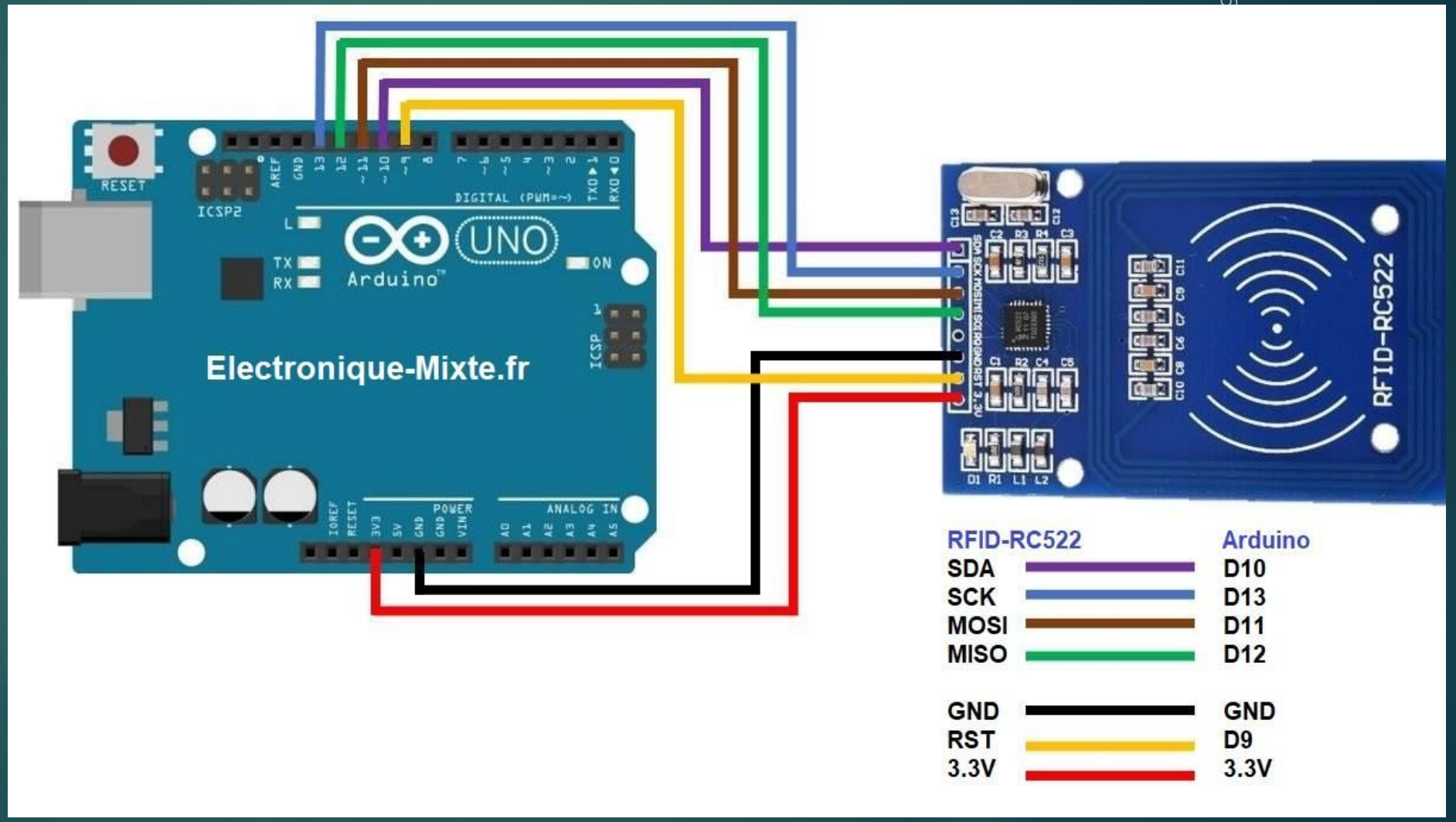


Simplified block diagram of the MFRC522

- Application

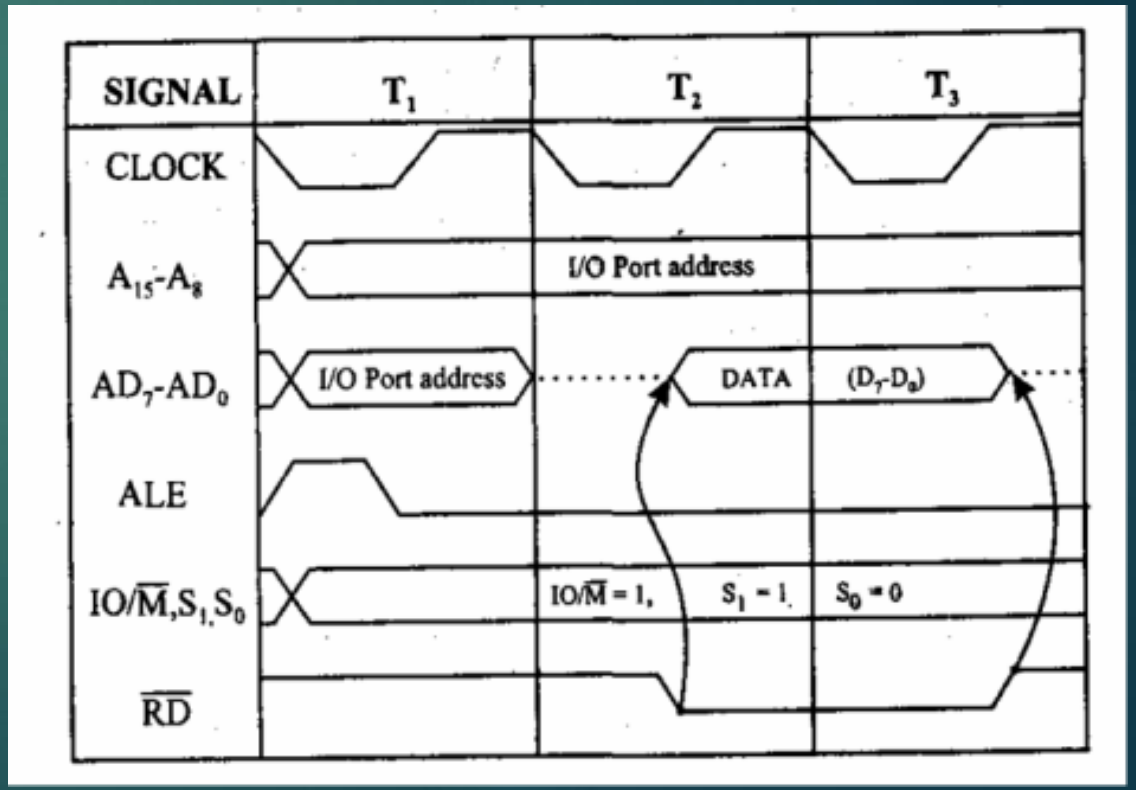
Module RFID pour l'accès sécurisé

- Montage



Gestion de temps des E/S

- L'entrée arrive à des moments aléatoires
  - Le traitement des problèmes de synchronisation est au cœur de l'informatique embarquée
    - Doit répondre à une entrée dans un délai donné
    - Doit délivrer les sorties en temps opportun
- La sortie ne doit être envoyée que lorsque le périphérique est prêt
  - Le périphérique peut ne pas voir la sortie si elle change trop rapidement
- Le processeur peut être responsable du timing
  - Interrogez les entrées à la cadence correcte (l'entrée doit être stable)
  - Activer et désactiver les sorties au bon rythme
  - Peut compliquer le programme



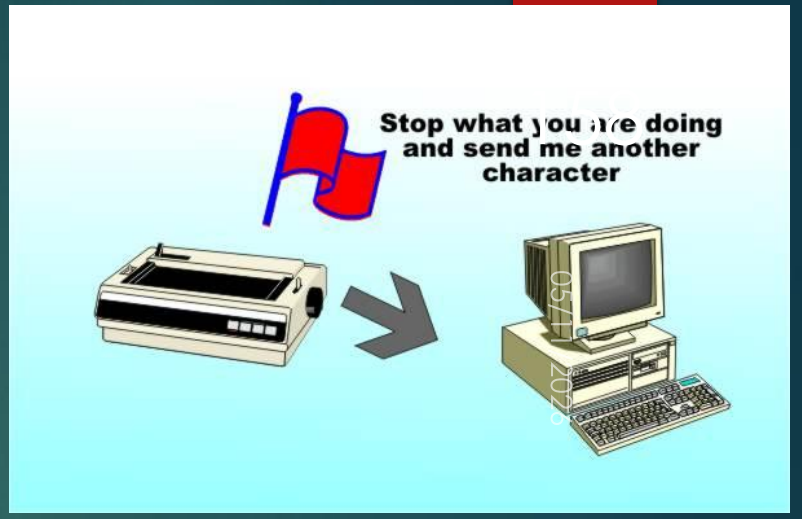
Interruptions et E/S

- Les Périphérique E / S peuvent avoir une ligne « données prête »
  - La scrutation n'est pas toujours souhaitable
  - Des temps très précis peuvent signifier une attente longue
  - Indiquer quand lire une entrée série à taux fixe
- Au lieu de cela, le périphérique d'E / S peut indiquer à la CPU quand elle est prête
  - Ceci est généralement fait en utilisant une interruption



Ghost Interrupts

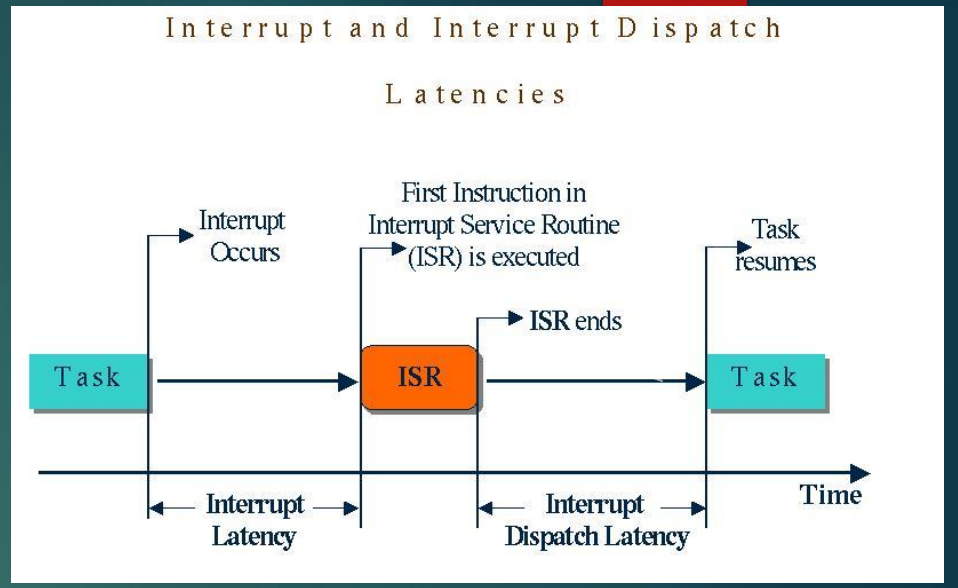
Point de vue périphérique

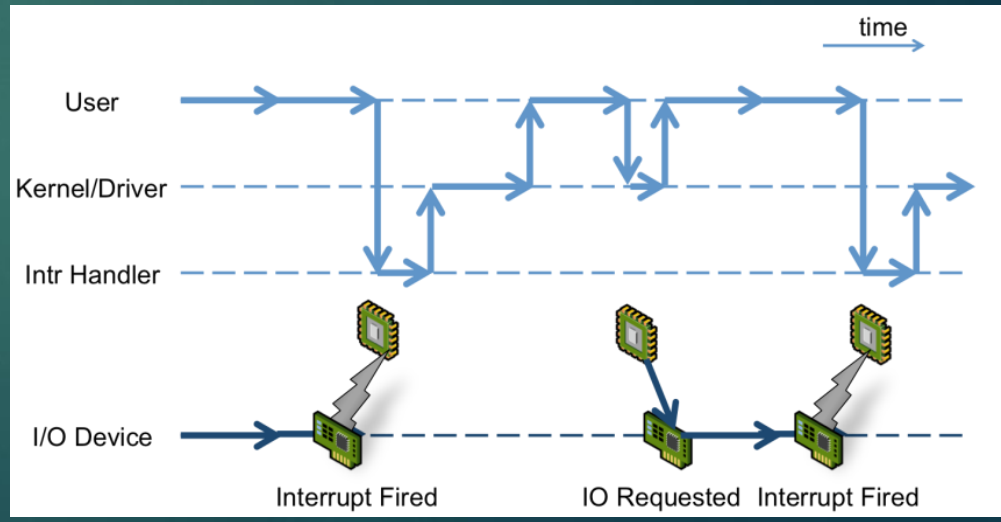


- Quand est ce qu'un appareil doit-il être interrompu?
  - Peut être évident (UART)
- Peut être flexible
  - Lorsqu'une ligne est activée (ou désactivée)
    - Ou la tension passe au dessus / en dessous d'un certain niveau
  - Conditions plus complexes
    - Quand un ensemble de lignes arrive à un certain état
    - Quand un ensemble de lignes change
  - Défini via un matériel ou un logiciel

Point de vue CPU

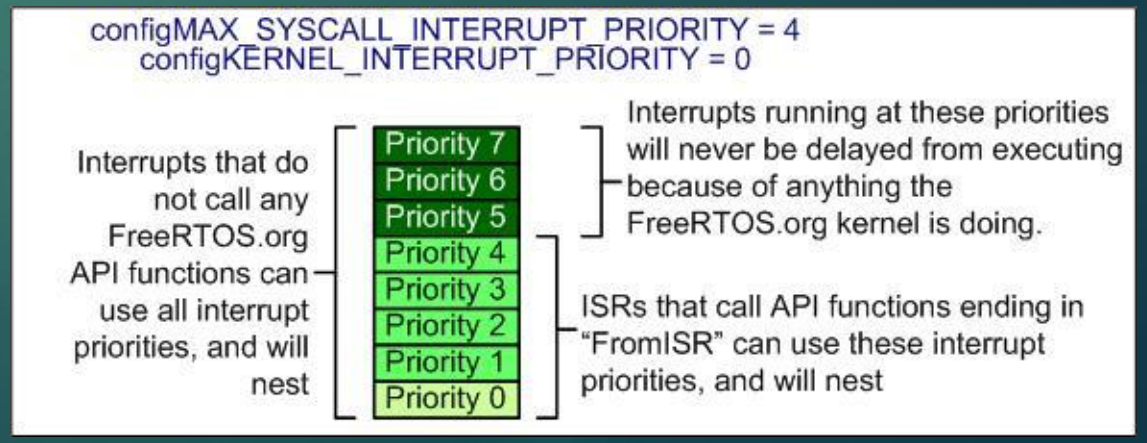
- La CPU a 2-16 lignes de demande d'interruption
  - IRQ: demande d'interruption
  - NMI: interruption non masquable
- Si un appareil définit ces lignes
  - Le processeur arrête ce qu'il fait
  - Commence à s'exécuter dans un contexte séparé à une adresse fixe





Priorité d'interruption et masquage

- Différentes lignes ont des priorités différentes
  - Une priorité plus élevée peut interrompre un traitement avec une priorité plus faible
  - La priorité plus basse attend que la priorité la plus élevée se termine
  - Peut gérer plusieurs interruptions à la fois
- Le CPU peut masquer les interruptions
  - Toutes les interruptions au-delà d'un certain niveau
  - elles attendent jusqu'à ce qu'elles soient démasquées
  - Les interruptions plus élevées masquent celles de priorité inférieure



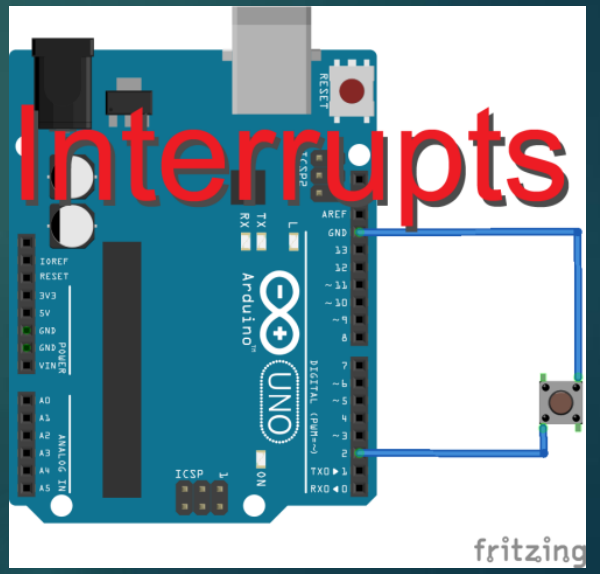
Programmation d'interruption



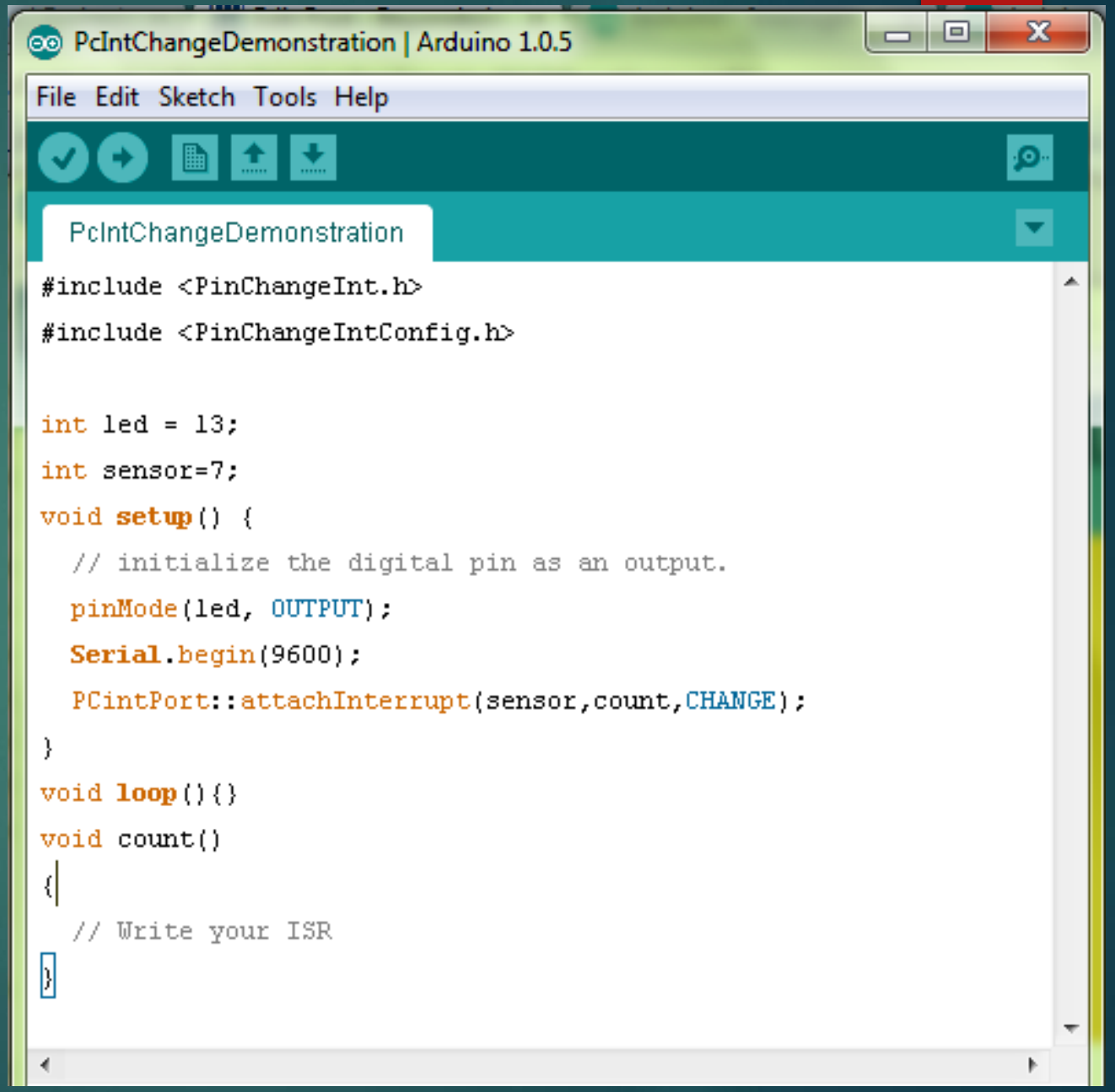
- Les interruptions introduisent le multitraitement
  - Besoin de s'inquiéter des conditions de course, de la synchronisation, de l'étreinte mortelle
- Ils doivent être codés avec précautions
  - Les données partagées peuvent être problématiques, en particulier. structures de données
  - Notez que même les entiers 16/32 bits peuvent avoir plusieurs valeurs
- Traiter comme une programmation multithread
  - Techniques de synchronisation non bloquantes
  - Régions critiques
    - Verrous: masquage d' interruptions

Arduino Interrupts

- L'utilisateur fournit une routine à appeler lorsqu'une interruption se produit
  - Pour les lignes numériques ou analogiques (de petits ensembles sont utilisables)
  - condition spécifiques sont réglables
- Les Interruptions peuvent être masquées
- Il existe des interruptions temporisées

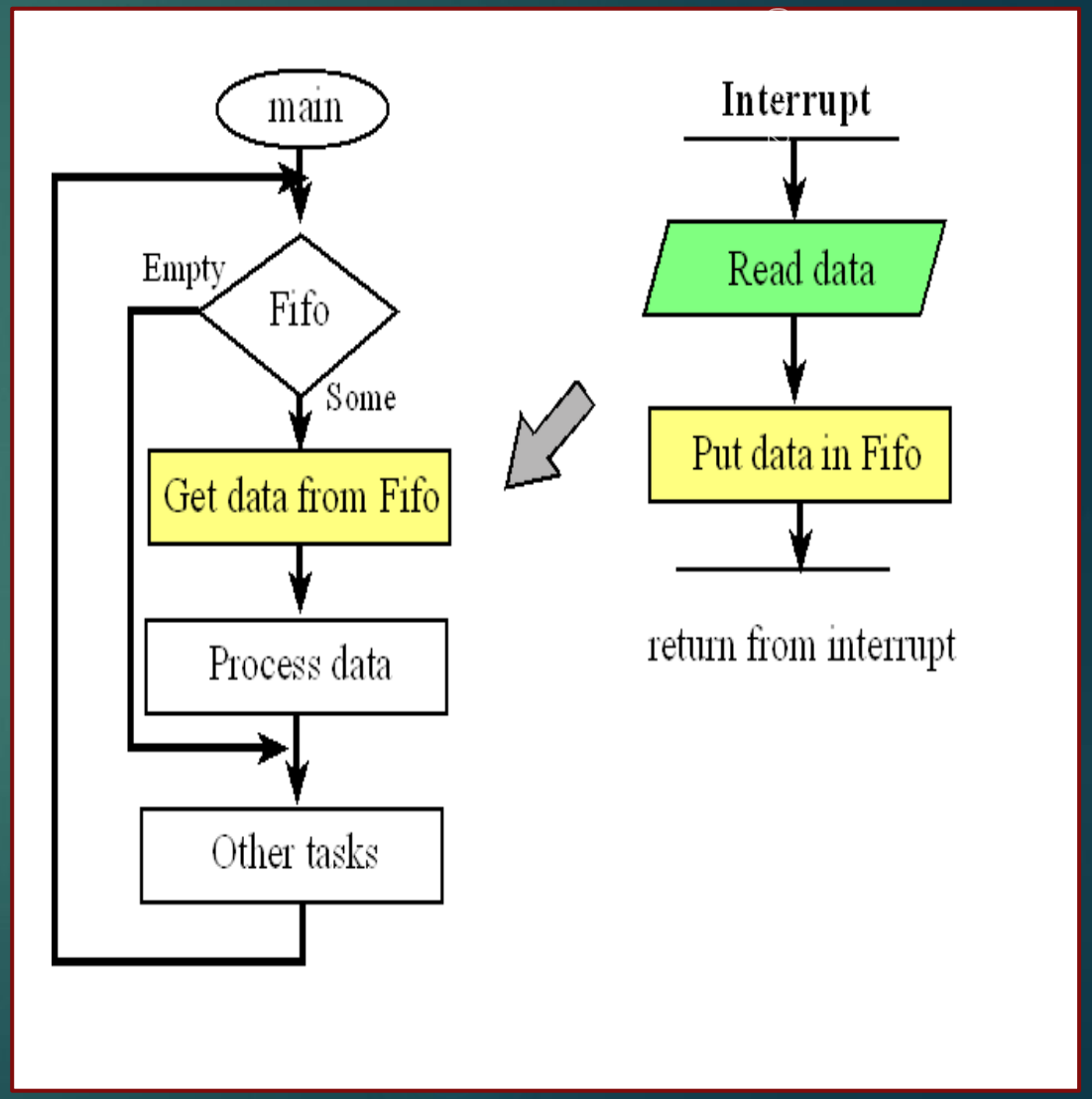


Codage Interruption Arduino (ISR)



Programmation de l'ISR

- Faire le minimum de traitement possible
- Routine d'interruption petite et rapide
  - Définir des indicateurs partagés pour un traitement ultérieur
  - Lire l'entrée et sauvegarder pour un traitement ultérieur
- Protéger l'accès à ces indicateurs



II-1 Utilisation des interruptions

- Les interruptions peuvent détecter de brèves impulsions sur les broches d'entrée. La scrutation peut manquer l'impulsion pendant que vous effectuez d'autres calculs.
- Les interruptions sont utiles pour réveiller un processeur en veille.
- Les interruptions peuvent être générées à un intervalle fixe pour un traitement répétitif.
- Et plus …

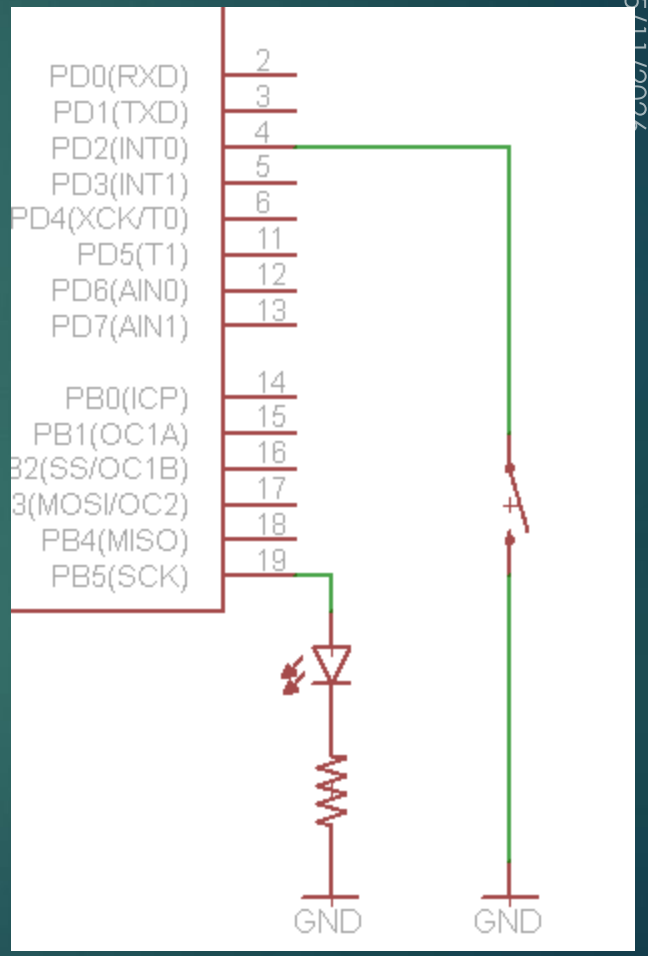
Exemple 1 (sans interruption)


```
const byte LED = 13, SW = 2;

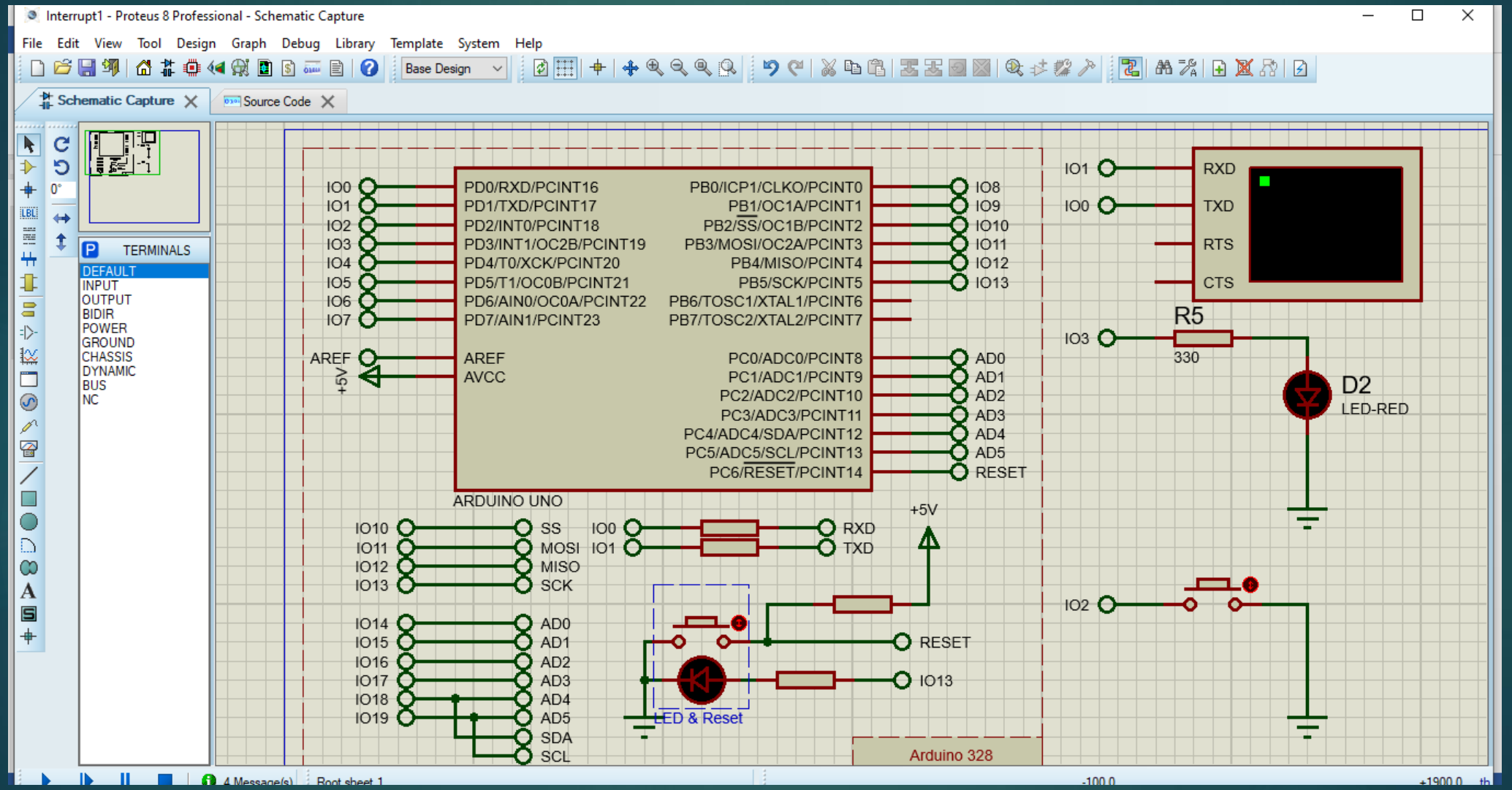
void setup() {
    pinMode(LED, OUTPUT);
    pinMode(SW, INPUT_PULLUP);
}

void handleSW() {
    digitalWrite(LED, digitalRead(SW));
}

void loop() {
    handleSW();
}
```



Montage sous Proteus



Exemple 2 (sans interruption)


```
const byte LED = 3, SW = 2;

void handleSW() {
    digitalWrite(LED, digitalRead(SW));
}

void handleOtherStuff() {
    delay(250);
}
```

```
void setup() {
    pinMode(LED, OUTPUT);
    pinMode(SW, INPUT_PULLUP);
}

void loop() {
    handleSW();
    handleOtherStuff();
}
```

Exemple 3 (avec interruption)


```
const byte LED = 13, SW = 2;

void handleSW() { // ISR
    digitalWrite(LED,
  digitalRead(SW));
}

void handleOtherStuff() {
    delay(250);
}
void setup() {
    pinMode(LED, OUTPUT);
    pinMode(SW,
  INPUT_PULLUP);
    attachInterrupt(INT0,
  handleSW, CHANGE);
}

void loop() {
    // handleSW(); ←
  commenté
    handleOtherStuff();
}
```

Exemple 3i (avec interruption)


```
const byte LED = 3, SW = 2;

void handleSW() { // ISR
int press;
press= digitalRead(SW);
If (press==1)
   attachInterrupt(INT0,
   handleSW2, FALLING);
 digitalWrite(LED,press);
}
void handleSW2() { // ISR
digitalWrite(LED,LOW);
attachInterrupt(INT0,
   handleSW, RISING);
}
void handleOtherStuff() {
    delay(250);
}

void setup() {
    pinMode(LED, OUTPUT);
    pinMode(SW,
   INPUT_PULLUP);
    attachInterrupt(INT0,
   handleSW, CHANGE);
}

void loop() {
    // handleSW(); ← commenté
    handleOtherStuff();
}
```

II-2 Traitant d'interruption (ISR)

- Les routines de traitement d'interruption doivent être courtes. Les interruptions étant désactivées lors de l'appel de l'ISR, les autres interruptions sont reportées.
- Ne pas appeler millis () ou delay () ou Serial ou…
- Cette routine est correct :

```
void myISR () {
    count++;
}
```

Retenons

- AVR ATMega328p a 2 interruptions externes
  - INT0 (sur la broche Arduino 2) et INT1 (sur la broche Arduino 3).
- Utilisez attachInterrupt (int #, nom ISR, mode) pour attacher un ISR à une interruption externe.
- Assurez-vous de fournir une définition de fonction pour l'ISR
- Choisissez le mode LOW, CHANGE, RISING, FALLING

Communication entre ISR et Programme principal

Nous voulons quelques fois savoir ce qui se passe lors de l'appel d'un ISR. Comment partageons-nous cette information?

Exemple 4


```
const byte LED = 13, SW = 2;

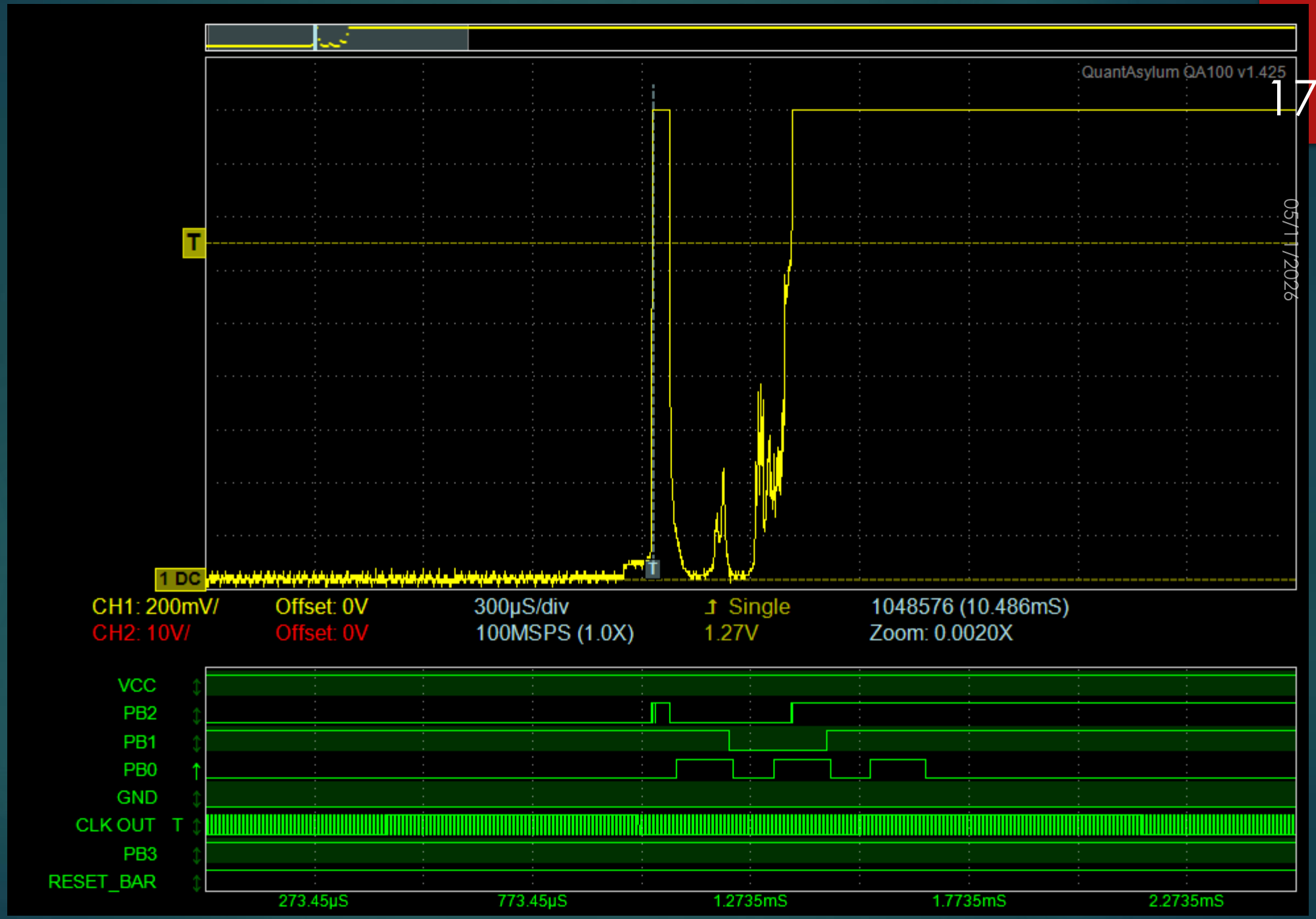
volatile unsigned char count = 0;
void handleSW () {
   digitalWrite(LED, digitalRead(SW)); count++;
}

unsigned char lastCount = -1;
void handleOtherStuff() {
    if (count != lastCount) {
        Serial.print("Count ");
        Serial.println(count);
        lastCount = count;
    }
}

void loop () {
    handleOtherStuff();
}
```

```
void setup () {
    //Start up the serial port
    Serial.begin(9600);
    Serial.println(F("Example4"));

    pinMode (LED, OUTPUT);
    pinMode (SW, INPUT_PULLUP);
    attachInterrupt(INT0,
        handleSW, CHANGE);
}
```


QuantAsylum QA100 v1.425
05/11/2026
T
1 DC
CH1: 200mV/
Offset: 0V
300µS/div
Single
1048576 (10.486mS)
CH2: 10V/
Offset: 0V
100MSPS (1.0X)
1.27V
Zoom: 0.0020X
VCC
PB2
PB1
PB0
GND
CLK OUT T
PB3
RESET_BAR
273.45µS
773.45µS
1.2735mS
1.7735mS
2.2735mS

III Le partage des données

- Une interruption peut arriver à tout moment.

- Si vous partagez une valeur multi-octets (par exemple, short int) entre un ISR et votre code, vous devez prendre des précautions supplémentaires.

```
volatile short count;
if (count == 256) …
```

```
1fa:    80 91 10 01     lds r24, 0x0110 ; count lower
1fe:    90 91 11 01     lds r25, 0x0111 ; count upper
202:    80 50           subi    r24, 0x00
204:    91 40           sbci    r25, 0x01
206:    69 f5           brne    .+90
```

Le partage des données (suite)


```
// Désactiver les interruptions et copier
noInterrupts();
short int myCount = count;
interrupts();
if (myCount == 256) …
```

```
1fa:   f8 94           cli
1fc:   80 91 10 01     lds r24, 0x0110
200:   90 91 11 01     lds r25, 0x0111
204:   78 94           sei
206:   80 50           subi    r24, 0x00
208:   91 40           sbci    r25, 0x01
20a:   69 f5           brne    .+90
```

Retenons

- Nous devons prendre des précautions lors du partage de données entre une ISR et le code principal
- Si le code principal utilise une variable qui est définie dans l'ISR, assurez-vous que cette variable est volatile
- interrupts (); et noInterrupts (); active/désactive toutes les interruptions
- detachInterrupt (int #) est également disponible pour désactiver une ISR

Interruption lors du changement sur broche

- La broche 2 pour INT0
- La broche 3 pour INT1
- Que se passe t'il pour les broches 0,1,4,5,6,…
- Les interruptions lors d'un changement sur une broche peuvent survenir sur toutes les broches

Pin change interrupts

- Sur les cartes Arduino Uno, ces interruptions peuvent être activées depuis n'importe quel broche, et même la totalité des 20 broches de l'Arduino (A0 à A5 et D0 à D13). Elles sont regroupées sous trois vecteurs d'interruption PCINT0, PCINT1 et PCINT2 pour l'intégralité des 20 broches.

Pin change interrupts

- Pin Change Interrupt Request 0 (connecteurs D8 à D13) (PCINT0_vect)
- Pin Change Interrupt Request 1 (connecteurs A0 à A5) (PCINT1_vect)
- Pin Change Interrupt Request 2 (connecteurs D0 à D7) (PCINT2_vect)
- Utiliser la librairie PinChangeInt.h

Exemple 5 avec PinChangeInt.h


```
const byte LED = 13, SW =  ;

volatile unsigned char count = 0;
void handleSW () {
   digitalWrite(LED, digitalRead(SW));
   count++;
}
unsigned char lastCount = -1;
void handleOtherStuff() {
    if (count != lastCount) {
        Serial.print("Count ");
        Serial.println(count);
        lastCount = count;
    }
}
```

```
void loop () {
    handleOtherStuff();
}
void setup () {
    //Start up the serial port
    Serial.begin(9600);
    Serial.println(F("Example4"));

    pinMode (LED, OUTPUT);
    pinMode (SW, INPUT_PULLUP);
    attachPCINT(digitalPinToPCINT(SW), handleSW,
   CHANGE);
}
```

attachPCINT(digitalPinToPCINT(SW), handleSW, CHANGE);

Sans Librairie

- La routine d'interruption devra être définie avec la macro ISR et le nom du vecteur d'interruption souhaité, comme ISR(PCINT0_vect) ISR(PCINT0_vect) .
- les macros sei() et cli() pour activer et désactiver les interruptions

Nom des sources d'interruption dans un programme Arduino (pré-définis)

- #define INT0_vect _VECTOR(1) /* External Interrupt Request 0 */
- #define INT1_vect _VECTOR(2) /* External Interrupt Request 1 */
- #define PCINT0_vect _VECTOR(3) /* Pin Change Interrupt Request 0 */
- #define PCINT1_vect _VECTOR(4) /* Pin Change Interrupt Request 1 */
- #define PCINT2_vect _VECTOR(5) /* Pin Change Interrupt Request 2 */
- #define WDT_vect _VECTOR(6) /* Watchdog Time-out Interrupt */
- #define TIMER2_COMPA_vect _VECTOR(7) /* Timer/Counter2 Compare Match A */
- #define TIMER2_COMPB_vect _VECTOR(8) /* Timer/Counter2 Compare Match A */
- #define TIMER2_OVF_vect _VECTOR(9) /* Timer/Counter2 Overflow */
- #define TIMER1_CAPT_vect _VECTOR(10) /* Timer/Counter1 Capture Event */ #define TIMER1_COMPA_vect _VECTOR(11) /* Timer/Counter1 Compare Match A */
- #define TIMER1_COMPB_vect _VECTOR(12) /* Timer/Counter1 Compare Match B */

Nom des sources d'interruption dans un programme Arduino (pré-définis)

- #define TIMER1_COMPB_vect _VECTOR(12) /* Timer/Counter1 Compare Match B */
- #define TIMER1_OVF_vect _VECTOR(13) /* Timer/Counter1 Overflow */
- #define TIMER0_COMPA_vect _VECTOR(14) /* TimerCounter0 Compare Match A */
- #define TIMER0_COMPB_vect _VECTOR(15) /* TimerCounter0 Compare Match B */
- #define TIMER0_OVF_vect _VECTOR(16) /* Timer/Couner0 Overflow */
- #define SPI_STC_vect _VECTOR(17) /* SPI Serial Transfer Complete */
- #define USART_RX_vect _VECTOR(18) /* USART Rx Complete */
- #define USART_UDRE_vect _VECTOR(19) /* USART, Data Register Empty */
- #define USART_TX_vect _VECTOR(20) /* USART Tx Complete */
- #define ADC_vect _VECTOR(21) /* ADC Conversion Complete */
- #define EE_READY_vect _VECTOR(22) /* EEPROM Ready */
- #define ANALOG_COMP_vect _VECTOR(23) /* Analog Comparator */
- #define TWI_vect _VECTOR(24) /* Two-wire Serial Interface */

Retenons

- Nous pouvons surveiller n'importe quelle broche et la faire générer une interruption
- Différentes broches peuvent avoir différents ISR

Exemple 6


```
#include <avr/sleep.h>
#include <PinChangeInterrupt.h>

void wake() { // ISR
    sleep_disable(); // first thing after waking from sleep:
    detachPCINT(digitalPinToPCINT(SW)); // stop LOW interrupt
}

void sleepNow()
{
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    noInterrupts(); // stop interrupts
    sleep_enable(); // enables sleep bit in MCUCR
     attachPCINT(digitalPinToPCINT(SW), wake, LOW);
    interrupts(); // allow interrupts
    sleep_cpu(); // here the device is put to sleep
}
```

Exercice

- Proposer un programme entièrement fonctionnel utilisant les interruptions.
- Tandis qu'il est éveillé, le voyant LED clignote à environ 2 Hz. Comme auparavant, handleSW est l'ISR pour une interruption.
- Lorsque nous appuyons sur le bouton poussoir, la variable count est incrémenté. Dans la fonction loop() lorsque count> 10 nous appelons sleepNow ().

Exercie correction

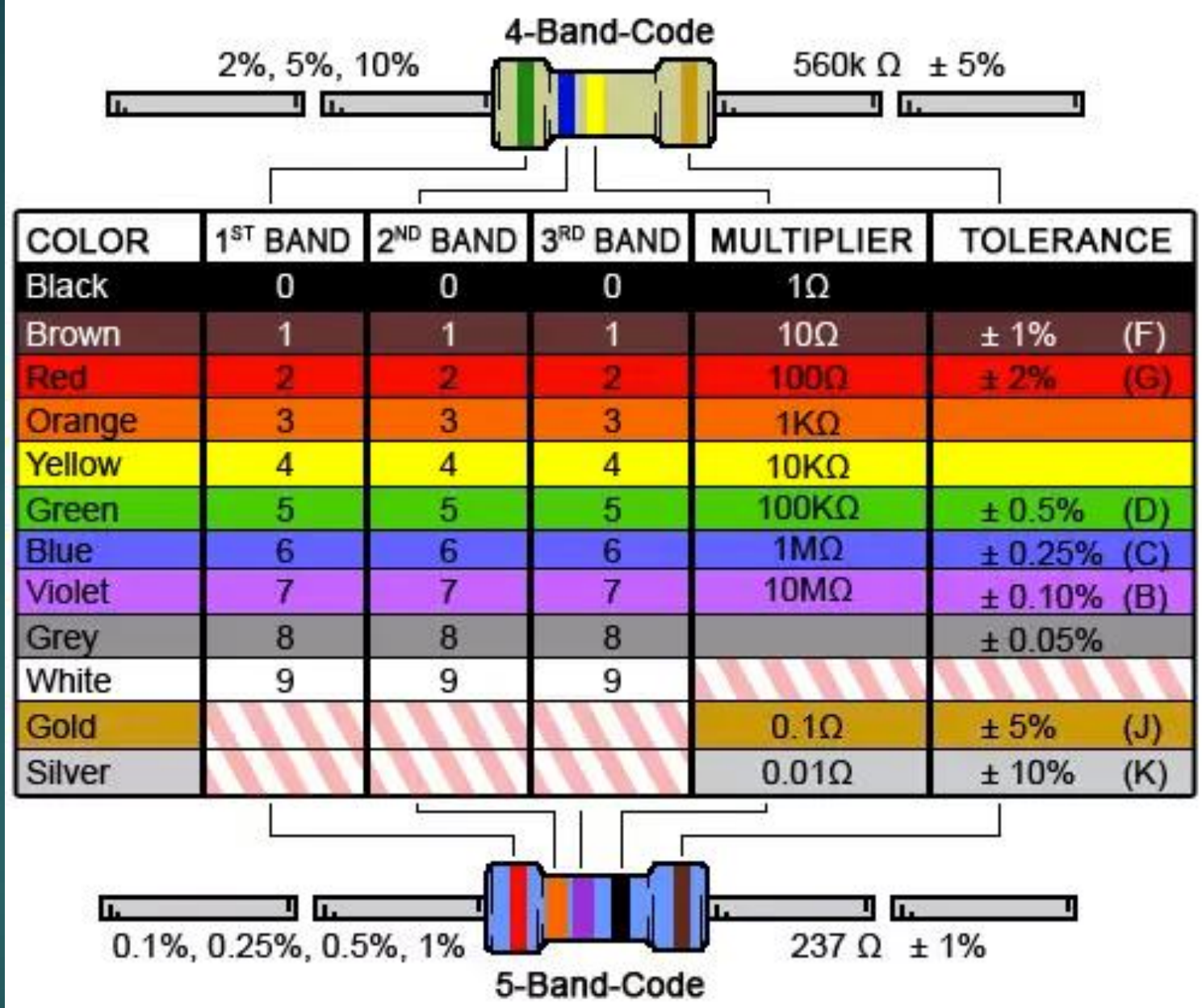

```
#include <avr/sleep.h>
 #include <PinChangeInterrupt.h>
const byte LED = 13, SW = 5;
volatile unsigned char count = 0;
 void wake() { // ISR    sleep_disable(); //
first thing after waking from sleep:
detachPCINT(digitalPinToPCINT(SW)); //
stop LOW interrupt }
void sleepNow()
{    set_sleep_mode(SLEEP_MODE_PWR
_DOWN);
noInterrupts(); // stop
interrupts    sleep_enable(); // enables
sleep bit in MCUCR
attachPCINT(digitalPinToPCINT(SW),
wake, LOW);    interrupts(); // allow
interrupts
sleep_cpu(); // here the device is put to
sleep }
void handleSW () {   //digitalWrite(LED,
digitalRead(SW));   count++; }
unsigned char lastCount = -1;
void handleOtherStuff() {   noInterrupts();
int mycount=count;
interrupts();
 if (count != lastCount)
{       Serial.print("Count ");
 Serial.println(count);       lastCount =
count;
 if
(mycount>=10){sleepNow();count=0;}    } }
void setup () {    //Start up the serial port
 Serial.begin(9600);
Serial.println("Example4");
 pinMode (LED, OUTPUT);
 pinMode (SW, INPUT_PULLUP);
attachPCINT(digitalPinToPCINT(SW),
handleSW, CHANGE); }
void loop ()
{    handleOtherStuff();    digitalWrite(LED,
HIGH);
delay(1000);    digitalWrite(LED,LOW);
delay(1000); }
```

Mesure du temps avec Arduino

- Les fonctions natives pour la mesure du temps
- Les interruptions Timer

Pour la suite

- Réaliser le mini projet



COLOR	1ST BAND	2ND BAND	3RD BAND	MULTIPLIER	TOLERANCE
Black	0	0	0	1Ω	
Brown	1	1	1	10Ω	± 1% (F)
Red	2	2	2	100Ω	± 2% (G)
Orange	3	3	3	1KΩ	
Yellow	4	4	4	10KΩ	
Green	5	5	5	100KΩ	± 0.5% (D)
Blue	6	6	6	1MΩ	± 0.25% (C)
Violet	7	7	7	10MΩ	± 0.10% (B)
Grey	8	8	8		± 0.05%
White	9	9	9		
Gold				0.1Ω	± 5% (J)
Silver				0.01Ω	± 10% (K)